Internship Report

Module : Stage INE1

Major: Advanced Software Engineering for Digital Services (ASEDS)

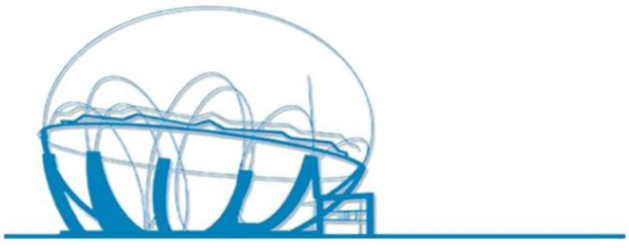
Design and Development of an AI-Powered Recruitment Platform Built on SAP BTP using ABAP RAP

Authored by :

Oualid AMTOUT

Supervised by :

Mr. Karim Ait Alla Cognitus - Encadrant



Agence Nationale de Réglementation des Télécommunications

Institut National des Postes et Télécommunications

Academic Year : 2025/2026

Dedication

To my beloved parents,

Whose unconditional love, continuous sacrifices, and prayers have guided my path and inspired me to strive for excellence. Words cannot express my deepest gratitude for everything you have given me.

To my family, brothers, and sisters,

For your unwavering support, continuous encouragement, and belief in my potential throughout my academic journey.

To my mentors, professors, and dear friends,

Whose valuable insights, camaraderie, and encouragement have enriched both my personal and professional growth.

Oualid AMTOUT

Acknowledgments

First and foremost, I would like to express my deepest gratitude to Almighty God for granting me the strength, perseverance, and health to successfully complete this internship and realize this project.

I would like to convey my sincere and profound gratitude to **Cognitus Corporation** for welcoming me into their team in Tangier and providing an intellectually stimulating, collaborative, and professional environment throughout my summer internship.

I extend my heartfelt gratitude to my industrial supervisor, **Mr. Karim Aït Alla**, Technical Lead at Cognitus, for the invaluable guidance, technical oversight, and regular follow-ups provided throughout the various stages of this project. The high standards and pragmatic feedback I received were instrumental in steering this work toward professional enterprise standards.

I also express my sincere thanks to the members of the Cognitus team in Tangier for their time, availability, and generous support. They introduced me to the intricate and powerful world of enterprise ERP systems and pointed me to training tracks and learning resources that proved decisive in expanding my technical horizon. Thanks to their encouragement and constructive advice, I was able to rapidly acquire modern SAP development practices and to design and implement this solution.

I also wish to thank the faculty of the **Advanced Software Engineering for Digital Services (ASEDS)** major at the **National Institute of Posts and Telecommunications (INPT)** for their high-quality pedagogy and continuous dedication to training skilled and rigorous engineers.

Finally, I would like to extend my sincere appreciation to the honorable members of the jury for taking the time to evaluate this report and assess the fruits of this work.

Résumé

Dans le paysage économique actuel, les entreprises modernes privilégient l'adoption d'un progiciel de gestion intégré (ERP) unique capable d'unifier et d'orchestrer l'ensemble de leurs fonctions opérationnelles, en particulier la gestion du capital humain (HCM). Conjointement, les avancées fulgurantes de l'intelligence artificielle (IA) et sa capacité d'analyse et de raisonnement sur des milliers de documents complexes ouvrent des perspectives inédites pour transformer les flux de recrutement et optimiser l'identification des meilleurs talents.

Réalisé au sein de Cognitus Corporation, ce rapport rend compte de mon stage d'initiation ingénieur effectué dans le cadre de la filière Advanced Software Engineering for Digital Services (ASEDS) à l'Institut National des Postes et Télécommunications (INPT). Constituant ma toute première immersion dans l'écosystème des systèmes d'information d'entreprise, cette expérience a débuté par une phase intensive de montée en compétences sur les technologies SAP modernes, fondée sur les cours officiels de SAP Learning : Exploring SAP Business Technology Platform, Learning Basic ABAP Programming et Building Transactional Apps with the ABAP RESTful Application Programming Model. Fort de ces acquis fondamentaux, j'ai entrepris le développement d'un projet applicatif complet selon les standards actuels de SAP.

La solution conçue fait le pont entre le recrutement externe et la gouvernance d'entreprise à travers une architecture à double facette : un portail web moderne et interactif dédié aux candidats, interconnecté à un environnement d'entreprise **SAP**. Ce socle permet aux gestionnaires RH de modéliser la structure organisationnelle, de publier des offres directement rattachées à des vacances de postes, et de piloter les candidatures via des pipelines d'évaluation structurés. Alimentée par un moteur d'IA pour le parsing sémantique des CV et développée avec **SAP ABAP RAP**, la plateforme assure une continuité fluide et automatisée depuis la présélection des profils jusqu'à la création de la fiche employé dans SAP.

Mots clés : ERP, Gestion du Capital Humain (HCM), SAP BTP, ABAP RAP, Clean Core, Intelligence Artificielle, Recrutement Intelligent, Structure Organisationnelle, React.

Summary

In today's economic landscape, modern enterprises prioritize adopting a single Enterprise Resource Planning (ERP) system capable of unifying and orchestrating all operational functions, particularly Human Capital Management (HCM). Simultaneously, the remarkable advances in Artificial Intelligence (AI) and its ability to analyze and reason over thousands of complex documents open unprecedented avenues to transform recruitment workflows and optimize the identification of top talent.

Conducted at Cognitus Corporation, this report reflects my introductory engineering internship completed within the Advanced Software Engineering for Digital Services (ASEDS) major at the National Institute of Posts and Telecommunications (INPT). Representing my very first immersion into the enterprise information systems ecosystem, this experience began with an intensive upskilling phase on modern SAP technologies, based on the official SAP Learning courses: Exploring SAP Business Technology Platform, Learning Basic ABAP Programming, and Building Transactional Apps with the ABAP RESTful Application Programming Model. Building upon these fundamental skills, I undertook the development of a full-scale application following current SAP standards.

The resulting solution bridges external recruitment and corporate governance through a dual-faceted architecture: a modern and interactive web portal dedicated to candidates, interconnected with an enterprise **SAP** environment. This foundation enables HR managers to model organizational structure, publish job offers directly mapped to vacancies, and manage applications through structured evaluation pipelines. Powered by an AI engine for semantic resume parsing and built with **SAP ABAP RAP**, the platform ensures a smooth, automated continuum from candidate pre-selection to employee file creation in SAP.

Key Words : ERP, Human Capital Management (HCM), SAP BTP, ABAP RAP, Clean Core, Artificial Intelligence, Smart Recruitment, Organizational Structure, React.

ملخص

في المشهد الاقتصادي الحالي، تفضل الشركات الحديثة اعتماد نظام ERP موحد قادر على إدارة وتنسيق كافة وظائفها التشغيلية، ولا سيما إدارة الرأسمال البشري (HCM). وفي الوقت ذاته، تفتح التطورات المتسارعة للذكاء الاصطناعي (AI) وقدرته على تحليل آلاف الوثائق المعقدة آفاقا جديدة لتطوير عمليات التوظيف وانتقاء أفضل الكفاءات.

يقدم هذا التقرير، المنجز بشركة Cognitus Corporation ، حصيلة تدريب المهندس ضمن مسلك ASEDS بالمعهد الوطني للبريد والمواصلات (INPT). وباعتباره أول تجربة لي في أنظمة المؤسسات، بدأت هذه المرحلة بتكوين مكثف في تقنيات SAP الحديثة من خلال دورات رسمية في منصة BTP وبرمجة ABAP ونموذج RAP ، مما مكنني من تطوير مشروع تطبيقي متكامل وفق معايير SAP الحالية.

يربط النظام المطور بين التوظيف الخارجي وإدارة المؤسسة عبر بنية مزدوجة: بوابة Web حديثة للمترشحين، متصلة بنظام SAP الداخلي. وتتيح هذه المنظومة للمسؤولين نمذجة الهيكل التنظيمي، ونشر عروض العمل، وتتبع المترشحين عبر مراحل تقييم محددة. وبفضل محرك الذكاء الاصطناعي لتحليل السير الذاتية وتطويره باستخدام SAP ABAP RAP ، يضمن النظام مسارا سلسا ومؤتمتا من الانتقاء الأولي حتى تسجيل الموظف في SAP .

الكلمات المفتاحية : نظام ERP ، إدارة الرأسمال البشري (HCM)، SAP BTP ، ABAP RAP ، الذكاء الاصطناعي، التوظيف الذكي، الهيكل التنظيمي، React .

List of Figures

1.1 Cognitus Corporation Logo . . . 2
1.2 Cognitus Corporation — Global Footprint . . . 3
1.3 Cognitus Tangier — Technopark, Avenue Mohammed V . . . 3
1.4 ERP System — Integrated Business Module Architecture . . . 6
1.5 ABAP RAP — Layered Architecture . . . 7
1.6 SAP Learning — Course Completion Badges (BTP100 and S4D400) . . . 12
1.7 SAP Learning — List of Completed Courses . . . 13
1.8 The Scrum Framework — Iterative Development Cycle . . . 14
1.9 Sprint Plan — Product Backlog Distributed over Four Two-Week Sprints . . . 15
1.10 Project Charter — Delivery Strategy Section . . . 16
1.11 Software Development Plan — Work Breakdown Structure . . . 17
1.12 Project Planning — Detailed Gantt Diagram . . . 20

3.1 Global System Use Case Diagram . . . 32

4.1 Global System Architecture of the Talently Platform . . . 34
4.2 ABAP RAP Backend – Four-Layer Architecture . . . 36
4.3 Sequence Diagram – Job Offer Management (Create, Modify, Delete) . . . 39
4.4 AI Evaluation Microservice – Resume Scoring Pipeline . . . 41
4.5 Sequence Diagram – Candidate Application and AI Evaluation . . . 43
4.6 Application Routing Tree of the Candidate Portal and the HR Cockpit . . . 44
4.7 Mockup of the Job Board of the Candidate Portal . . . 45
4.8 Mockup of the Job Offers List of the HR Cockpit . . . 46
4.9 Class Diagram of the Talently Data Model (CDS Interface Views) . . . 47

5.1 SAP Business Technology Platform Logo . . . 50
5.2 SAP HANA Cloud Logo . . . 50

5.3 Eclipse and ABAP Development Tools Logo . . . 51
5.4 SAP Fiori Logo . . . 51
5.5 Vite Logo . . . 52
5.6 Tailwind CSS Logo . . . 52
5.7 Clerk Logo . . . 52
5.8 FastAPI Logo . . . 53
5.9 OpenAI Logo . . . 53
5.10 PyMuPDF Logo . . . 53
5.11 Docker Logo . . . 54
5.12 GitHub Logo . . . 54
5.13 Creation of an Organizational Unit in Draft Mode . . . 56
5.14 Organizational Units List After the First Creation . . . 56
5.15 List of the Three Organizational Units . . . 57
5.16 Value Help of the Parent Unit . . . 57
5.17 Validation Error on the Parent Unit . . . 58
5.18 Data & AI Unit in Display Mode . . . 58
5.19 Positions Section of SAP Development in Edit Mode . . . 59
5.20 Details of a Position . . . 59
5.21 First Position in the Positions Table . . . 60
5.22 Positions of the SAP Development Unit . . . 60
5.23 Deletion Request of an Organizational Unit . . . 61
5.24 Deletion Refused for a Unit That Still Has Positions . . . 61
5.25 Job Offers List Report . . . 62
5.26 Job Offer in Draft Mode . . . 62
5.27 Details of a Pipeline Step . . . 63
5.28 Pipeline Steps of the Job Offer . . . 63
5.29 Job Board of the Candidate Portal . . . 64
5.30 Job Board for a Visitor Who Is Not Signed In . . . 65
5.31 Candidate Sign-In with Clerk . . . 65
5.32 Job Details Page . . . 66
5.33 Smart Apply Form . . . 66
5.34 Application Confirmation on the Portal . . . 67
5.35 Published Job Offer with Its Applications . . . 68

5.36 Selection of Applications and AI Evaluation Request 68
5.37 Applications with Their AI Scores 69
5.38 Application Details in the HR Cockpit 69
5.39 Moving a Candidate to the Next Step 70
5.40 Candidate Progress on the Portal 71
5.41 Last Step of the Pipeline with Accept Candidate Enabled 71
5.42 Accept Candidate Dialog 72
5.43 Rejection of a Candidate 72
5.44 Talent Pool 73
5.45 Accepted Application on the Portal 73
5.46 Onboarding Form of the Candidate Portal 74
5.47 Submitted Onboarding Form in the HR Cockpit 74
5.48 Employees List After the Hiring 75
5.49 Employee Object Page and Promote Dialog 75
5.50 Execution of the Microservice Unit Tests (pytest) 77
5.51 Execution of the RAP Unit Tests (ABAP Unit) 77

List of Tables

1.1 Internship Agreement Administrative Specifications (*Convention de stage*) 9
1.2 Main Project Risks and Mitigation Measures 18
1.3 Implementation Schedule — Phases, Timeframes, and Key Milestones 19

3.1 Comparative Analysis — Current Methods vs. Our Proposed Solution 28

4.1 Business Objects of the Talently Backend 37
4.2 Main Business Actions of the Backend 38
4.3 Evaluation Rubric of the Software Engineering Intern Role 42

5.1 Unit Test Suites and Results 76
5.2 Evaluation of the AI Engine 78
5.3 Validation of the Functional Requirements 79
5.4 Validation of the Non-Functional Requirements 80
5.5 Measured Response Times 80

Listings

C.1 Behavior definition of the job offer (excerpt) 90
C.2 Validation `validateBeforePublish` of the job offer 91
C.3 Deterministic score computation (`services/scoring_engine.py`) 92
C.4 Start of the text sent to the scoring model (`data/transform.py`) 93

Contents

Dedication i

Acknowledgments ii

Résumé iii

Summary iv

Arabic Abstract v

List of Figures viii

List of Tables ix

List of Listings x

Table of Contents xiv

1 General Project Context 1

Introduction . . . 1

1.1 Cognitus Corporation . . . 2

1.1.1 About Cognitus Corporation . . . 2

1.1.2 Cognitus Tangier — Global Delivery and Support Center . . . 3

1.1.3 Business Area of Activity . . . 4

1.1.4 Enterprise Resource Planning (ERP) . . . 5

1.1.4.1 What is an ERP System? . . . 5

1.1.5 SAP — Systems, Applications, and Products . . . 7

1.1.5.1 SAP S/4HANA . . . 7

1.1.5.2 SAP Business Technology Platform (BTP) . . . 7

1.1.5.3 ABAP and the RESTful Application Programming Model (RAP) 7
1.1.6 Internship Overview 8
1.1.6.1 Internship Framework 8
1.1.6.2 Project Objectives (*Objectifs du projet de stage*) 9
1.1.6.3 Project Planning and Management 10
1.1.6.3.1 A. Formation et Montée en Compétences — Technical Training & Upskilling 11
1.1.6.3.2 B. Conduite de Projet — The Scrum Framework 13
1.1.6.3.3 C. Gouvernance du Projet — Formal Governance Artifacts 15
1.1.6.3.4 D. Planning d'Exécution — Implementation Schedule & Gantt Diagram 18
Conclusion 21

2 Project Context and Objectives 22
Introduction 22
2.1 General Project 23
2.1.1 General Context 23
2.1.2 Problem Statement 23
2.1.3 Project Objectives 24
Conclusion 25

3 Analysis and Specifications 26
Introduction 26
3.1 Existing System Study 27
3.1.1 Current Recruitment Methods 27
3.1.2 Particularities of Our Solution 27
3.1.3 Comparative Synthesis 28
3.2 Requirements Identification and Analysis 29
3.2.1 Feature Identification and System Actors 29
3.2.1.1 Primary Actors 29
3.2.1.2 Secondary Actors 29
3.2.2 Functional Requirements 29
3.2.3 Non-Functional Requirements 31

3.2.4 Global Use Case Diagram . . . 32
3.2.4.1 User Stories . . . 32
Conclusion . . . 32

4 Solution Design and Architecture 33
Introduction . . . 33
4.1 Application Architecture . . . 34
4.1.1 Global System Architecture . . . 34
4.1.2 Simplified Backend Architecture . . . 35
4.1.2.1 Sequence Diagram: Job Offer Management . . . 39
4.1.3 Simplified AI Agent Architecture . . . 40
4.1.3.1 Sequence Diagram: Candidate Application and AI Evaluation . . . 43
4.1.4 UI/UX Expectations . . . 44
4.2 Detailed Design . . . 47
4.2.1 Class Diagram . . . 47
Conclusion . . . 48

5 Implementation, Testing, and Results 49
Introduction . . . 49
5.1 Development Environment . . . 50
5.1.1 SAP Cloud Ecosystem . . . 50
5.1.2 Web, AI and DevOps Tooling . . . 52
5.2 End-to-End Walkthrough . . . 54
5.2.1 Organizational Structure Setup . . . 55
5.2.2 Job Offer Creation and Publication . . . 61
5.2.3 Candidate Application on the Portal . . . 64
5.2.4 AI Evaluation and Recruiter Review . . . 67
5.2.5 Pipeline Progression and Hiring Decision . . . 70
5.2.6 Digital Onboarding . . . 73
5.2.7 Employee Creation and Personnel Administration . . . 75
5.3 Testing . . . 76
5.3.1 Unit Testing . . . 76
5.3.2 AI Engine Capability . . . 78

5.4 Validation 79
5.4.1 Functional Validation 79
5.4.2 Non-Functional Validation 79
Conclusion 81

General Conclusion and Perspectives **82**

A Glossary **85**

B Acronyms **87**

C Complementary Code Excerpts **90**
C.1 Behavior Definition of the Job Offer 90
C.2 Validation Before Publication 91
C.3 Score Computation in the AI Microservice 92
C.4 Anonymized Scoring Input 93

Bibliography **95**

Chapter 1

General Project Context

Introduction

I had the opportunity to complete my first internship at Cognitus Corporation, specifically within its Global Delivery and Support Center (GDSC) located in Tangier, Morocco. This experience constituted my very first immersion into the world of enterprise information systems, and more precisely into the SAP ecosystem. In the sections that follow, I will introduce Cognitus Corporation as a global organization, shed light on its Tangier office and its strategic role within the group, and then present the technical project I was entrusted with during this internship period.

1.1 Cognitus Corporation

1.1.1 About Cognitus Corporation



Figure 1.1: Cognitus Corporation Logo

Cognitus Corporation is a global SAP Gold Partner and a leading provider of cloud ERP implementations, SAP consulting, and digital transformation services. Founded with the mission of helping organizations unlock the full potential of SAP technology, Cognitus has built over two decades of deep industry expertise across more than eleven verticals, including manufacturing, public sector, retail, healthcare, and financial services.

The company offers a comprehensive range of SAP-centered services, spanning end-to-end implementation of SAP S/4HANA, RISE with SAP, and GROW with SAP programs, as well as specialized consulting in areas such as supply chain optimization, contract lifecycle management (CLM), and real-time billing. In addition to its consulting capabilities, Cognitus has developed a suite of proprietary AI-enabled software assets designed to accelerate data migration, automate compliance workflows in government contracting environments, and streamline business process transformation.

In 2025, Cognitus was acquired by IBM, reinforcing IBM's strategic position in the global SAP services market. This acquisition underscores Cognitus's standing as a recognized leader in RISE with SAP and GROW with SAP advisory, and consolidates its portfolio of accelerators and industry-specific SAP solutions within IBM's broader technology ecosystem.

Throughout its growth, Cognitus has maintained a strong commitment to delivering on-shore, off-shore, and nearshore support models, enabling clients around the world to maintain and continuously optimize their SAP environments in a cost-effective and agile manner.

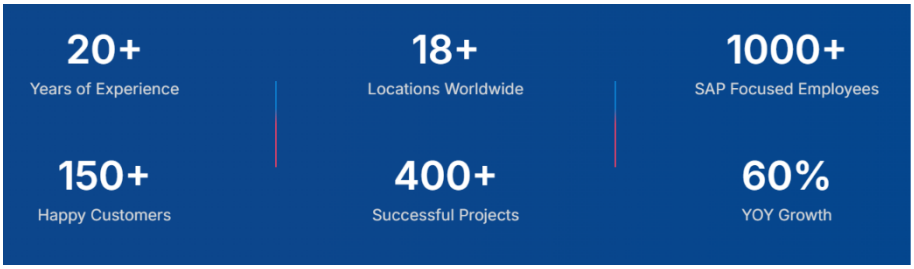


Figure 1.2: Cognitus Corporation — Global Footprint

Cognitus operates across more than 18 global offices, with its headquarters in the United States and a strategic delivery center in Tangier, Morocco. The company serves over 150 enterprise clients, employs more than 1,000 SAP-focused specialists, and has successfully delivered over 400 SAP projects. Its model blends on-shore, off-shore, and nearshore delivery, offering clients both cost efficiency and cultural alignment across time zones.

1.1.2 Cognitus Tangier — Global Delivery and Support Center



Figure 1.3: Cognitus Tangier — Technopark, Avenue Mohammed V

Established in 2021, the Cognitus office in Tangier, Morocco serves as the company's Global Delivery and Support Center (GDSC) for the EMEA region and beyond. Strategically located at Technopark, Avenue Mohammed V, Tangier 90000, this center was designed to leverage the multilingual and highly skilled Moroccan talent pool — capable of operating in French, Arabic, English, Spanish, and Italian — to deliver seamless support and implementation services across different time zones.

The Tangier GDSC plays a pivotal role within the Cognitus global operations model. It functions as a nearshore hub that bridges the gap between client needs in Europe, the Middle East, Africa, and

North America, and the delivery teams responsible for SAP system configuration, ABAP development, WRICEF object construction, testing, and post-go-live support.

Within this center, cross-functional teams collaborate on live SAP projects, supporting clients with S/4HANA implementations and custom extensions. The center embodies Cognitus's philosophy of combining global reach with localized expertise, offering clients the responsiveness and cultural alignment of a nearshore model with the cost efficiency typically associated with offshore delivery.

1.1.3 Business Area of Activity

Cognitus Corporation operates at the intersection of enterprise technology and business transformation, focusing exclusively on the SAP ecosystem. Its business activity is organized around four major pillars that together cover the full lifecycle of an SAP program:

- **SAP Implementation & Transformation:** End-to-end delivery of SAP programs, from initial assessment and blueprint design through go-live and hypercare. Cognitus specializes in *RISE with SAP* (private cloud migrations) and *GROW with SAP* (public cloud adoption for mid-market), leveraging pre-built industry templates to accelerate project timelines significantly.
- **Proprietary Accelerator Assets:** Cognitus distinguishes itself through a suite of SAP-endorsed, AI-enabled software products, including:
  - *CIS-GovCon* — a government contracting compliance solution aligned with DCAA, DCMA, FAR, and CAS regulations;
  - *Contract Lifecycle Management (CLM)* — an AI-powered tool automating contract authoring, approval, and monitoring;
  - *Data Migration Accelerator* — a low-code, AI-assisted platform for migrating legacy data to S/4HANA;
  - *Real-Time Billing* — an accelerator for project-based billing and revenue recognition.
- **Application Management Services (AMS):** 24/7 system support, proactive optimization, basis administration, and performance monitoring for clients already on SAP, through on-shore, off-shore, and nearshore delivery models.
- **Strategic Consulting:** Assessment services, M&A integration technology roadmaps, digital transformation strategy, and organizational change management aligned to SAP programs.

Industries Served. Cognitus concentrates on sectors characterized by high operational complexity and regulatory requirements:

- **Aerospace & Defense** — Government contracting, DCAA/DCMA compliance, MRP;
- **Manufacturing** — Supply chain optimization, production planning, plant maintenance;
- **Life Sciences** — Serialization, quality management, regulatory traceability;
- **Energy, Oil & Gas** — Asset management, project systems, real-time billing;
- **Professional Services** — Project accounting, CLM, resource management;
- **Consumer Products** — Demand planning, distribution, trade promotions;
- **Public Sector** — Federal financials, grants management, GovCon compliance.

Notable Clients. Among Cognitus's reference clients, several high-profile success stories illustrate the depth and speed of its delivery approach:

- **Boom Supersonic** — Implemented SAP S/4HANA in under three weeks to meet commercialization and regulatory compliance requirements;
- **Phlow Corporation** — Adopted RISE with SAP to overcome siloed operations and stringent pharmaceutical regulatory requirements;
- **SunnySky Products** — Standardized multi-site operations using SAP S/4HANA Public Cloud via the GROW with SAP program;
- **Volumetric Building Companies** — Completed a full ERP modernization in three months using GROW with SAP Fast.

1.1.4 Enterprise Resource Planning (ERP)

1.1.4.1 What is an ERP System?

An **Enterprise Resource Planning (ERP)** system is an integrated suite of software applications designed to manage and automate an organization's core business processes across a single, unified platform. Rather than maintaining isolated systems for finance, procurement, human resources, manufacturing, and logistics, an ERP brings all these functions under one shared database and one coherent

process model, enabling real-time visibility, consistent data, and end-to-end process automation across the enterprise.

The fundamental value proposition of an ERP system rests on three pillars:

1. **Integration:** All business modules share a single source of truth. A sales order automatically triggers inventory checks, production planning, procurement, and financial accounting entries — without any manual re-entry of data.

2. **Standardization:** ERP systems enforce best-practice process models (often called *business blueprints*) that align internal workflows with industry standards, reducing errors and audit risk.

3. **Real-Time Visibility:** Decision-makers at all levels gain instant access to consolidated operational data, enabling faster and better-informed decisions.

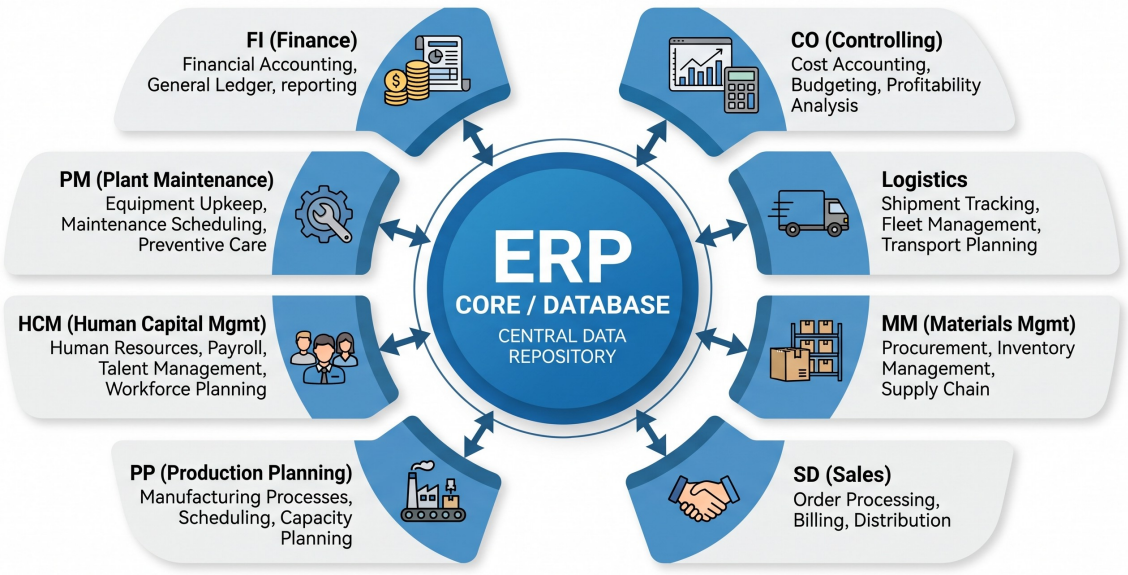


Figure 1.4: ERP System — Integrated Business Module Architecture

Modern ERP systems have evolved significantly since their origins in the 1960s–1970s. Early *Material Requirements Planning (MRP)* systems focused solely on manufacturing inventory. The concept expanded in the 1990s into full *ERP* suites covering the entire enterprise, and has since continued to evolve toward cloud-native, AI-augmented platforms capable of processing massive transactional volumes in real time.

In the context of this project, the ERP system at stake is the **SAP Human Capital Management (HCM)** module, which manages the organization's entire people-related data: employee master records, organizational structure, positions, and job offers. The web recruitment platform developed during this internship consumes and produces data within this SAP HCM layer through standard APIs.

1.1.5 SAP — Systems, Applications, and Products

1.1.5.1 SAP S/4HANA

SAP S/4HANA is SAP's next-generation ERP suite, built natively on the **SAP HANA in-memory database**. Unlike its predecessor SAP ECC, S/4HANA delivers real-time transactional analytics, a simplified data model, and a modern web-based experience through **SAP Fiori**. It covers all core enterprise functions through dedicated modules — notably **HCM** (Human Capital Management) for personnel administration, organizational management, and recruitment, which is the module at the heart of this project.

1.1.5.2 SAP Business Technology Platform (BTP)

SAP BTP is SAP's cloud-native *Platform-as-a-Service (PaaS)*, designed to integrate, extend, and build applications across the SAP ecosystem. It unifies four capability areas: **Database & Data Management** (HANA Cloud, Datasphere), **Analytics** (SAP Analytics Cloud), **Application Development & Automation** (CAP, SAP Build), and **Integration** (SAP Integration Suite, API Management).

1.1.5.3 ABAP and the RESTful Application Programming Model (RAP)

ABAP (*Advanced Business Application Programming*) is SAP's proprietary programming language, in use since the early 1980s. Its modern cloud-ready form is the **ABAP RESTful Application Programming Model (RAP)**, which structures development mainly in three clean layers:

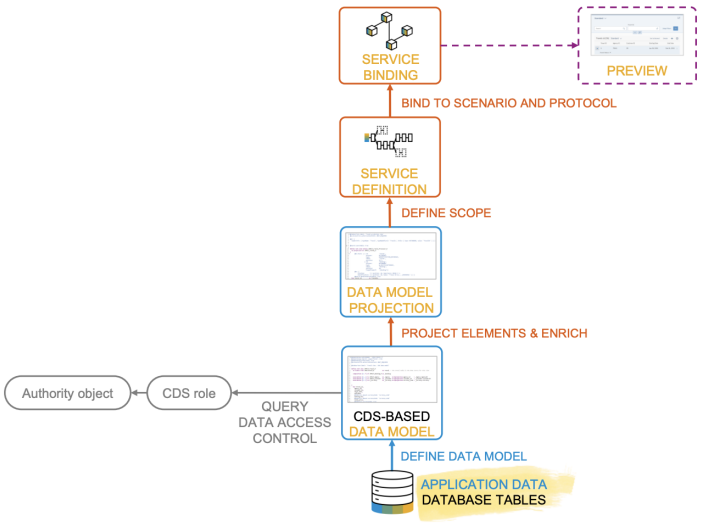


Figure 1.5: ABAP RAP — Layered Architecture

1. **CDS View Entities** — define the data model directly on the HANA database;
2. **Behavior Definition (BDEF)** — specify allowed operations (CRUD, custom actions, business logic) in ABAP;
3. **Service Definition & Binding** — expose entities as **OData V4** endpoints consumable by web apps, Fiori, or mobile.

1.1.6 Internship Overview

This section provides a structured overview of the technical internship carried out at Cognitus Corporation. It situates the project within the academic curriculum of the INPT, outlines the administrative and supervisory framework governing the internship, defines the core technical objectives of the assignment, and details the project planning, governance methodology, and technical ramp-up undertaken during this two-month immersion.

1.1.6.1 Internship Framework

The internship was conducted as the mandatory engineering initiation and technical internship (*Stage INE1*) for the ASEDS program at the INPT, Rabat, under the auspices of the *Agence Nationale de Réglementation des Télécommunications* (ANRT).

For me, this experience represented my very first professional internship and my initial immersion into the world of enterprise information systems and Enterprise Resource Planning (ERP). Carried out on-site (*Présentiel*) at the Cognitus Global Delivery and Support Center (GDSC) in Technopark, Tangier, the internship spanned two months, from **June 15, 2026, to August 15, 2026**: eight weeks of work followed by a closure week.

All contractual, institutional, and organizational modalities were formalized in the official internship agreement (*Convention de stage*). Table 1.1 summarizes the primary administrative parameters of this engagement.

Item / Field	Specification / Details
Host Enterprise	Cognitus Corporation — EMEA GDSC (Tangier, Morocco)
Signatory Representative	**Mr. Mohamed Reghioui** *Function:* General Manager EMEA *Email:* mreghioui@cognitus.com *Phone:* +212 (0) 661 93 99 35
Industrial Supervisor	**Mr. Karim Aït Alla** *Function:* Technical Lead *Email:* kaitalla@cognitus.one *Phone:* +212 (0) 611 66 58 81
Official Assignment Title	*Développement des extensions pour les implémentations SAP* (Development of Extensions for SAP Implementations)
Internship Period	June 15, 2026 – August 15, 2026 (2 months: 8 weeks of work and a closure week)
Working Modality	On-site (*Présentiel*) at Technopark Tangier
Core Scope & Missions	• Comprehensive understanding of enterprise ERP architecture and business workflows; • Acquisition of foundational ABAP development and SAP Fiori UI concepts; • Understanding, design, and construction of WRICEF objects (Workflows, Reports, Interfaces, Conversions, Enhancements, and Forms).

Table 1.1: Internship Agreement Administrative Specifications (*Convention de stage*)

1.1.6.2 Project Objectives (*Objectifs du projet de stage*)

Within the broader scope of developing custom extensions for SAP implementations, the specific project assigned to me was titled: **"Design and Implementation of an Intelligent Talent Acquisition and Employee Administration System Using SAP BTP, ABAP RAP, and Artificial Intelligence"**.

Recruitment and personnel onboarding are among the most critical operational pillars of enter-

prise Human Capital Management (HCM). However, traditional enterprise recruitment pipelines face significant challenges: corporate recruiters are frequently overwhelmed by high volumes of incoming applications, manual resume screening is notoriously labor-intensive and subjective, and legacy SAP GUI transaction screens are ill-suited for external candidates and modern mobile-first HR users.

To solve this dilemma, the project sought to bridge the gap between a modern, consumer-grade candidate experience and the enterprise-grade SAP back-end through six core objectives:

- **SAP HCM Data Modeling via ABAP RAP:** Construct Core Data Services (CDS) view entities defining the fundamental HR data models (Organizational Units, Job Positions, Recruitment Job Offers, and Candidate Profiles) directly on the SAP HANA database layer.
- **Business Logic & Behavior Implementation:** Define transactional behavior definitions (BDEF) in ABAP Cloud to enforce business rules, automated determinations, data validations, and state transitions across the candidate lifecycle.
- **Secure OData V4 API Exposure:** Expose the RAP business entities through SAP Service Definitions and Service Bindings as standard OData V4 RESTful APIs, facilitating seamless, decoupled communication with external web applications.
- **AI-Powered Resume Analysis & Candidate Scoring:** Integrate an intelligent AI screening service capable of parsing semi-structured CV documents (PDF, Word, or text), extracting candidate competencies and work histories, and automatically calculating a rubric-based match score against the requirements of the job.
- **Decoupled External Web Portal:** Build an intuitive, responsive web portal where candidates can browse openings, submit applications, and track their progress without any direct access to the SAP backend, while recruiters review and rank the candidates in an SAP Fiori cockpit.
- **WRICEF Construction & Clean Core Compliance:** Implement the required WRICEF objects (Interfaces, Reports, Enhancements) while adhering to SAP's *Clean Core* principles to preserve system upgradability.

1.1.6.3 Project Planning and Management

To ensure the successful delivery of an enterprise-grade cloud extension within the two-month internship timeframe, this project adopted a rigorous engineering management framework. This structure

integrated an intensive preliminary training curriculum on SAP Learning, learning about HCM and S/4HANA, and an Agile development lifecycle governed by the Scrum framework.

1.1.6.3.1 A. Formation et Montée en Compétences — Technical Training & Upskilling

Entering the SAP enterprise ecosystem required a dedicated, intensive preparation phase. The first weeks of the internship were dedicated to technical upskilling on **SAP Learning** [6], through three official courses totalling about **41 hours** (4 h for BTP100, 17 h for S4D400, and 20 h for S4D437) with hands-on exercises in the SAP BTP ABAP Environment, alongside an exploration of SAP and Human Capital Management domain concepts:

1. **Exploring SAP Business Technology Platform (BTP100)** — *Level: Beginner*

   Core Curriculum: Covered the foundational multi-cloud architecture of SAP BTP, core infrastructure providers, security and identity provisioning, runtime environments — Cloud Foundry, Kyma, and ABAP Cloud — and side-by-side extension paradigms.

   Direct Project Value: Provided the architectural blueprint to design the recruitment platform as a decoupled cloud application on SAP BTP, strictly adhering to SAP's **Clean Core** strategy by leaving standard ERP core tables untouched.

2. **Learning Basic ABAP Programming (S4D400)** — *Level: Beginner*

   Core Curriculum: Covered the basics of ABAP programming in the SAP BTP ABAP Environment: data types and internal tables, SQL access to database tables, modern expressions such as `VALUE`, `CORRESPONDING`, and `FILTER`, object-oriented modularization with classes and interfaces, and the Eclipse-based ABAP Development Tools.

   Direct Project Value: Established the programmatic foundation required to write clean, type-safe business logic, custom calculations, and validation routines.

3. **Building Transactional Apps with the ABAP RESTful Application Programming Model (S4D437)** — *Level: Advanced*

   Core Curriculum: Delivered deep mastery of Core Data Services view entities, transactional Behavior Definitions, the RAP interaction phase and save sequence, transactional draft handling, custom business actions, determinations, validations, dynamic feature control, and publishing OData V4 Web API and UI services.

   Direct Project Value: Formed the functional and transactional backbone of the platform, enabling robust draft state management for incomplete requisitions, fine-grained entity validation, atomic

employee onboarding commits, and seamless API exposure for both the React web portal and the Python AI microservice.

4. **SAP HCM Domain Research & Organizational Modeling**

   In addition to technical frameworks, substantial self-directed research was conducted into standard **SAP Human Capital Management** business processes. This covered the structural relationship between **Organizational Units** and **Positions** — enforcing vacancy states: *Vacant*, *Filled*, *Frozen*, and budgeted compensation ceilings — as well as classical personnel **Infotypes** for Organizational Assignment, Personal Data, Addresses, and Bank Details.

   Direct Project Value: Enabled the reverse-engineering and modernization of standard SAP HCM structures into native CDS data models and informed the design of the atomic onboarding action that generates complete Employee Master Records.

The completion of the courses is recorded in SAP Learning. The BTP100 and S4D400 courses award a course completion badge (Figure 1.6), while S4D437 records the completion without a badge. As the learning list shows (Figure 1.7), BTP100 was completed on June 15, 2026, and the S4D400 and S4D437 courses, started during the internship, were formally completed on October 11, 2026.



Figure 1.6: SAP Learning — Course Completion Badges (BTP100 and S4D400)

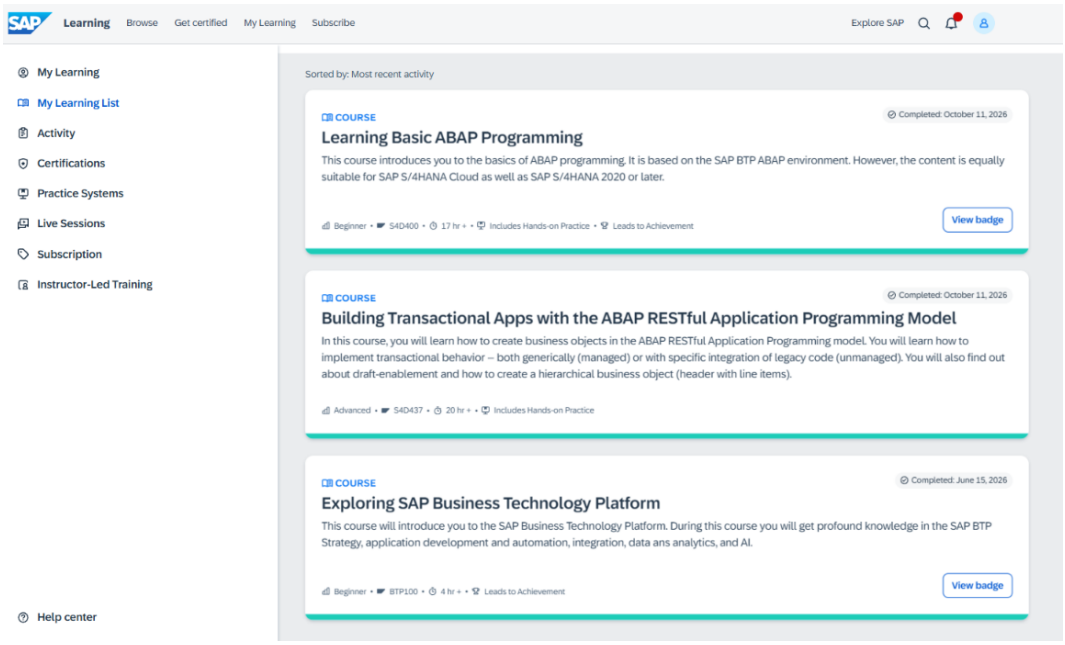


Figure 1.7: SAP Learning — List of Completed Courses

1.1.6.3.2 B. Conduite de Projet — The Scrum Framework

To manage developmental complexity and foster rapid adaptability, the project was conducted following the **Scrum Framework** [16], the premier agile methodology for iterative, high-value product engineering.

Foundational Scrum Principles. Implementing the project under Scrum best practices relied on three foundational pillars:

- **Autonomous, Cross-Functional Execution** — Rapidly pivoting technical scopes and adapting to architectural feedback through close collaboration between the intern and the industrial supervisor.
- **Iterative Time-Boxed Sprints** — Organizing the development lifecycle into short, structured two-week iterations facilitating continuous integration, regular testing, and progressive feature delivery.
- **Product Backlog Prioritization** — Systematically consolidating functional and non-functional requirements into a dynamic, centralized backlog that evolved as technical discoveries were made.

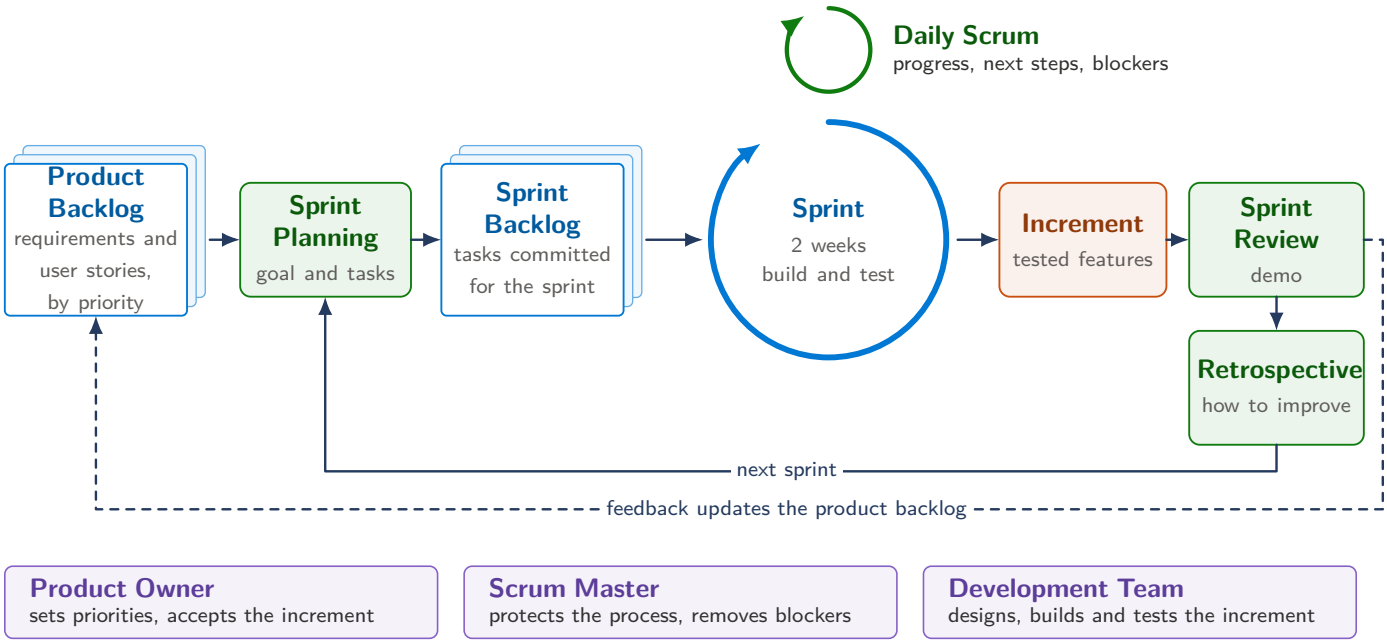


Figure 1.8: The Scrum Framework — Iterative Development Cycle

Scrum Roles & Synergy. The project established clear ownership across the three canonical Scrum roles:

- **Product Owner** — Represented by the enterprise leadership and Technical Lead, Mr. Karim Aït Alla, acting as the proxy for corporate expectations, resolving requirement ambiguities, and establishing backlog priorities.
- **Scrum Master** — Ensured rigorous adherence to Agile practices, monitored sprint cadence, eliminated technical roadblocks such as cloud quota allocations and API connectivity issues, and safeguarded project milestones.
- **Development Team** — Executed autonomously by the intern, Mr. Oualid AMTOUT, assuming full technical responsibility for architectural modeling, RAP backend coding, React portal frontend engineering, AI microservice development, and testing.

Core Scrum Ceremonies. The iterative lifecycle was punctuated by four essential events:

- **Sprint Planning** — Conducted at the start of each iteration to decompose user stories from the Product Backlog into discrete technical tasks and commit to sprint goals.
- **Sprint Execution** — The active engineering phase encompassing CDS data modeling, behavior definition programming, UI development, and API integration.

- **Sprint Review & Retrospective** — Concluding demonstrations where working increments were presented to the supervisor, followed by retrospective evaluations to optimize developmental efficiency for subsequent sprints.
- **Backlog Refinement** — Continuous reassessment and estimation of upcoming user stories to adapt to emerging technical constraints.

Sprint Plan. The product backlog was distributed over four two-week sprints, followed by a closure week (Figure 1.9). Each sprint had a goal and a set of backlog items taken from the work packages of the Software Development Plan: the first two sprints were devoted to the SAP training and the scoping of the project, the third to the specification and the data model, and the fourth to the integrated prototype, its tests, and its deployment. The backlog items were followed as task lists kept with the project documentation, and the source code was versioned on GitHub.

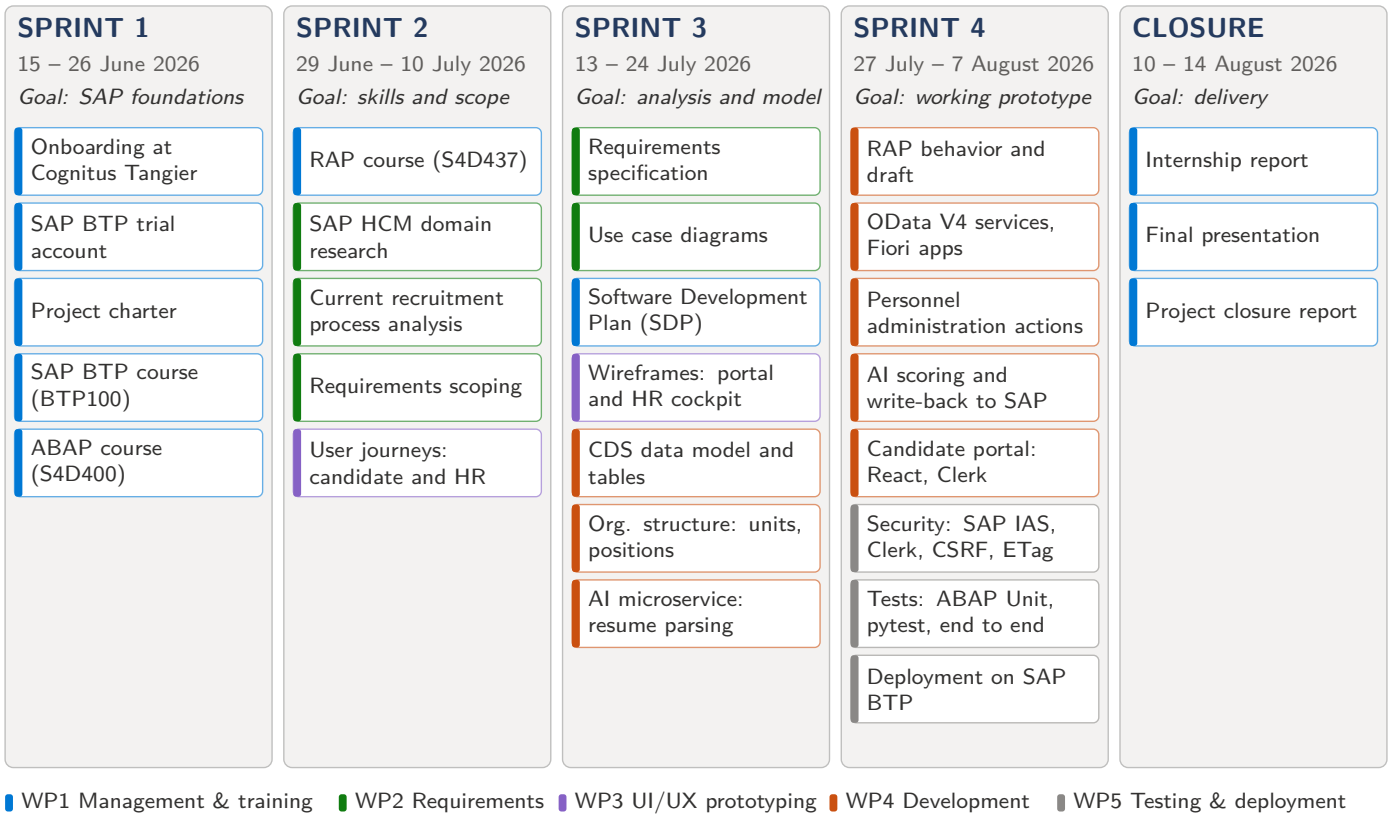
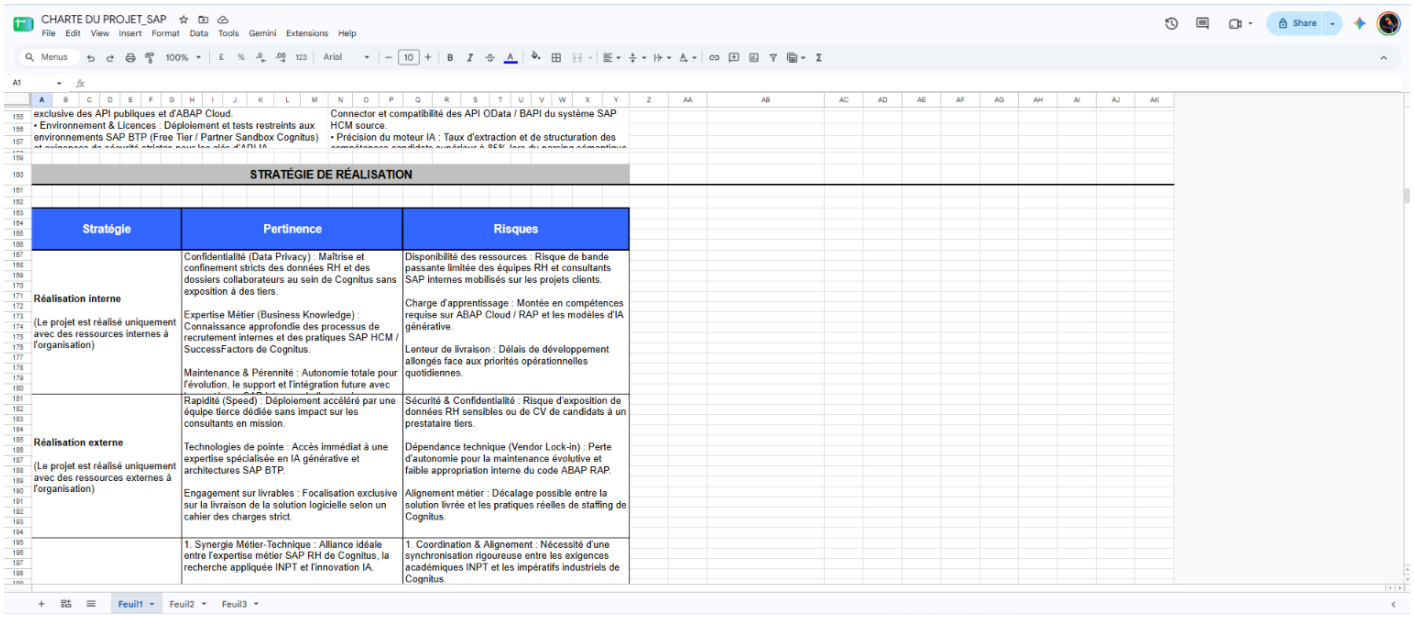


Figure 1.9: Sprint Plan — Product Backlog Distributed over Four Two-Week Sprints

1.1.6.3.3 C. Gouvernance du Projet — Formal Governance Artifacts Project governance and task monitoring were anchored in two formal enterprise management documents alongside a detailed Gantt schedule:

- **Project Charter:** This document formalizes the context, goal, and objectives of the project, its scope (inclusions and exclusions), the chosen delivery strategy, the stakeholder analysis, the main risks with their mitigation, a preliminary budget and schedule, and the value expected from the solution. Figure 1.10 shows its delivery strategy section, which compares an internal, an external, and a joint delivery.



exclusive des API publiques et d'ABAP Cloud.
• Environnement & Licences : Déploiement et tests restreints aux environnements SAP BTP (Free Tier / Partner Sandbox Cognitus)

Connector et compatibilité des API OData / BAPI du système SAP HCM source.
• Précision du moteur IA : Taux d'extraction et de structuration des

STRATÉGIE DE RÉALISATION

Stratégie	Pertinence	Risques
Réalisation interne (Le projet est réalisé uniquement avec des ressources internes à l'organisation)	Confidentialité (Data Privacy) : Maîtrise et confinement stricts des données RH et des dossiers collaborateurs au sein de Cognitus sans exposition à des tiers. Expertise Métier (Business Knowledge) : Connaissance approfondie des processus de recrutement internes et des pratiques SAP HCM / SuccessFactors de Cognitus. Maintenance & Pérennité : Autonomie totale pour l'évolution, le support et l'intégration future avec	Disponibilité des ressources : Risque de bande passante limitée des équipes RH et consultants SAP internes mobilisés sur les projets clients. Charge d'apprentissage : Montée en compétences requise sur ABAP Cloud / RAP et les modèles d'IA générative. Lenteur de livraison : Délais de développement allongés face aux priorités opérationnelles quotidiennes.
Réalisation externe (Le projet est réalisé uniquement avec des ressources externes à l'organisation)	Rapidité (Speed) : Déploiement accéléré par une équipe tierce dédiée sans impact sur les consultants en mission. Technologies de pointe : Accès immédiat à une expertise spécialisée en IA générative et architectures SAP BTP. Engagement sur livrables : Focalisation exclusive sur la livraison de la solution logicielle selon un cahier des charges strict.	Sécurité & Confidentialité : Risque d'exposition de données RH sensibles ou de CV de candidats à un prestataire tiers. Dépendance technique (Vendor Lock-in) : Perte d'autonomie pour la maintenance évolutive et faible appropriation interne du code ABAP RAP. Alignement métier : Décalage possible entre la solution livrée et les pratiques réelles de staffing de Cognitus.
	1. Synergie Métier-Technique : Alliance idéale entre l'expertise métier SAP RH de Cognitus, la recherche appliquée INPT et l'innovation IA.	1. Coordination & Alignement : Nécessité d'une synchronisation rigoureuse entre les exigences académiques INPT et les impératifs industriels de Cognitus.

Figure 1.10: Project Charter — Delivery Strategy Section

- **Software Development Plan:** The SDP is the operational plan of the project. Its work breakdown structure organizes the work into **five work packages** (Figure 1.11):

  1. **WP1 — Project Management and Training:** project charter, development plan, SAP BTP trial account, SAP courses, internship report, and closure report.
  2. **WP2 — Requirements & Specifications Analysis:** analysis of the current process, SAP HCM recruitment and personnel administration workflows, functional and non-functional requirements, use cases, comparison of language models, and conceptual data model.
  3. **WP3 — UI/UX Prototyping:** user journeys, wireframes of the candidate portal and of the HR cockpit, application and onboarding forms, and visual style.
  4. **WP4 — Backend & Frontend Development:** database tables, CDS views, behavior definitions and business objects, OData V4 services, the React candidate portal, and the FastAPI AI microservice.
  5. **WP5 — Testing & Deployment:** unit and integration tests, tests of the recruitment pipeline and of the onboarding, edge cases, end-to-end tests, and deployment on SAP BTP.

This report follows the same structure: the analysis of Chapter 3 and the design of Chapter 4 cover WP2 and WP3, and the realization and tests of Chapter 5 cover WP4 and WP5.

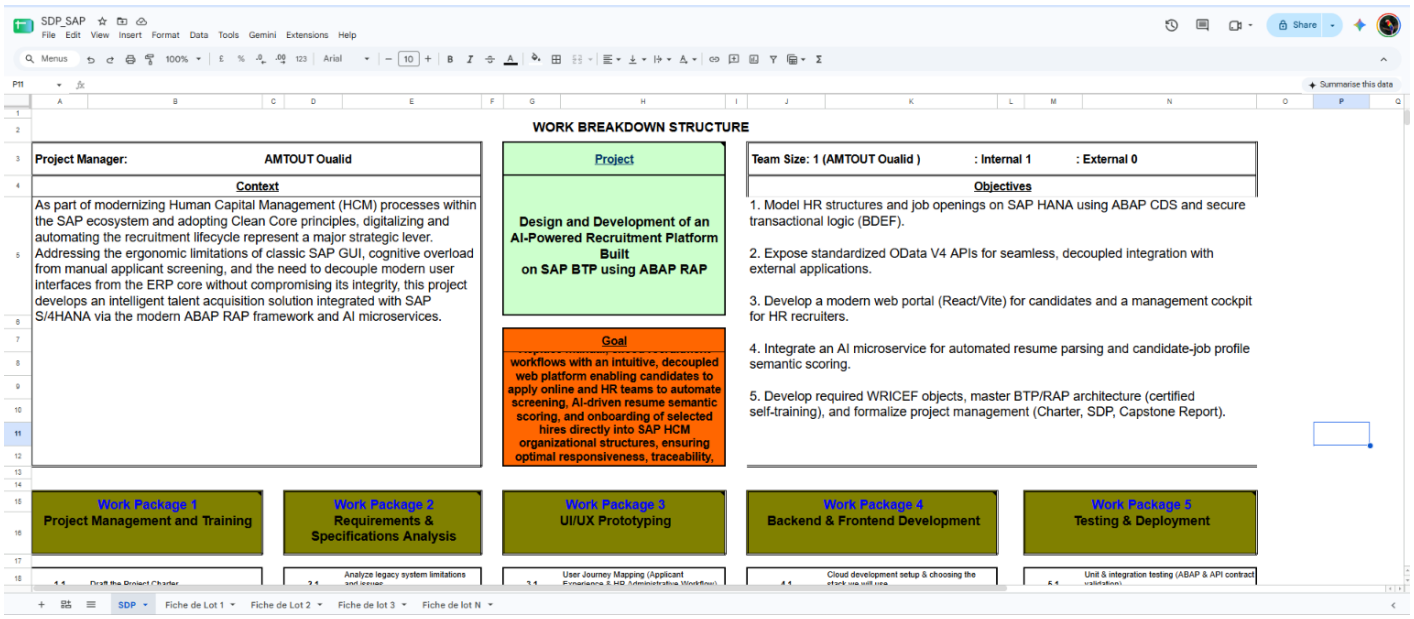


Figure 1.11: Software Development Plan — Work Breakdown Structure

The main risks identified in the project charter, with their assessment and the measures taken to reduce them, are summarized in Table 1.2.

Table 1.2: Main Project Risks and Mitigation Measures

Risk	**Impact**	**Probability**	**Criticality**	**Category**	**Mitigation**
R1. Integration of the three components: OData V4 calls from the React portal (CSRF, ETag, CORS, sessions of the trial account) and write-back of the AI score	High	Medium	High	Technical	Dedicated RAP actions (`applyToJob`, `setAIEvaluation`), development proxy for the portal, and early end-to-end tests.
R2. Quality of the AI evaluation: incomplete parsing of heterogeneous resumes or wrong assessment of the skills	Medium	Medium	Medium	Functional, AI	Normalized prompts, strict JSON output, score computed in Python, human decision required, and tests on fictitious resumes of different levels.
R3. Confidentiality: exposure of the personal data of candidates or employees	Critical	Low	High	Security, legal	Separate authentication (SAP IAS for HR, Clerk for candidates), HTTPS, fictitious test resumes, and an AI microservice without database.
R4. User adoption: reluctance toward new screens and algorithmic suggestions	Medium	Low	Low	Organizational	Standard SAP Fiori Elements interfaces, screen mockups, and regular demonstrations.
R5. Trial account and schedule: session expiry, limits of the SAP BTP trial account, and risk of exceeding the planned weeks	High	Low	Medium	Planning, infrastructure	Scrum with two-week sprints, prioritized minimum viable product, and regular backup of the code with abapGit and GitHub.

1.1.6.3.4 D. Planning d'Exécution — Implementation Schedule & Gantt Diagram The eight weeks of work were organized in four phases, followed by a closure week, as summarized in Table 1.3.

Table 1.3: Implementation Schedule — Phases, Timeframes, and Key Milestones

Phase	**Timeframe**	**Focus Area & Key Milestones**
Phase 1: Onboarding & Training	Weeks 1–4	Team integration at Technopark Tangier, SAP Learning courses (BTP100, S4D400, S4D437), project charter, and initial requirement scoping.
Phase 2: Analysis & Data Modeling	Weeks 5–6	Software Development Plan, requirements and UML use cases, CDS data model, and organizational structure modeling.
Phase 3: Development & Integration	Weeks 6–7	RAP behavior definitions, draft handling, OData V4 services, Fiori Elements applications, FastAPI AI microservice, and React candidate portal.
Phase 4: Testing & Finalization	Week 8	Unit and end-to-end tests, feature control validation, deployment on SAP BTP, and technical documentation.
Closure	Week 9	Internship report, presentation, and project closure report.

The detailed schedule, with its tasks, dependencies, and milestones, was modeled with **GanttProject** [17], as illustrated in Figure 1.12.

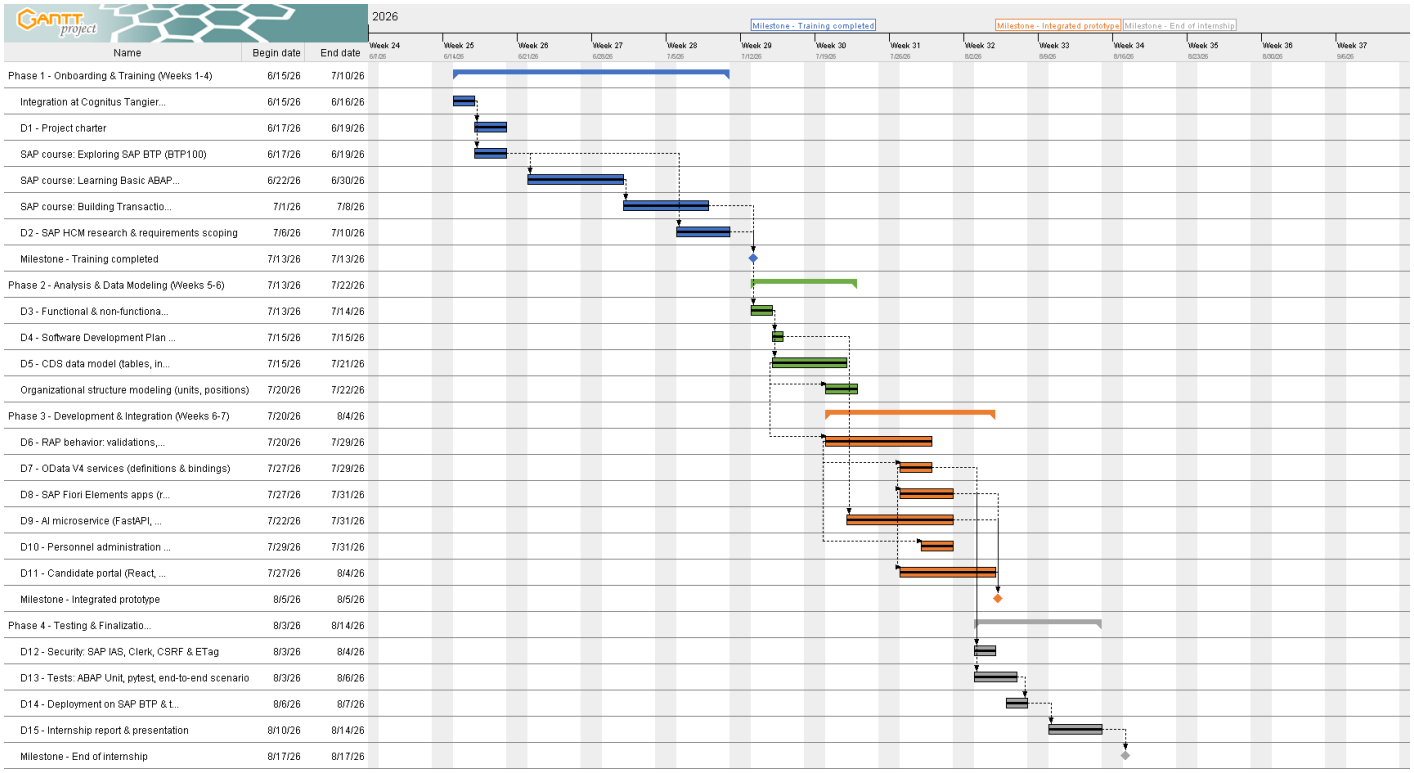


Figure 1.12: Project Planning — Detailed Gantt Diagram

Conclusion

This chapter introduced Cognitus Corporation, a global SAP Gold Partner now part of IBM, and its Tangier Global Delivery and Support Center where this work was carried out. We examined the company's core business areas, key industries, and notable client portfolio, then established the conceptual foundations of ERP systems and the modern SAP technology stack underpinning this project. Finally, we detailed the institutional framework, technical objectives, training curriculum, Scrum-based project governance, and structured implementation schedule that guided the two-month internship. The following chapter presents the project context and objectives in greater detail.

Chapter 2

Project Context and Objectives

Introduction

This chapter delineates the foundational context and strategic perimeter of the project. It begins by establishing the general context of enterprise software modernization, examining how digital transformation and cloud architectures intersect with corporate talent acquisition. Next, it details the core operational and architectural problem statement that motivated this work, analyzing the systemic bottlenecks of manual screening, rigid legacy ERP workflows, semantic keyword limitations, and master data isolation. Finally, it articulates the primary project objectives guiding the design and development of the proposed cloud-native, AI-powered recruitment platform built on SAP BTP using ABAP RAP.

2.1 General Project

2.1.1 General Context

Enterprise Resource Planning (ERP) systems constitute the central digital backbone of modern organizations, orchestrating mission-critical financial and human capital processes. However, while core transactional systems have largely transitioned to scalable cloud architectures, corporate recruitment workflows often remain fragmented, relying on manual screening methods, disconnected point solutions, and legacy procedures that introduce substantial administrative overhead and hiring latency.

Building upon intensive technical upskilling on SAP Business Technology Platform (SAP BTP), ABAP Cloud, and the ABAP RESTful Application Programming Model (RAP), this project addresses these operational hurdles. It aims to design and implement an AI-powered talent acquisition platform that unifies modern candidate self-service with robust enterprise SAP HCM governance, while strictly adhering to SAP's Clean Core extensibility principles.

2.1.2 Problem Statement

Despite the strategic importance of talent acquisition, enterprise recruitment workflows face several operational and architectural bottlenecks that can be synthesized as follows:

- **Manual Screening Overload and Latency:** Inundated by large application volumes, recruiters face severe cognitive fatigue from manual resume review, substantially inflating time-to-hire and introducing subjective evaluation biases.
- **Workflow Rigidity and Poor Ergonomics:** Legacy ERP systems feature complex transactional interfaces designed strictly for back-office administration, lacking candidate self-service portals and agile pipeline tracking workflows.
- **Semantic Inadequacy in Traditional ATS:** Conventional keyword-matching tools lack semantic reasoning, frequently rejecting qualified applicants due to vocabulary mismatches while rewarding candidates who practice artificial keyword stuffing.
- **Enterprise ERP Isolation and Manual Re-entry:** Disconnected third-party recruitment tools force manual transcription of hired candidate data into SAP HCM records, creating data discrepancies, operational delays, and Clean Core upgrade risks.

2.1.3 Project Objectives

In direct response to the operational bottlenecks and technological limitations identified in the problem statement, this project aims to design and implement an enterprise-grade, cloud-native recruitment and talent acquisition platform. The primary objectives guiding the realization of this system are defined as follows:

- **Needs Analysis and Technology Selection**

  Conduct a rigorous requirements analysis across the recruitment and onboarding lifecycle, evaluating and selecting an optimal technology stack capable of supporting enterprise scalability, multi-cloud flexibility, and intelligent automation.

- **Native SAP Ecosystem and ERP Integrability**

  Build the platform natively within the SAP cloud ecosystem (SAP BTP) using modern ABAP Cloud, ensuring seamless integration with existing enterprise ERP infrastructure while strictly adhering to Clean Core principles.

- **Organizational Structure and Job Offer Governance**

  Enable HR administrators to model organizational units, create and allocate positions linked to those units, define and publish job offers, and oversee the full recruitment pipeline directly anchored to organizational vacancies.

- **AI Resume Analysis Engine**

  Develop a multi-stage AI microservice that uses a large language model to parse resumes into structured profiles, enrich them with the candidate's GitHub projects, and score each candidate against the job offer with explainable results.

- **Comprehensive Quality Assurance, Verification, and Cloud Deployment**

  Test the backend business logic, the AI microservice, and the complete recruitment process at several levels (unit tests, an end-to-end scenario, an evaluation of the AI engine, and measurements of security and response times) to verify the reliability and data integrity of the platform deployed on SAP BTP.

Conclusion

This chapter has established the foundational framework of the project, analyzing the general context of enterprise ERP and recruitment modernization. It has articulated the core problem statement, highlighting the operational bottlenecks, user experience deficiencies, semantic limitations of keyword ATS, and architectural disconnects with central HR systems. In response to these challenges, the chapter defined the strategic objectives of the project, spanning technology selection, native SAP BTP Clean Core integration, organizational structure modeling, AI resume analysis, and rigorous end-to-end verification. Having delineated the contextual perimeter and goals of the initiative, the next chapter will present the comprehensive analysis and detailed specifications of the proposed platform.

Chapter 3

Analysis and Specifications

Introduction

This chapter evaluates the current enterprise recruitment methods, identifies their systemic limitations, and presents the distinctive advantages of our proposed platform. It then formally specifies the functional and non-functional requirements alongside global use case modeling.

3.1 Existing System Study

3.1.1 Current Recruitment Methods

Enterprise recruitment today relies on three dominant technology models, each with distinct operational characteristics:

- **Commercial SaaS ATS Platforms** — Workday Recruiting, Oracle Taleo, Greenhouse, iCIMS Talent Cloud
- **Legacy On-Premise ERP Modules** — SAP HCM Personnel Administration, SAP Recruitment, SAP Organizational Management via SAP GUI
- **Hybrid Manual Workflows** — Spreadsheet-based tracking, email-driven communication, disconnected document storage

While these approaches offer partial solutions — SaaS platforms provide modern candidate portals and pipeline visualization, ERP modules ensure data consistency within the corporate system, and manual workflows offer flexibility — they share a common set of structural deficiencies. SaaS platforms operate in isolation from the enterprise ERP core, forcing HR staff to manually re-enter candidate data after hiring, a slow and error-prone step on records that later feed payroll. Legacy SAP GUI screens are inaccessible to external candidates and offer **no self-service**. Resume screening relies either on manual reading or on rigid keyword matching, which can reject qualified candidates whose resumes do not use the expected words. Furthermore, any customization of on-premise ERP recruitment workflows requires invasive core modifications that violate SAP's Clean Core paradigm and generate significant technical debt.

3.1.2 Particularities of Our Solution

Our platform addresses every limitation identified above through three architectural pillars:

Native SAP BTP Clean Core Integration. Upon HR approval, a single RAP action creates the employee master record and its child records from the verified onboarding data, in less than half a second in our measurements and without any manual re-entry. Built as a side-by-side extension on SAP BTP using ABAP Cloud and SAP HANA Cloud, the architecture leaves the standard ERP objects untouched, which keeps the core upgradable (Clean Core).

AI-Assisted Resume Screening. A Python microservice uses a large language model (GPT-4o-mini) to parse each resume into a structured profile, enrich it with the candidate's GitHub projects, and score it against the rubric of the role and the requirements of the job offer, with evidence for each criterion. The final score is computed deterministically in Python. The evaluation runs in the background, takes about half a minute to a minute per resume, and costs about 0.003 USD. Transparent per-criterion scores with natural-language justifications keep the recruiter in control of the decision.

Dual Modern User Experience. A responsive React/Vite candidate portal enables job discovery, one-click application, real-time pipeline tracking, and onboarding self-service. An enterprise SAP Fiori Elements cockpit equips recruiters with draft handling, configurable pipelines, and one-click stage progression.

3.1.3 Comparative Synthesis

Table 3.1: Comparative Analysis — Current Methods vs. Our Proposed Solution

Dimension	**SaaS ATS**	**Legacy SAP GUI**	**Our Solution**
ERP Data Integration	Isolated silo, manual re-entry	Native but internal only	**Native & decoupled via SAP BTP**
Employee Creation	Manual re-entry after hiring	Manual admin action	**Automatic, about 0.4 s, no re-entry**
Candidate Self-Service	Web portal available	Non-existent	**Responsive React portal**
Resume Screening	Keyword matching	Manual reading	**AI rubric scoring with evidence, in the background**
Recruiter Interface	Modern dashboard	Archaic GUI screens	**SAP Fiori cockpit with drafts**
Clean Core Safety	External subscription	Core modifications	**No SAP standard object modified**

3.2 Requirements Identification and Analysis

3.2.1 Feature Identification and System Actors

The system engages external applicants through a public web portal and empowers internal HR personnel via an enterprise cockpit on SAP BTP. The actors are organized into two categories:

3.2.1.1 Primary Actors

1. **External Candidate** — Prospective applicants who discover job openings, submit multi-format applications, track pipeline progress, and complete the digital onboarding dossier upon selection.

2. **HR Recruiter** — Operational HR professionals who create and publish job requisitions, configure pipeline stages, consult AI evaluations, advance or reject applicants, and curate the Talent Pool.

3. **HR Administrator** — Senior HR personnel who manage the organizational hierarchy and positions, audit onboarding submissions, and execute the atomic action generating SAP Employee Master Records.

3.2.1.2 Secondary Actors

1. **AI Evaluation Microservice** — Background service that reads resumes, extracts structured data, computes rubric-based match scores, and returns explainable evaluations that are stored with the application in SAP.

2. **SAP ERP Backbone** — Central system of record that maintains employee master data, enforces organizational structures, and guarantees transactional integrity via OData V4 services.

3.2.2 Functional Requirements

The functional requirements define the core capabilities delivered by the platform, leveraging the SAP RAP managed scenario for transactional governance and draft handling:

1. **Organizational Structure and Position Management**

    - **Unit Administration:** HR administrators create, update, and organize hierarchical organizational units with departmental relationships.

- **Position Allocation:** Positions are anchored to organizational units with budgeted salary ranges and occupancy states — Vacant, Filled, or Frozen.
- **Position Validation:** Job offers can only be assigned to authorized positions currently in Vacant status.

2. **Job Requisition and Pipeline Lifecycle**

- **Draft Handling:** Recruiters prepare and edit job requisitions in draft mode; unfinished work is saved automatically without affecting active records.
- **Pipeline Stages:** Recruiters configure multi-step pipelines per job offer with customizable sequence order.
- **Requisition Actions:** Dedicated actions publish, close, or reopen job requisitions.
- **Feature Control:** Dynamic field editability and action availability adapt to the current requisition status.

3. **Public Candidate Portal**

- **Job Discovery:** Candidates browse published offers, filter opportunities, and view role specifications.
- **Application Submission:** Candidates submit applications with contact details and upload resumes in PDF, Word, or plain text.
- **Tracking and Self-Service:** Candidates track real-time pipeline progression or withdraw their application.

4. **AI Resume Analysis**

- **Document Parsing:** The AI pipeline normalizes heterogeneous resumes (PDF, DOCX, TXT) and extracts structured candidate data.
- **Rubric Scoring:** A matching score is computed between the candidate's experience and the requirements of the role and of the job offer, with key strengths and areas for improvement.
- **Evaluation Integration:** Scores and evaluation dossiers are transmitted to the SAP backend to rank applicants.

5. **Candidate Evaluation and Stage Progression**

- **Review Cockpit:** Recruiters view AI-generated scores and manage applicant progression across pipeline stages.
- **Stage Actions:** Recruiters advance, put on hold, reject, or accept candidates, record HR notes, and send e-mails to the candidates.

6. **Automated Onboarding and Employee Creation**

   - **Onboarding Wizard:** Accepted candidates complete a secure form with personal, banking, and emergency contact information.
   - **Atomic Employee Creation:** HR administrators trigger an approval action that generates the Employee Master Record and updates the position status from Vacant to Filled in a single transaction.

7. **Employee Lifecycle and Talent Pool**

   - **Personnel Administration:** HR personnel update employee details, execute promotions, or reassign employees across units.
   - **Talent Pool:** Non-selected candidates are cataloged in a centralized pool for future re-engagement.

3.2.3 Non-Functional Requirements

Beyond its functional scope, the platform must satisfy the following quality requirements:

- **Security:** CSRF token validation, instance-level authorization controls, and dynamic feature toggling based on entity state.
- **Performance:** SAP HANA Cloud in-memory processing keeps the SAP operations under one second, and the AI evaluation runs asynchronously, so that users never wait for it.
- **Maintainability:** Clean Core compliance (ABAP for Cloud Development, released APIs only, behavior definitions in strict mode 2) and clear RAP layer separation.
- **Availability:** Transactional draft mechanism preserves in-progress data across session interruptions.
- **Interoperability:** Standardized OData V4 service contracts enable consumption by SAP Fiori, React, and Python clients.

3.2.4 Global Use Case Diagram

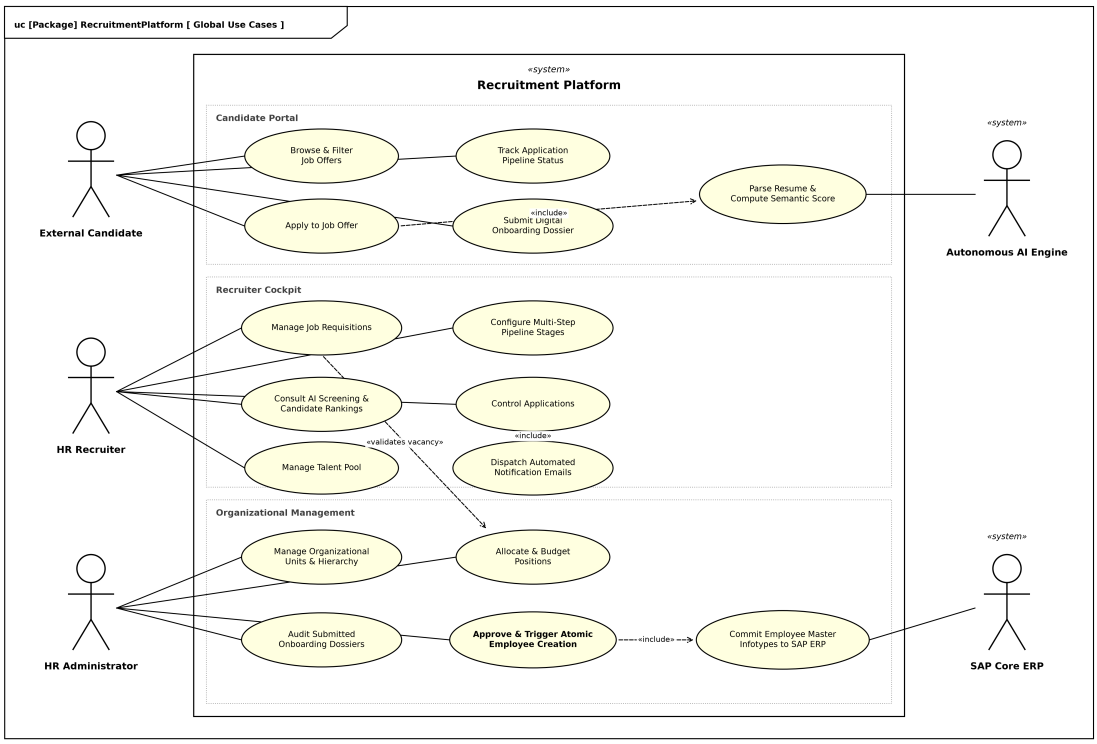


Figure 3.1: Global System Use Case Diagram

3.2.4.1 User Stories

External Candidate: Discovers a published job offer → submits application with resume → tracks real-time pipeline progression → completes onboarding dossier upon acceptance.

HR Recruiter: Creates job requisition in draft mode → configures pipeline stages and publishes → consults AI-generated candidate rankings → advances or rejects applicants through each stage.

HR Administrator: Models organizational hierarchy and creates vacant positions → audits submitted onboarding dossiers → triggers atomic employee creation → position status updates from Vacant to Filled.

Conclusion

This chapter analyzed the existing recruitment landscape, compared it with our platform in a comparative synthesis, and established a complete specification of functional and non-functional requirements with global use case modeling. The next chapter details the architecture and the design of the solution.

Chapter 4

Solution Design and Architecture

Introduction

This chapter presents the design of the Talently platform. It first describes the global architecture of the solution, then details its two core components, the SAP BTP ABAP RAP backend and the AI evaluation microservice, before presenting the user interfaces and the detailed design through class and sequence diagrams.

4.1 Application Architecture

4.1.1 Global System Architecture

Talently follows a modular and decoupled architecture that respects SAP's Clean Core principle: all business data and rules live in an SAP BTP extension, while the AI processing runs in a separate microservice. Figure 4.1 gives an overview of the solution.

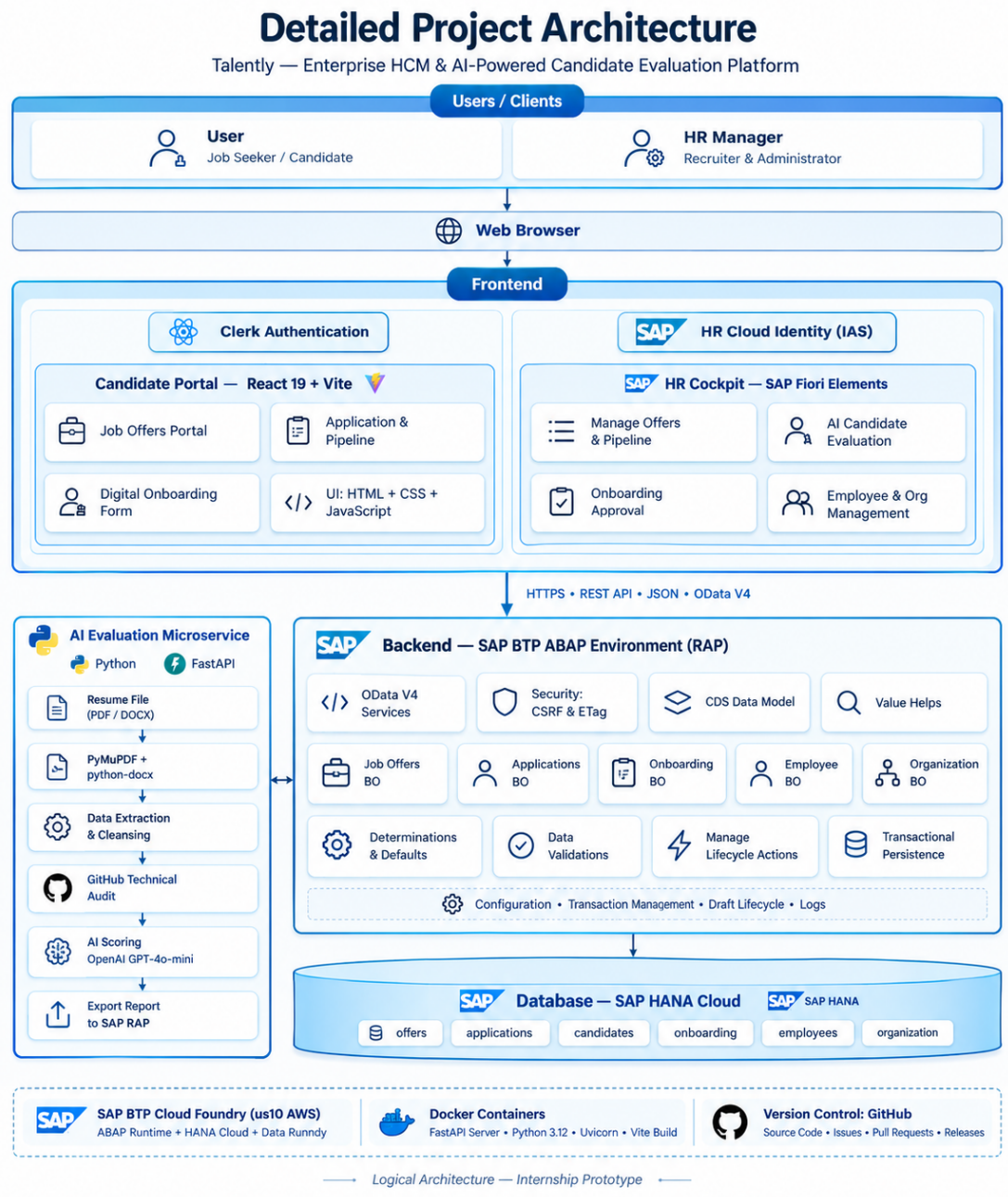


Figure 4.1: Global System Architecture of the Talently Platform

- **Users:** two profiles access the platform through a web browser: the *candidate* (job seeker) and

the *HR manager* (recruiter and administrator).

- **Frontend:** two separate portals, each with its own identity provider. The **candidate portal** (React 19 + Vite) offers the job board, the application and pipeline tracking, and the onboarding form, with authentication handled by **Clerk**. The **HR cockpit** (SAP Fiori Elements) offers offer and pipeline management, AI candidate evaluation, onboarding approval, and employee and organization management, with authentication handled by **SAP Cloud Identity Services (IAS)**.
- **Backend (SAP BTP ABAP Environment, RAP):** the system of record. It exposes OData V4 services protected by CSRF tokens and ETags, and implements the five business domains (job offers, applications, onboarding, employees, and organization) as RAP business objects with determinations, validations, and lifecycle actions.
- **AI evaluation microservice (Python, FastAPI):** analyzes resumes, audits the candidate's GitHub activity, scores the candidate with GPT-4o-mini, and returns the result to the portal, which writes it to the backend.
- **Database (SAP HANA Cloud):** stores the active and draft records of all HR domains.
- **Infrastructure:** SAP BTP Cloud Foundry (us10 region) hosts the ABAP runtime and HANA Cloud, the FastAPI service is packaged as a Docker image, and GitHub manages the source code. In the prototype, the AI microservice and the candidate portal run on the development machine, the portal through the Vite development server and its proxy.

All exchanges use HTTPS and JSON: REST for the AI microservice and OData V4 for the SAP backend.

The end-to-end flow is as follows. A candidate applies through the portal, and the application is stored in SAP. The resume is sent to the AI microservice, whose score is written back to the application. The recruiter reviews the candidate in the Fiori cockpit, moves them through the pipeline, and accepts them. The candidate then completes the onboarding form, and its approval by HR creates the employee master record in a single RAP transaction, without any manual re-entry.

4.1.2 Simplified Backend Architecture

The backend is built with the ABAP RESTful Application Programming Model (RAP) [1] on the SAP BTP ABAP Environment [2], using the *managed scenario with draft*. In this scenario, the developer declares the data model and the business rules, while the SAP framework handles request processing,

transactional buffering, locking, draft storage, and database access. Figure 4.2 shows the four layers of the backend.

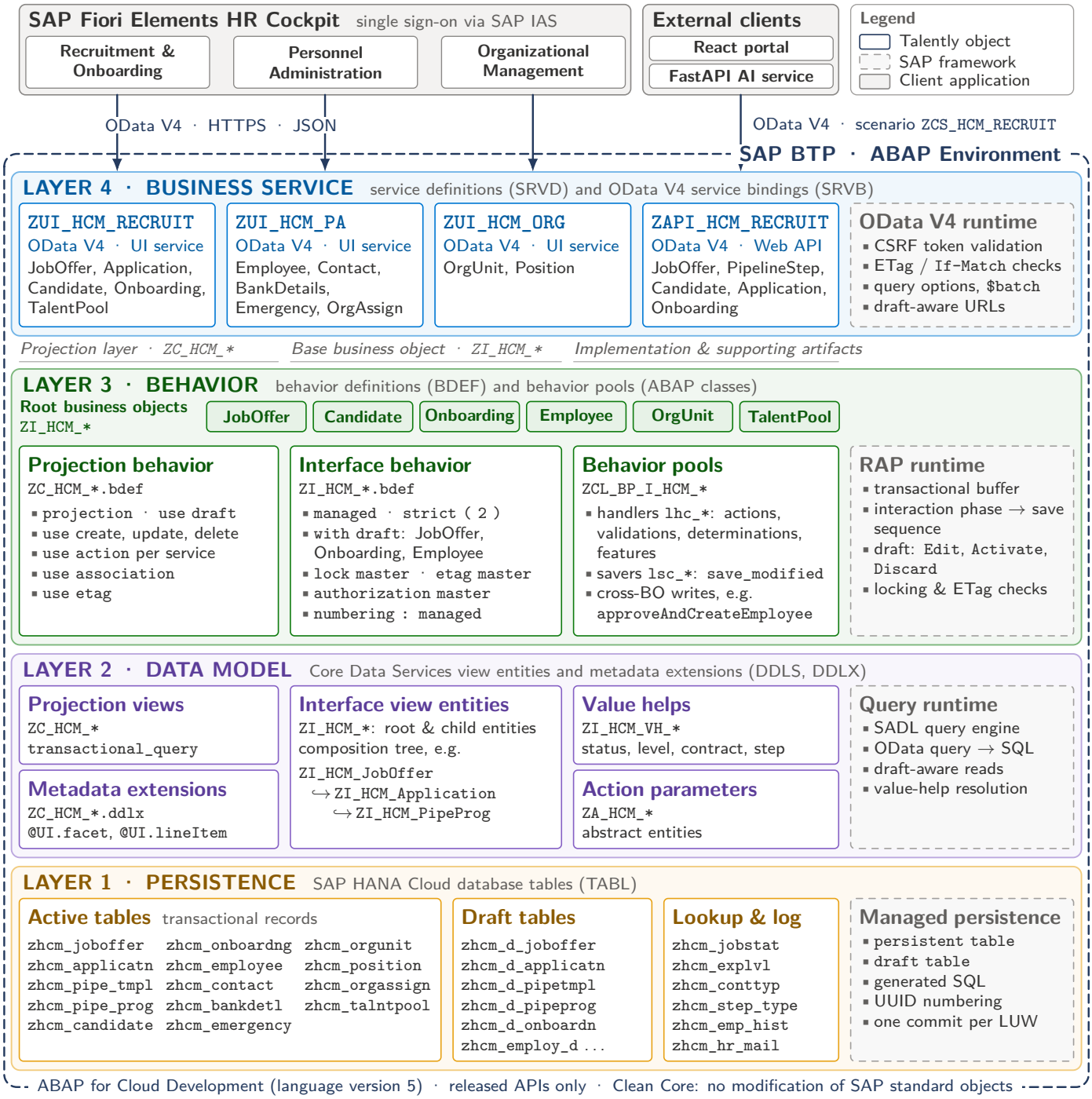


Figure 4.2: ABAP RAP Backend – Four-Layer Architecture

- **Business service layer:** service definitions (SRVD) and OData V4 [7] service bindings (SRVB). Three UI services serve the Fiori cockpit (`ZUI_HCM_RECRUIT`, `ZUI_HCM_PA`, `ZUI_HCM_ORG`), and one Web API, `ZAPI_HCM_RECRUIT`, is published through the communication scenario `ZCS_HCM_RECRUIT` for external integrations. In the prototype, the candidate portal calls the recruitment service

ZUI_HCM_RECRUIT through a development proxy, both for the applications and for writing back the AI results.

- **Behavior layer:** behavior definitions (BDEF) declare, for each business object, its operations, actions, validations, determinations, locking, ETag, and draft handling (`managed`, `strict ( 2 )`, `with draft`). The logic is implemented in behavior pools (`ZCL_BP_I_HCM_*`), split into handler classes (`lhc_*`) and saver classes (`lsc_*`).
- **Data model layer:** Core Data Services (CDS) view entities [3]. Interface views (`ZI_HCM_*`) define the canonical data model and its compositions, projection views (`ZC_HCM_*`) adapt it to each service, and metadata extensions carry the Fiori UI annotations.
- **Persistence layer:** SAP HANA Cloud tables: active tables (`zhcm_joboffer`, `zhcm_applicatn`, `zhcm_employee`, ...), draft tables (`zhcm_d_joboffer`, ...), and lookup and log tables (`zhcm_jobstat`, `zhcm_emp_hist`, ...).

The dashed boxes of Figure 4.2 show what the SAP framework provides at each layer (CSRF and ETag checks, transactional buffer, query engine, SQL generation), so the Talently code contains only business rules. The backend is written in ABAP for Cloud Development using released APIs only, so no SAP standard object is modified.

Business objects. Table 4.1 lists the six business objects of the backend. A child entity is created, locked, and saved together with its root: for example, an application belongs to its job offer, so locking an offer protects its whole recruitment pipeline. The candidate, the onboarding form, and the employee are separate roots because each has its own lifecycle. Draft handling is enabled only for the objects that HR users edit in several steps, and all keys are UUIDs generated by the framework.

Table 4.1: Business Objects of the Talently Backend

Business object	Child entities	Draft	Service
`ZI_HCM_OrgUnit`	`ZI_HCM_Position`	Yes	`ZUI_HCM_ORG`
`ZI_HCM_JobOffer`	`ZI_HCM_PipeTmpl`, `ZI_HCM_Application` → `ZI_HCM_PipeProg`	Yes	`ZUI_HCM_RECRUIT`, `ZAPI_HCM_RECRUIT`
`ZI_HCM_Candidate`	–	No	`ZUI_HCM_RECRUIT`, `ZAPI_HCM_RECRUIT`
`ZI_HCM_Onboarding`	–	Yes	`ZUI_HCM_RECRUIT`, `ZAPI_HCM_RECRUIT`

Business object	Child entities	Draft	Service
ZI_HCM_Employee	ZI_HCM_Contact, ZI_HCM_BankDetl, ZI_HCM_Emergency, ZI_HCM_OrgAssign	Yes	ZUI_HCM_PA
ZI_HCM_TalentPool	–	No	ZUI_HCM_RECRUIT

Main business actions. Table 4.2 summarizes the main actions. Their availability is computed for each instance (feature control): for example, *Publish* is only enabled for a draft or closed offer, and *Accept to Post* only when the candidate has reached the last pipeline step. In addition, validations block inconsistent data before it is saved: mandatory fields, application deadline, vacant position, duplicate applications, and at least one pipeline step before publishing.

Table 4.2: Main Business Actions of the Backend

Action	Business object	Effect
publishOffer, closeOffer	Job offer	Publishes the offer on the portal or closes it.
applyToJob	Candidate	Creates an application at the first pipeline step.
setAIEvaluation, evaluateWithAI	Application	Stores the AI score, recommendation, and summary, or flags the application for a new evaluation (*PENDING*).
passToNextStep	Application	Moves the candidate to the next pipeline step.
acceptToPost	Application	Accepts the candidate, prepares the onboarding form, and closes the offer when all positions are filled.
rejectCandidate	Application	Rejects the candidate and adds them to the talent pool.
approveAndCreateEmployee	Onboarding	Creates the employee master record with contact, bank, emergency, and assignment data.
promoteEmployee, transferEmployee, terminateEmployee	Employee	Records personnel events in the employment history.

Transaction model. RAP executes every request in two phases. During the *interaction phase*, the handler classes apply the changes to an in-memory transactional buffer. During the *save sequence*, all validations run; if they pass, the buffer is written and committed as a single database transaction, otherwise nothing is saved. Because RAP forbids database writes during the interaction phase, the records of other business objects are written by the saver classes during the save sequence, following a *buffer-then-save* pattern. For example, `approveAndCreateEmployee` buffers the form data, and the saver class `lsc_ZI_HCM_ONBOARDING` creates the employee during the save sequence. The hiring is therefore atomic: either the employee record is fully created and the form approved, or nothing changes.

4.1.2.1 Sequence Diagram: Job Offer Management

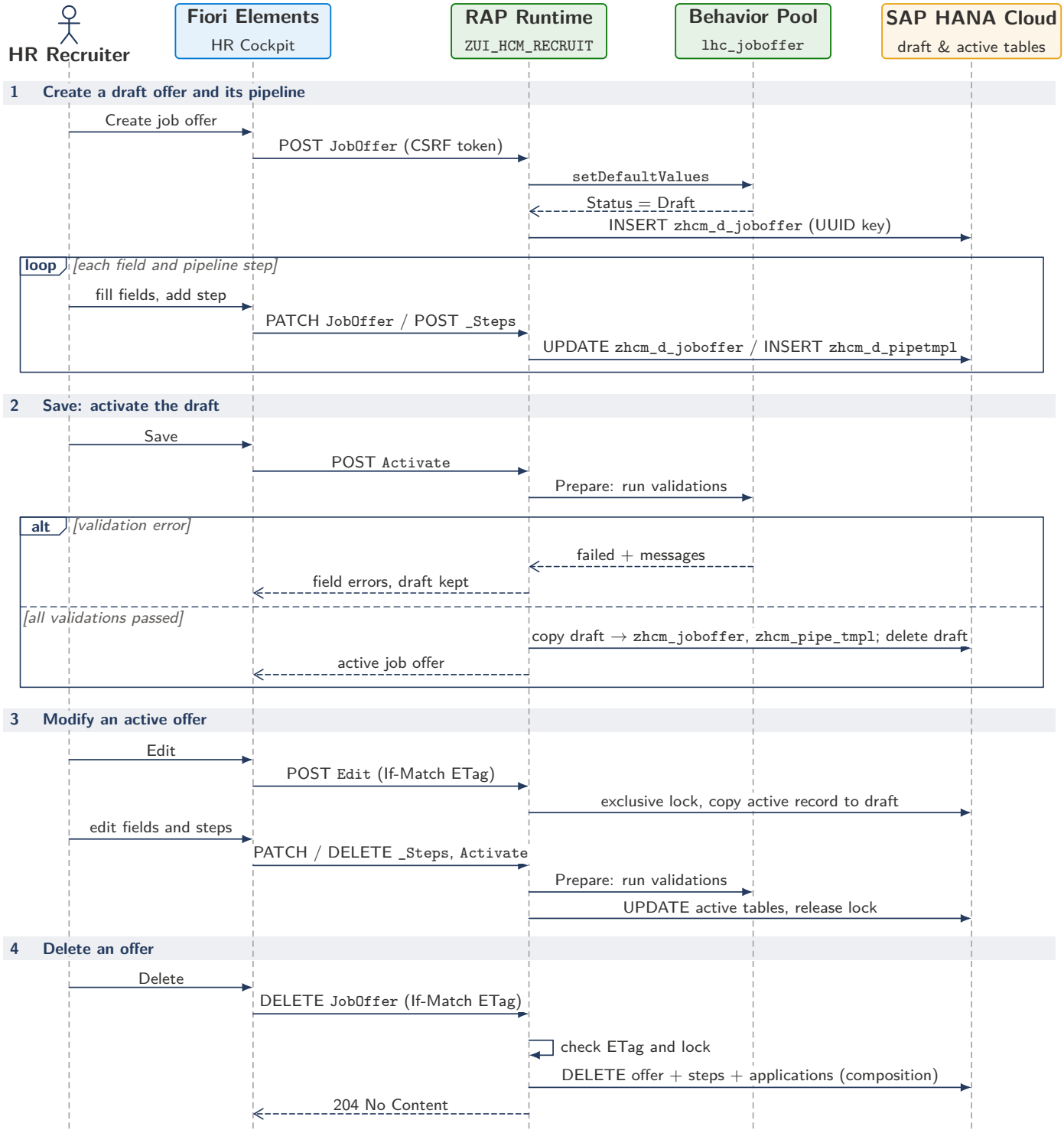


Figure 4.3: Sequence Diagram – Job Offer Management (Create, Modify, Delete)

The diagram follows the four operations of a recruiter on a job offer:

1. **Create:** the RAP runtime creates a draft with a UUID key and the status *Draft*; the fields and pipeline steps are saved automatically in the draft tables.

2. **Save:** the *Activate* action runs the validations. On error, the draft is kept; otherwise, the offer and its steps are copied to the active tables in one transaction.

3. **Modify:** the *Edit* action locks the offer and copies it into a new draft, so two recruiters cannot change it at the same time.

4. **Delete:** protected by the ETag, it removes the offer together with its pipeline steps and applications (composition).

4.1.3 Simplified AI Agent Architecture

The AI evaluation engine is a stateless Python microservice built with FastAPI. It is kept outside SAP so that slow or unpredictable LLM processing never affects ERP transactions. It receives a resume together with the target job offer, and returns a structured, explainable score. Figure 4.4 shows its processing pipeline.

The pipeline runs in six stages:

1. **Input:** the resume is received as a Google Drive link or an uploaded file (PDF, DOCX, TXT, or MD), together with the job offer description, through the REST endpoints `/evaluate/score` and `/evaluate/full`.

2. **Text extraction:** the document is downloaded from Google Drive (or received as a file) and converted into clean Markdown (PyMuPDF for PDF, python-docx for Word).

3. **Structured parsing:** six LLM calls extract the basics, work experience, education, skills, projects, and awards into the standard JSON Resume model [11], validated with Pydantic.

4. **GitHub enrichment:** the candidate's GitHub profile is detected automatically. The GitHub API [13] provides the repositories and commit statistics, each project is classified as open source or personal, the LLM selects the seven most relevant ones, and their README quality is checked.

5. **Rubric scoring:** GPT-4o-mini (temperature 0.1) evaluates the candidate against the rubric of the target role, defined in Jinja2 templates, and against the text of the job offer. The text sent to the model contains no personal identifiers: the name, contact details, location, institution, and grades are left out. The model must answer in a strict JSON schema [10] with evidence for each category, key strengths, and areas for improvement.

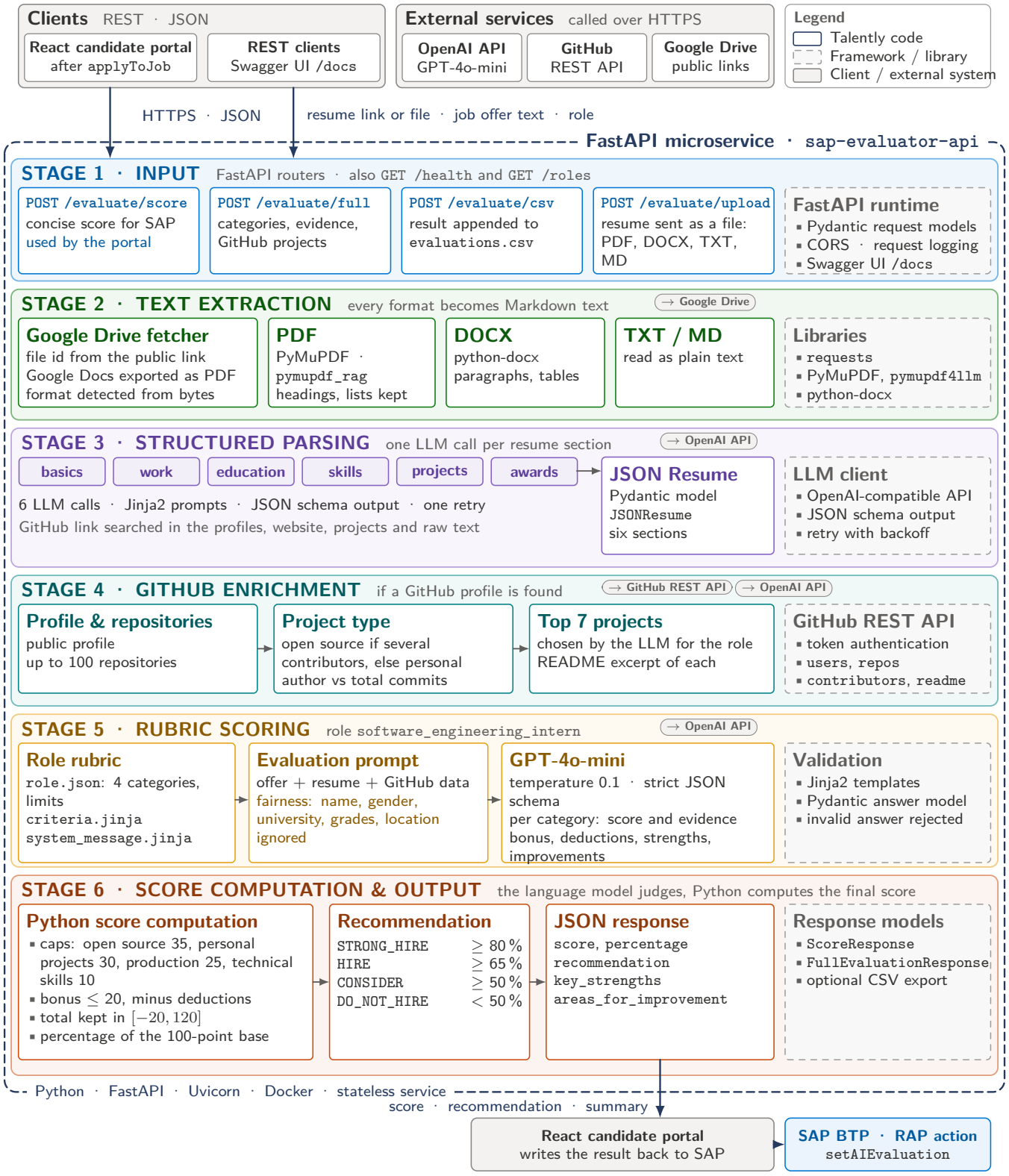


Figure 4.4: AI Evaluation Microservice – Resume Scoring Pipeline

6. **Score computation and write-back:** the final score is computed in Python (category caps, bonus, deductions, and a total bounded to $[-20, 120]$), a recommendation is derived, and the result is returned to the portal, which writes it to the application in SAP through the RAP action `setAIEvaluation`.

Table 4.3: Evaluation Rubric of the Software Engineering Intern Role

Criterion	**Points**
Open source contributions	0 – 35
Personal projects	0 – 30
Production experience	0 – 25
Technical skills	0 – 10
Bonus (e.g., Google Summer of Code, startup experience, portfolio)	up to +20
Deductions (e.g., tutorial-only projects, projects without links or README)	$-N$
Recommendation: **STRONG_HIRE** ($\geq 80\,\%$), **HIRE** ($\geq 65\,\%$), **CONSIDER** ($\geq 50\,\%$), **DO_NOT_HIRE** ($< 50\,\%$).	

Two design choices make the evaluation trustworthy. First, the LLM judges but Python calculates: the final number is computed deterministically from the LLM's structured answer, so it always respects the rubric limits. Second, the evaluation is designed to be fair: the prompt forbids using the candidate's name, gender, university, grades, or location, and these details are not even sent to the scoring model. This second measure was added after the fairness test presented in Chapter 5. The score only supports the recruiter, who keeps the final decision in the Fiori cockpit.

4.1.3.1 Sequence Diagram: Candidate Application and AI Evaluation

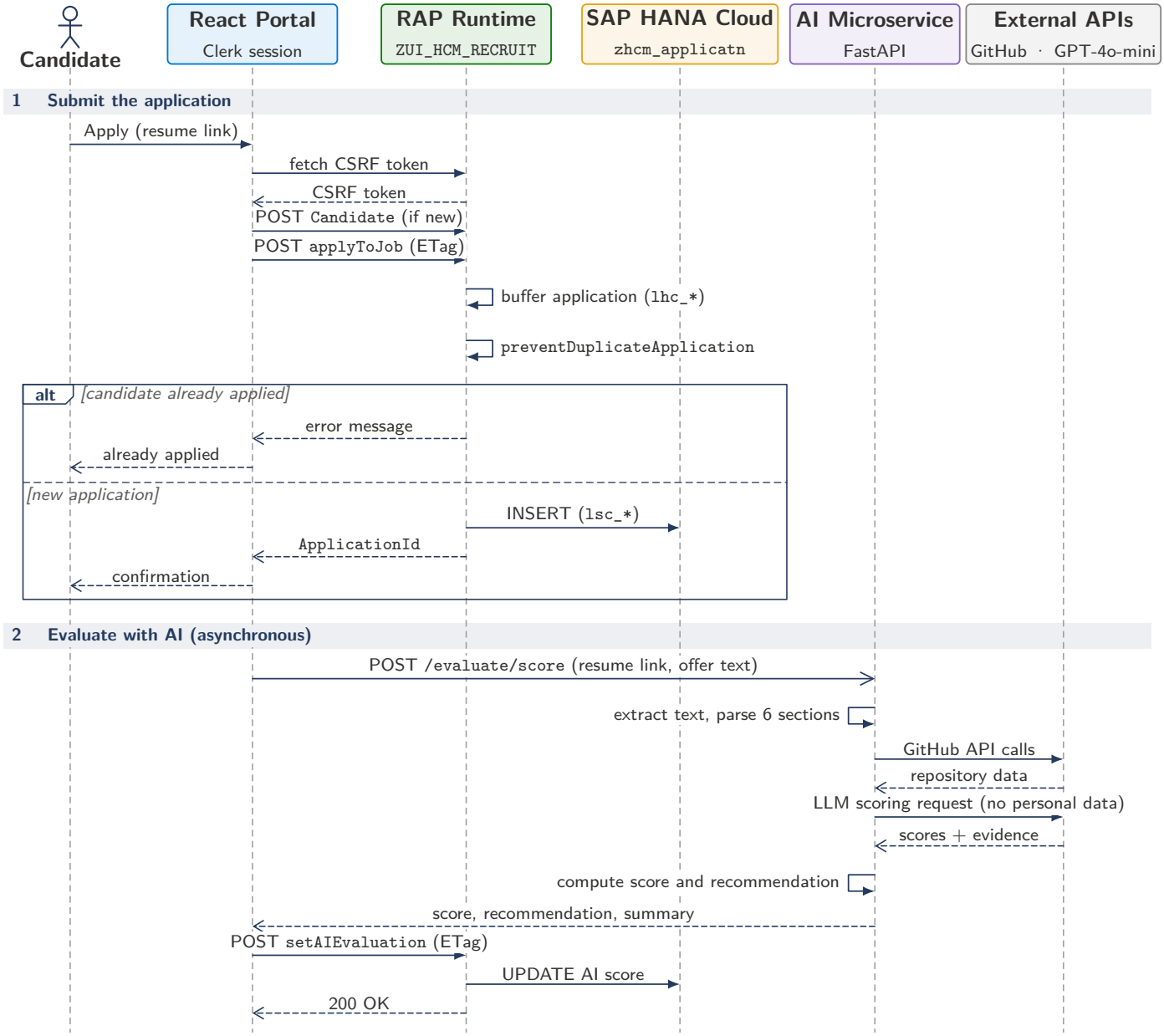


Figure 4.5: Sequence Diagram – Candidate Application and AI Evaluation

The scenario runs in two phases:

1. **Application:** the portal obtains a CSRF token and calls `applyToJob` on the service `ZUI_HCM_RECRUIT`. The application is inserted only if the duplicate-application check passes.

2. **AI evaluation:** the portal sends the resume link and the offer text to the AI microservice asynchronously, so the candidate does not wait. The service parses the resume, queries GitHub

and GPT-4o-mini, and computes the score; the portal then writes the result back through `setAIEvaluation`, where it becomes visible to the recruiter in the Fiori cockpit.

4.1.4 UI/UX Expectations

Talently has two interfaces for two audiences. The candidate portal is a free React design focused on simplicity and self-service, while the HR cockpit follows the SAP Fiori design guidelines [5]: its screens are generated by SAP Fiori Elements [4] from the annotations of the CDS projection views, using the standard *List Report* and *Object Page* floorplans. Figure 4.6 shows the navigation of both interfaces.

```
Candidate Portal (React 19, React Router)
/                     Job Board: published offers, search and filters
|-- /jobs/:id         Job Details: description, skills, pipeline steps, apply form
|-- /applications     My Applications: offers applied to, progress tracking
|-- /onboarding/:id   Onboarding Form: personal, legal, bank and emergency data
|-- /talent-pool      Talent Pool: non-selected candidates (HR role only)
`-- *                 Fallback page
Sign-in / Sign-up     Clerk modal, available from every page

HR Cockpit (SAP Fiori Elements: List Report -> Object Page)
Recruitment & Onboarding  (ZUI_HCM_RECRUIT)
|-- Job Offers -> Job Offer: General Information, Pipeline Steps, Applications
|   `-- Application: details, AI score, Pipeline Progress
|-- Onboarding Forms -> Onboarding Form: personal, bank and legal data
`-- Talent Pool
Personnel Administration  (ZUI_HCM_PA)
`-- Employees -> Employee: Contact, Bank Details, Emergency, Assignment
Organizational Management (ZUI_HCM_ORG)
`-- Organizational Units -> Organizational Unit: Positions
```

Figure 4.6: Application Routing Tree of the Candidate Portal and the HR Cockpit

The main screens were first sketched as low-fidelity wireframes with Excalidraw, which fixed the layout of each screen and the information it shows. The sketches were then refined into higher-fidelity mockups. Figure 4.7 shows the mockup of the job board of the candidate portal, with its four zones: navigation header, search and filter bar, filter sidebar, and job card. Figure 4.8 shows the mockup of the job offers list of the HR cockpit, built on the standard Fiori List Report floorplan: header, filter bar, table toolbar with the lifecycle actions, and data table.

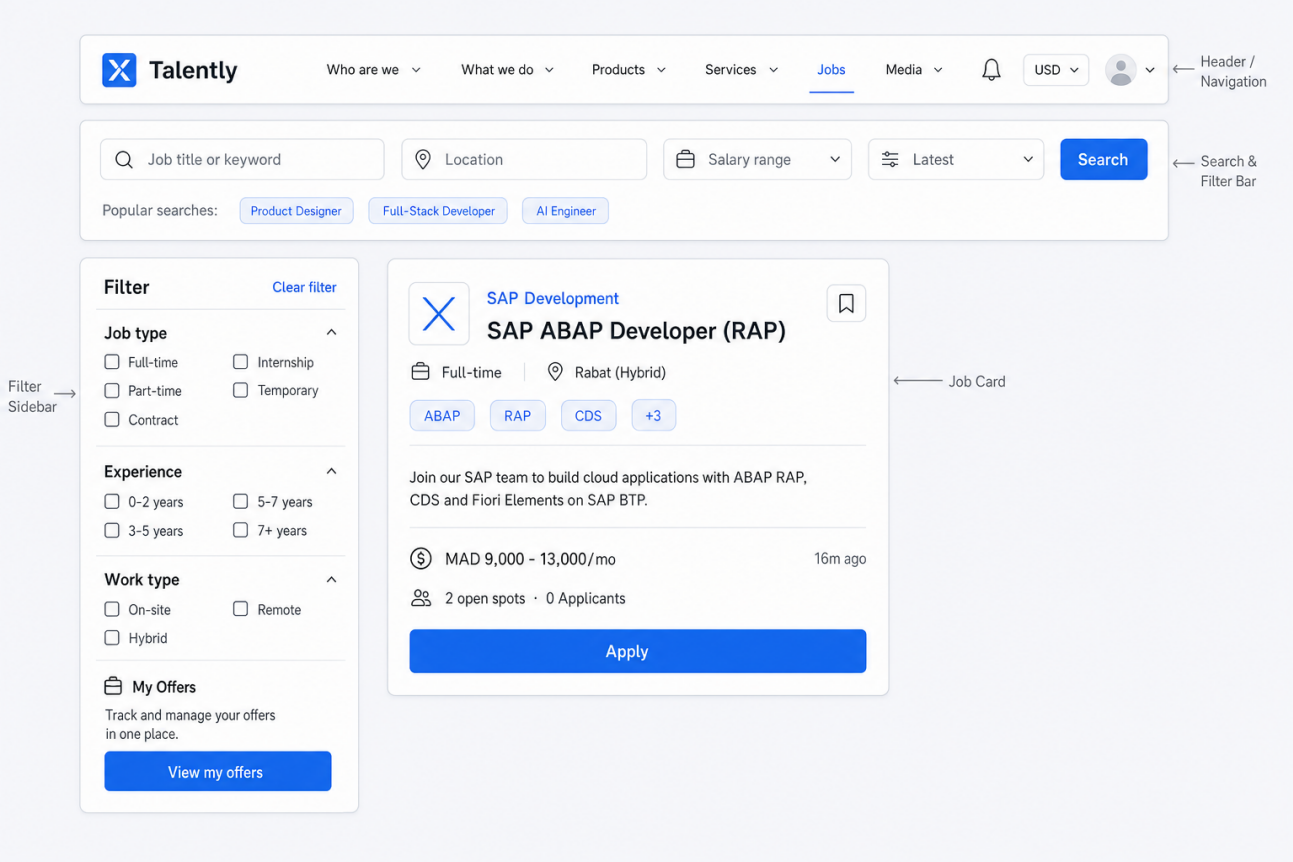


Figure 4.7: Mockup of the Job Board of the Candidate Portal

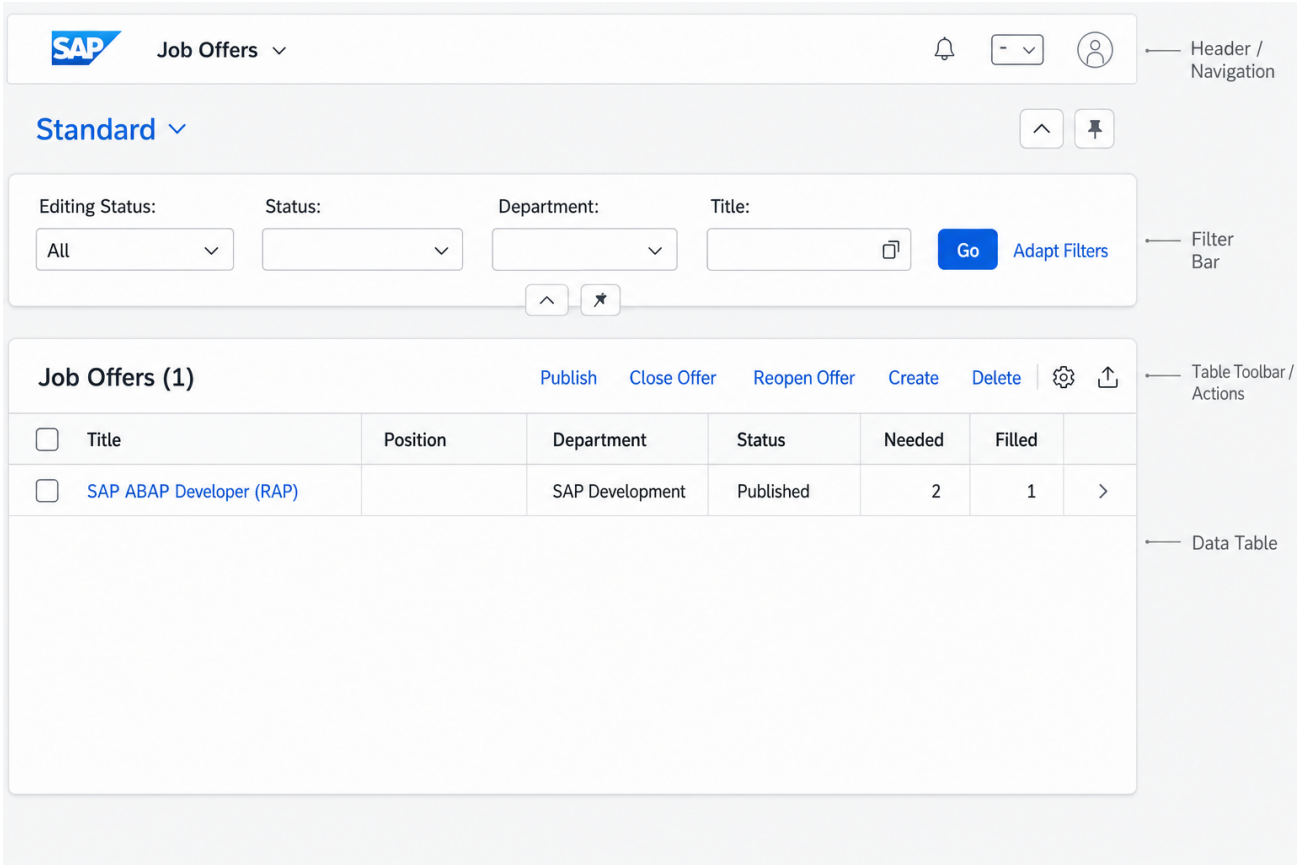


Figure 4.8: Mockup of the Job Offers List of the HR Cockpit

4.2 Detailed Design

4.2.1 Class Diagram

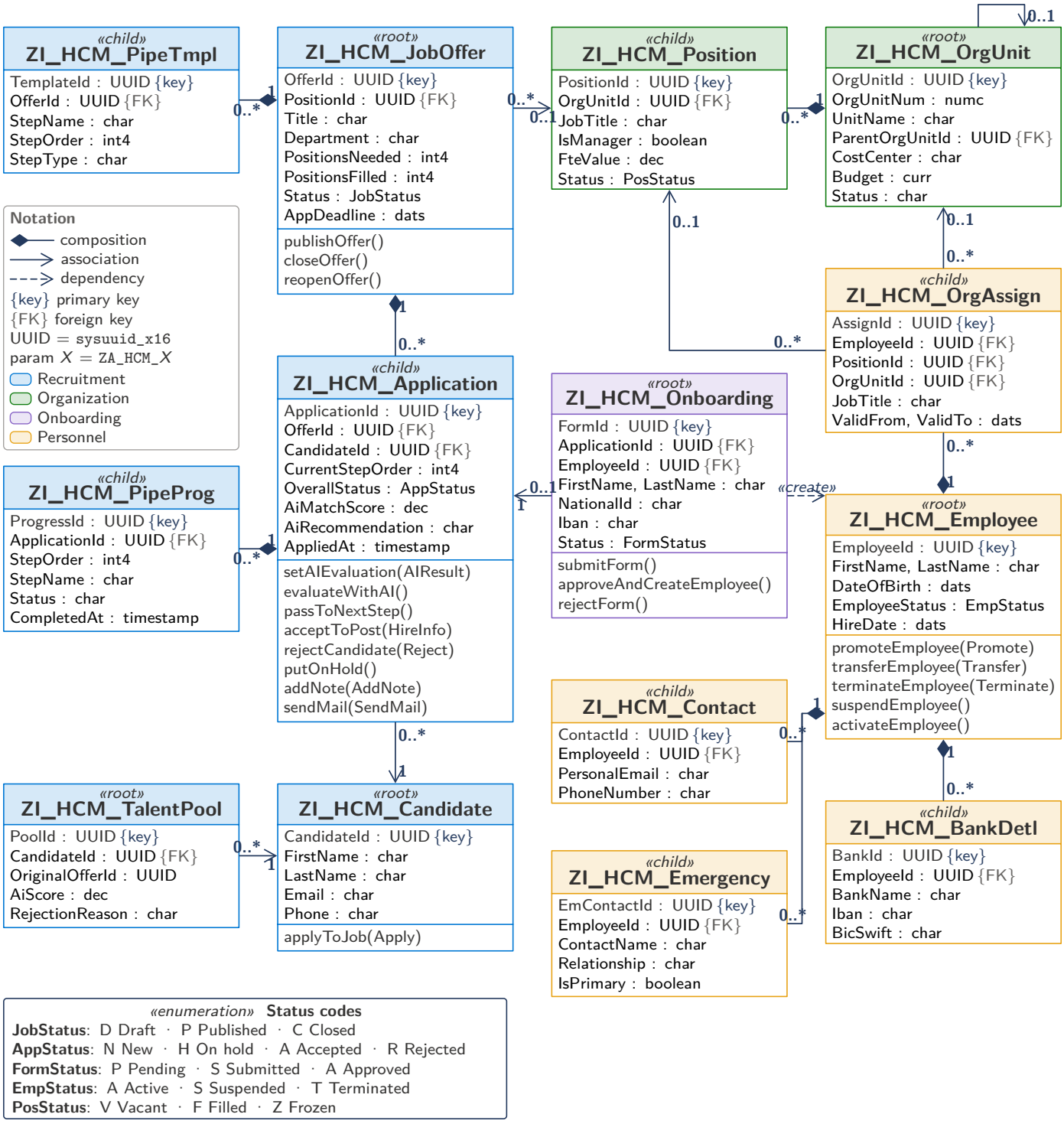


Figure 4.9: Class Diagram of the Talently Data Model (CDS Interface Views)

Each class is a CDS interface view (`ZI_HCM_*`) showing its main fields and its RAP actions, colored by domain. Three types of relationships are used:

- **Composition** (filled diamond): the child is created, locked, and deleted with its parent (1 to 0..*): job offer to pipeline steps and applications, application to progress records, organizational unit to positions, and employee to contact, bank, emergency, and assignment records.
- **Association** (arrow): a link between independent objects: application to candidate (0..* to 1), onboarding form to application (0..1 to 1), job offer to position (0..* to 0..1), talent-pool entry to candidate, assignment to position and organizational unit, and organizational unit to its parent unit (0..1).
- **Dependency** (dashed arrow): the onboarding form creates the employee through `approveAndCreateEmployee`.

All keys are UUIDs, the {FK} fields implement the associations, and the status codes are listed in the enumeration box.

Conclusion

This chapter presented the design of Talently. The global architecture separates the SAP BTP backend, which holds all business data and rules, from the AI microservice and the two user interfaces. The RAP backend was described through its layers, business objects, actions, and transaction model, and the AI engine through its six-stage pipeline and scoring rubric. The sequence diagrams, the routing tree, and the class diagram detailed the main scenarios and the data model. The next chapter presents the tools used to build the solution, demonstrates it end to end, and evaluates it.

Chapter 5

Implementation, Testing, and Results

Introduction

This chapter presents the realization of Talently. It first introduces the tools and technologies used to build the platform. It then demonstrates the solution through a complete end-to-end scenario, from the creation of the organizational structure to the creation of a new employee. Finally, it presents the tests carried out and validates the functional and non-functional requirements defined in Chapter 3.

5.1 Development Environment

The platform combines the SAP cloud ecosystem, used for the transactional core and the HR cockpit, with open-source web and AI technologies, used for the candidate portal and the evaluation engine.

5.1.1 SAP Cloud Ecosystem

- **SAP BTP ABAP Environment**



Figure 5.1: SAP Business Technology Platform Logo

SAP Business Technology Platform (BTP) is SAP's cloud platform for building extensions around SAP solutions. Its ABAP Environment provides a managed ABAP runtime in the cloud, restricted to the ABAP Cloud development model and to released APIs. In Talently, it hosts the whole backend: the CDS data model, the behavior definitions, the business logic, and the OData V4 services. It was chosen because it allows the HCM extension to be built side by side with the ERP core, in line with SAP's Clean Core strategy.

- **SAP HANA Cloud**



Figure 5.2: SAP HANA Cloud Logo

SAP HANA Cloud is the in-memory, column-oriented database provided with the ABAP Environment. It stores the active and draft tables of all Talently business objects. Its in-memory processing gives fast reads for the Fiori list reports, and it is fully managed by the platform, so no database administration was required.

- **Eclipse with ABAP Development Tools (ADT)**



Figure 5.3: Eclipse and ABAP Development Tools Logo

ABAP Development Tools (ADT) is the official Eclipse-based environment for ABAP Cloud development. It was used to create and activate all development objects (tables, CDS views, behavior definitions, classes, service definitions, and service bindings), to preview the Fiori applications, and to debug the business logic. The sources were versioned with abapGit [8], which serializes the ABAP objects into files stored in the GitHub repository.

- **SAP Fiori Elements**



Figure 5.4: SAP Fiori Logo

SAP Fiori Elements generates enterprise user interfaces from OData services and CDS annotations, using predefined floorplans such as the List Report and the Object Page. It was used to build the whole HR cockpit without writing user interface code: the layouts, tables, action buttons, and draft handling are derived from the metadata extensions of the projection views. This guarantees a consistent SAP user experience and greatly reduces development time.

- **SAP Cloud Identity Services**

SAP Cloud Identity Services (Identity Authentication Service, IAS) manages the identities of the HR users. It provides single sign-on to the Fiori cockpit and is the trusted identity provider of the ABAP Environment, so internal users never share the authentication mechanism of the external candidates.

5.1.2 Web, AI and DevOps Tooling

- **React 19 and Vite**



Figure 5.5: Vite Logo

React is the most widely used JavaScript library for building component-based user interfaces, and Vite is a fast build tool and development server [14]. Together, they were used to build the candidate portal: job board, job details, application form, progress tracking, and onboarding form. React was chosen for its large ecosystem and component reuse, and Vite for its near-instant reload during development and its optimized production build.

- **Tailwind CSS**



Figure 5.6: Tailwind CSS Logo

Tailwind CSS is a utility-first CSS framework. It was used to style the portal directly inside the components and to make every page responsive, without maintaining separate style sheets.

- **Clerk**



Figure 5.7: Clerk Logo

Clerk is an authentication service for web applications [15]. It provides the sign-in and sign-up screens, social login, and session management of the candidate portal. Delegating candidate authentica-

tion to Clerk keeps external identities separate from the SAP identity provider and avoids implementing password management.

- **Python 3.12 and FastAPI**



Figure 5.8: FastAPI Logo

FastAPI is a modern Python framework for building REST APIs [9]. It validates every request and response with Pydantic models and automatically generates interactive documentation (Swagger). It was chosen for the AI microservice because Python offers the richest ecosystem for document processing and LLM integration, and because FastAPI is fast, lightweight, and easy to test.

- **OpenAI GPT-4o-mini**



Figure 5.9: OpenAI Logo

GPT-4o-mini is a compact large language model from OpenAI. It parses the resume sections, selects the most relevant GitHub projects, and scores the candidate against the rubric. It was chosen for its good reasoning quality at a low cost per evaluation and for its support of structured outputs, which forces the model to answer in a strict JSON schema [10].

- **PyMuPDF and python-docx**



Figure 5.10: PyMuPDF Logo

PyMuPDF [12] and python-docx are Python libraries for reading PDF and Word documents. They

convert the submitted resumes into clean Markdown text, preserving headings and lists, before the text is sent to the language model.

- **Docker**



Figure 5.11: Docker Logo

Docker packages an application with all its dependencies into a portable container. The AI microservice is delivered as a Docker image, so it runs identically on the development machine and on any server, independently of the local Python installation.

- **Git and GitHub**



Figure 5.12: GitHub Logo

Git was used for version control and GitHub as the central repository of the three components of the project: the ABAP sources (through abapGit), the AI microservice, and the React portal. GitHub also kept the full history of every change.

5.2 End-to-End Walkthrough

This section demonstrates Talently through a complete scenario executed on the SAP BTP ABAP trial system. Two actors take part: the HR administrator, who works in the SAP Fiori applications of the HR cockpit, and the candidates, who use the React portal. The test data are three organizational units (Information Technology, SAP Development, and Data & AI), three vacant positions in SAP Development, one job offer *SAP ABAP Developer (RAP)* for two positions with a three-step pipeline,

and two candidates with different profiles: Salma Idrissi, a junior developer with a first RAP experience, and Omar Tazi, a sales assistant without IT experience. The journey is the following:

1. **Organization:** the HR administrator models the organizational units and their vacant positions.
2. **Job offer:** the HR administrator creates the job offer, defines its pipeline, and publishes it.
3. **Application:** the candidates find the offer on the portal, sign in, and apply.
4. **AI evaluation:** the AI engine scores each resume, and the HR administrator reviews the applications.
5. **Decision:** the HR administrator moves the candidates through the pipeline, accepts one, and rejects the other.
6. **Onboarding:** the accepted candidate completes the onboarding form, and the HR administrator reviews it.
7. **Employee administration:** the approval creates the employee, who is then managed in the Personnel Administration application.

Each screenshot is followed by a description of the actions performed by the user and of the options available on the screen.

5.2.1 Organizational Structure Setup

The organization is modeled by the HR administrator in the *Organizational Management* application, exposed by the service binding `ZUI_HCM_ORG`.

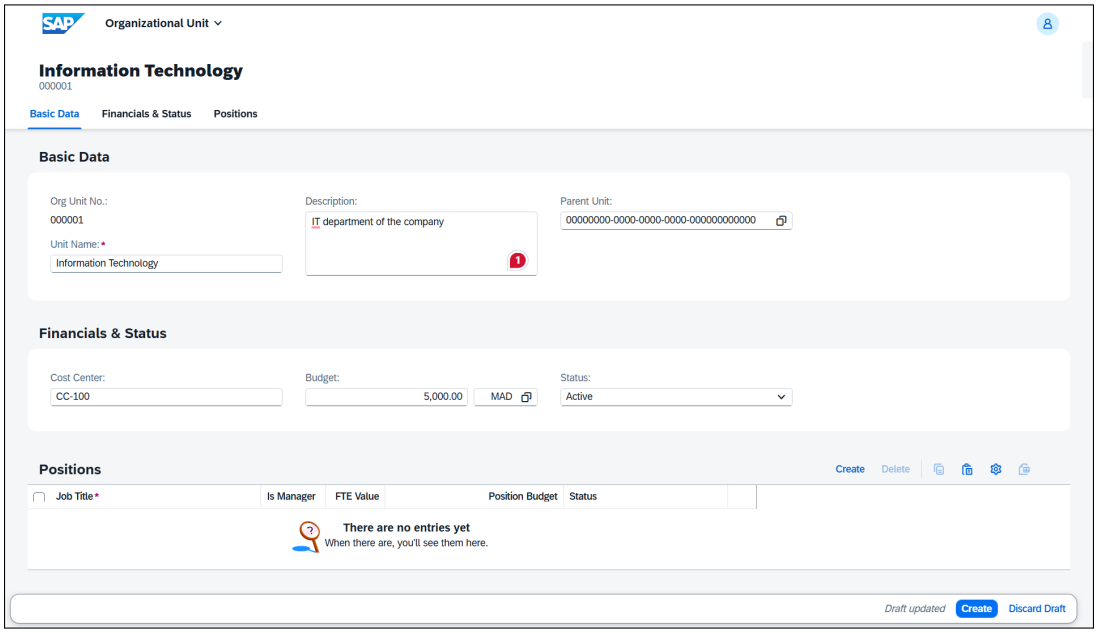


Figure 5.13: Creation of an Organizational Unit in Draft Mode

The HR administrator opens the Organizational Management application and clicks *Create* to add the first unit, Information Technology (Figure 5.13). The object page opens in draft mode and has three sections, reached from the tabs at the top: *Basic Data*, *Financials & Status*, and *Positions*. The unit number (000001) is assigned automatically. The user enters the unit name, which is mandatory, a description, the cost center (CC-100), the budget with its currency (5,000 MAD), and the status, and can select a parent unit with the value help. Every change is kept in the draft, as the *Draft updated* indicator shows, so no data is lost. The user saves the unit with the *Create* button at the bottom of the page or cancels it with *Discard Draft*.

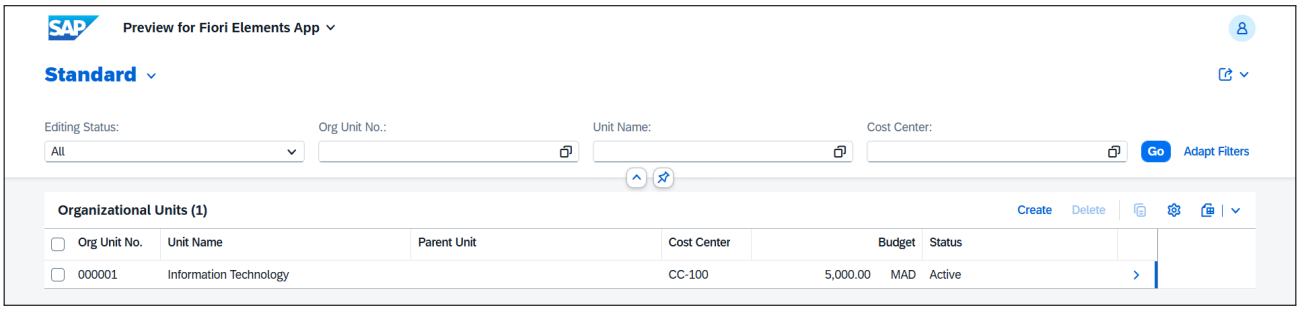


Figure 5.14: Organizational Units List After the First Creation

After saving, the HR administrator returns to the list report, which now shows the Information Technology unit with its number, cost center, budget, and status (Figure 5.14). The filter bar at the top searches the units by editing status, unit number, unit name, or cost center: the user fills the filters and clicks *Go*, and *Adapt Filters* adds more filter fields. Above the table, *Create* adds a new unit and

Delete removes the selected units. The settings icon changes the visible columns, and the export icon exports the list to a spreadsheet. The arrow at the end of a row opens the unit.

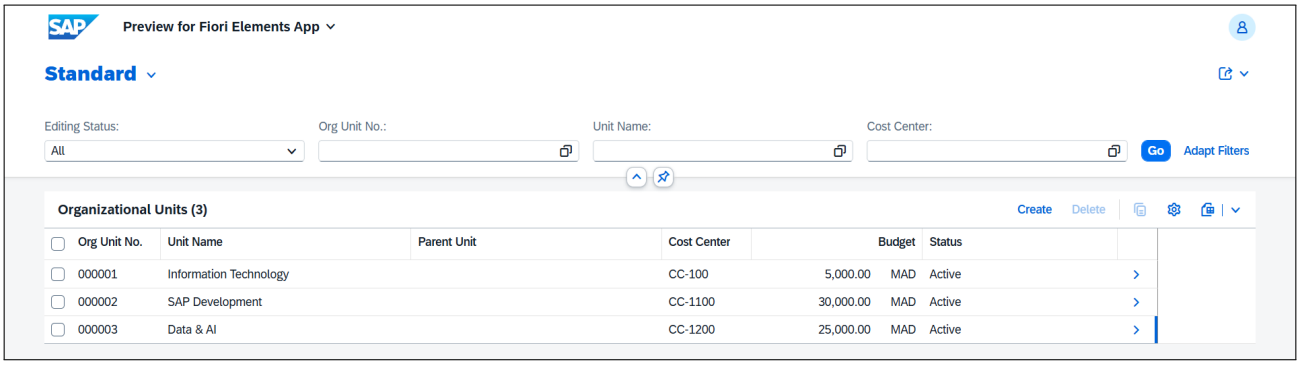


Figure 5.15: List of the Three Organizational Units

The HR administrator repeats the same steps to create the units SAP Development (cost center CC-1100, budget 30,000 MAD) and Data & AI (cost center CC-1200, budget 25,000 MAD). The list report now contains the three units, all with the status *Active* (Figure 5.15). At this stage, the units are still independent: the *Parent Unit* column is empty, and the hierarchy is built in the next step by editing the two sub-units.

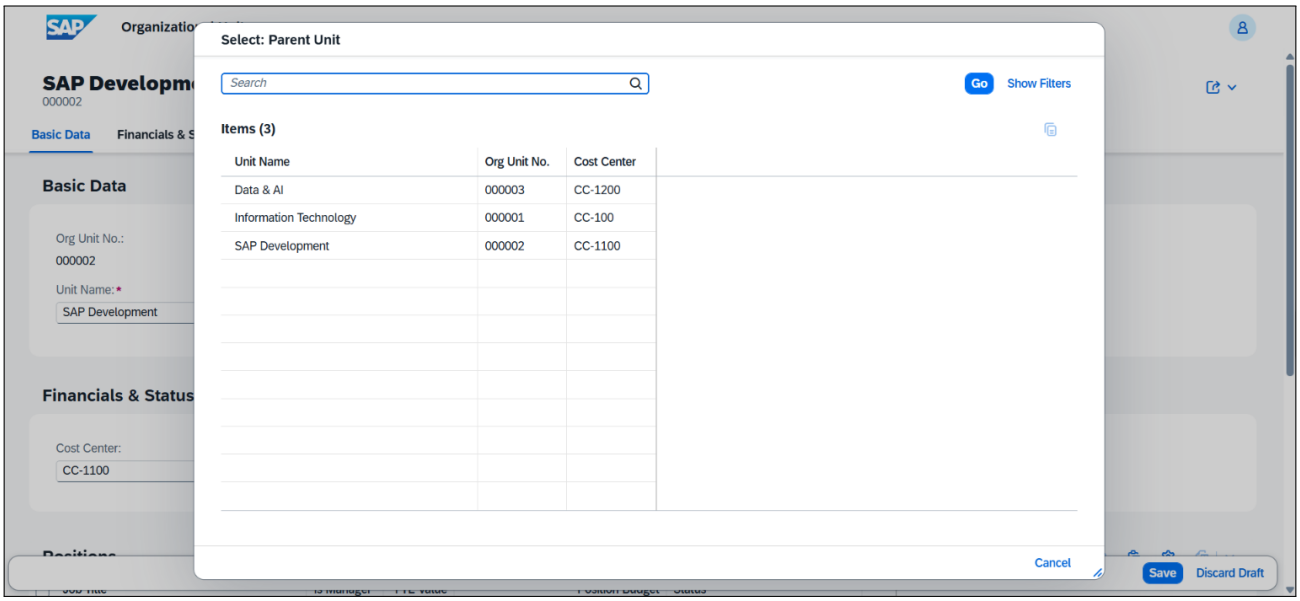


Figure 5.16: Value Help of the Parent Unit

To build the hierarchy, the HR administrator opens SAP Development, clicks *Edit*, and opens the value help of the *Parent Unit* field (Figure 5.16). A dialog lists the existing units with their name, number, and cost center. The user can search a unit with the search field and *Go*, display more filters with *Show Filters*, select the parent by clicking its row, or close the dialog with *Cancel*. Information Technology is selected as the parent of SAP Development, and the same is done for Data & AI.

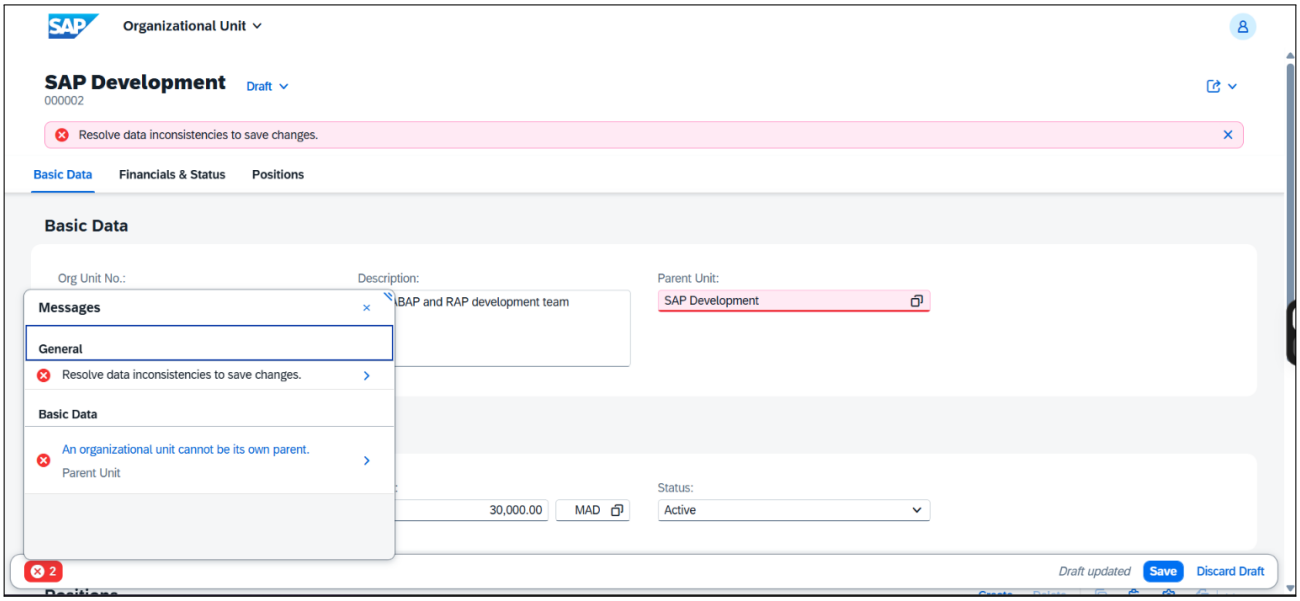


Figure 5.17: Validation Error on the Parent Unit

The business rules are checked when the draft is saved. If the HR administrator selects SAP Development as its own parent and clicks *Save*, the backend validation blocks the save (Figure 5.17). The wrong field is highlighted in red, a banner asks to resolve the data inconsistencies, and the message popover, opened from the red counter at the bottom left, lists the errors, here *An organizational unit cannot be its own parent.* Clicking a message leads to the related field, so the user corrects the value and saves again.

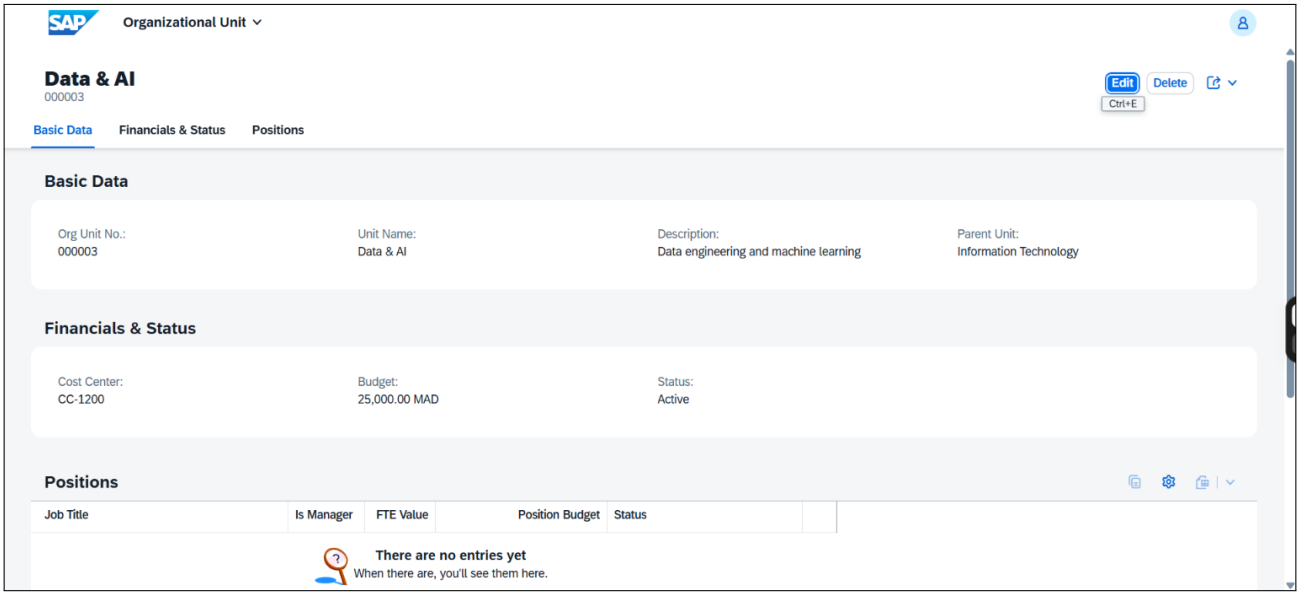


Figure 5.18: Data & AI Unit in Display Mode

Once saved, the unit is displayed in read-only mode (Figure 5.18). The Data & AI unit now shows Information Technology as its parent unit, together with its description, cost center, budget, and status.

In display mode, the header offers two buttons: *Edit* (shortcut Ctrl+E), which opens a new draft of the unit, and *Delete*, which removes it. The *Positions* section is still empty, because positions are added in edit mode.

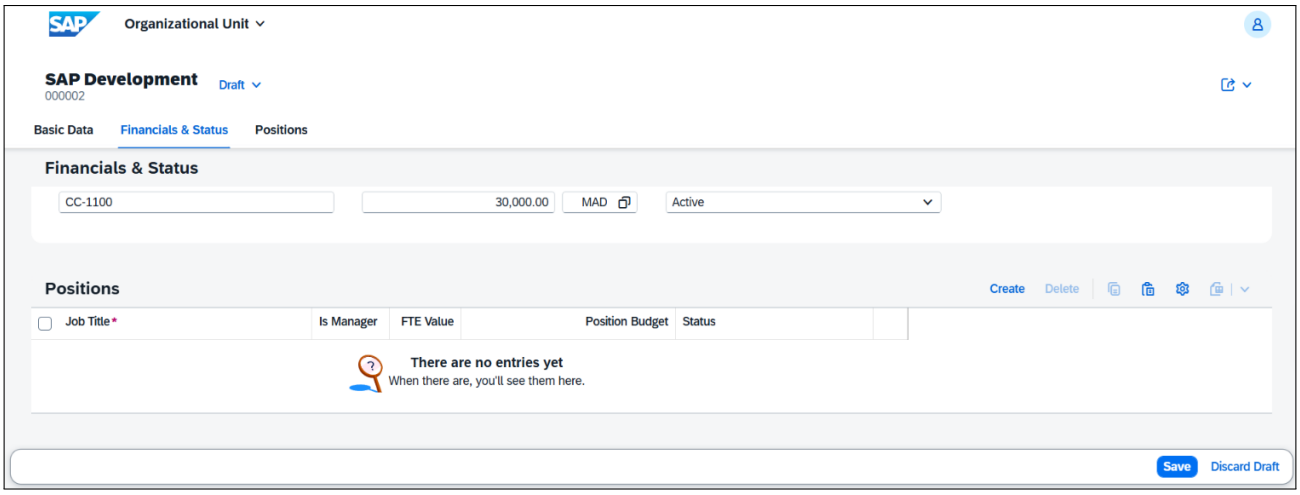


Figure 5.19: Positions Section of SAP Development in Edit Mode

The HR administrator then opens SAP Development and clicks *Edit* to create its positions (Figure 5.19). In edit mode, the *Positions* table offers the *Create* button, which adds a new position, and *Delete*, which removes the selected positions. For each position, the table shows the job title, which is mandatory, the manager flag, the full-time equivalent (FTE), the position budget, and the status. The unit stays in draft until the user saves it.

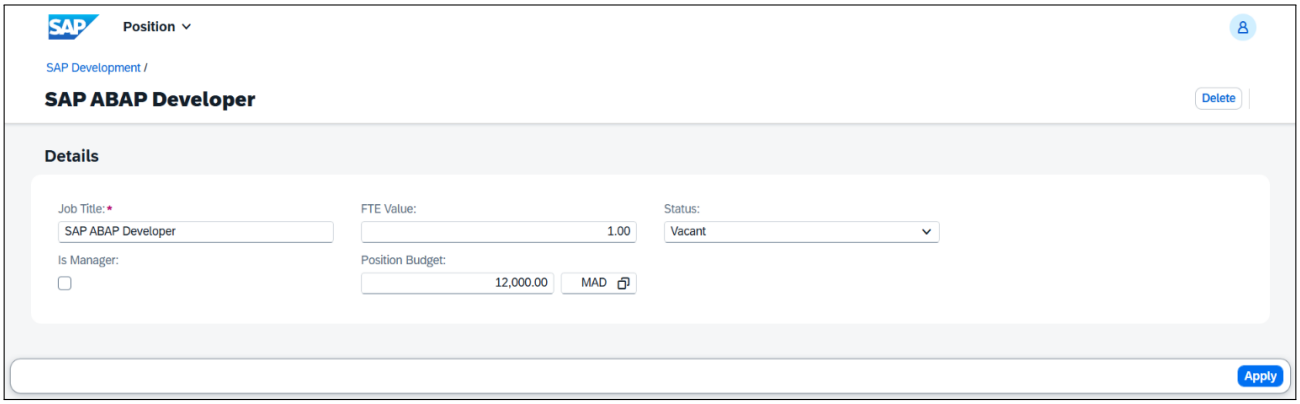


Figure 5.20: Details of a Position

Each position also has its own details page, linked to its unit by the breadcrumb *SAP Development* at the top (Figure 5.20). The HR administrator enters the job title *SAP ABAP Developer*, the FTE value (1.00), and the position budget (12,000 MAD), selects the status *Vacant* in the drop-down list, and can tick *Is Manager* for a management position. The *Apply* button returns to the unit with the new position, and *Delete* removes the position.

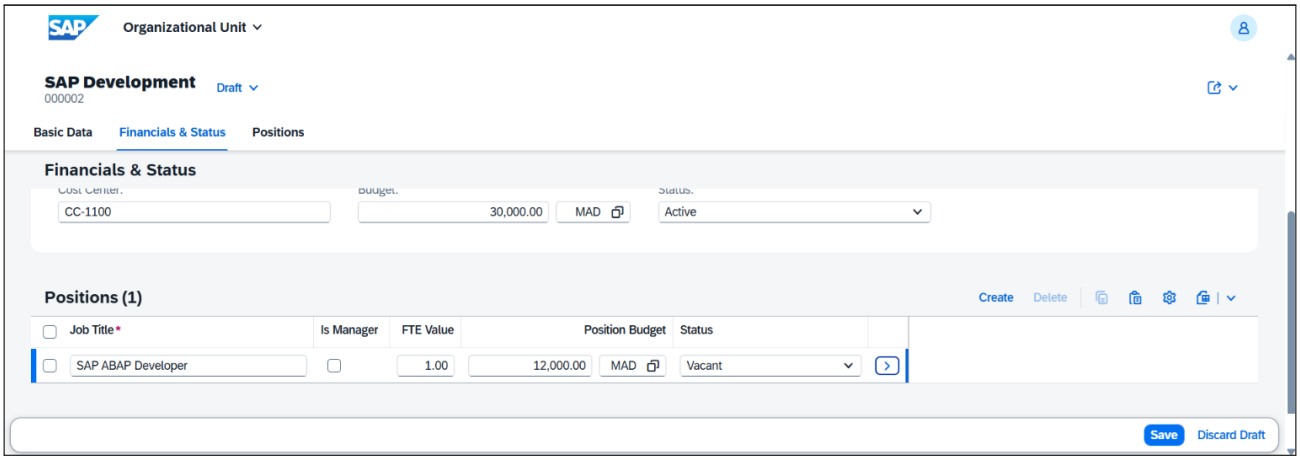


Figure 5.21: First Position in the Positions Table

After *Apply*, the position appears as a row of the *Positions* table of SAP Development, where its fields can still be edited directly (Figure 5.21). The unit is still a draft, as the *Draft* label next to its name shows. The HR administrator can add more positions with *Create* and confirms all the changes with *Save* at the bottom of the page, or cancels them with *Discard Draft*.

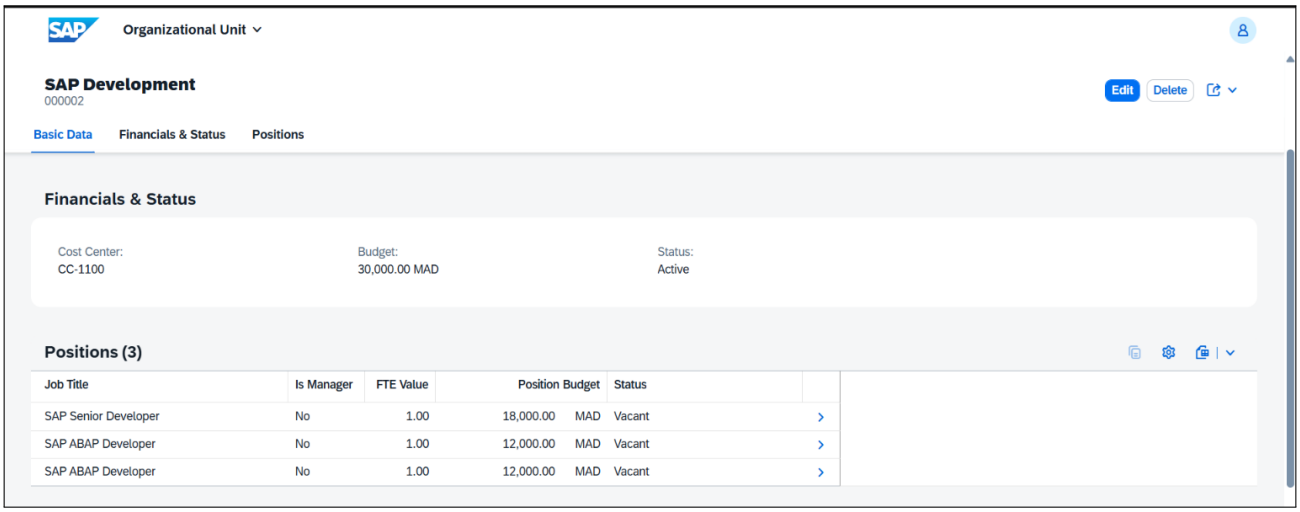


Figure 5.22: Positions of the SAP Development Unit

The HR administrator adds a second *SAP ABAP Developer* position and one *SAP Senior Developer* position (budget 18,000 MAD), then clicks *Save* (Figure 5.22). The unit returns to display mode with its three positions, all with the status *Vacant*. The two SAP ABAP Developer positions are the ones to fill through the recruitment, and the senior position can later be used for a promotion. The *Edit* and *Delete* buttons are available again in the header.

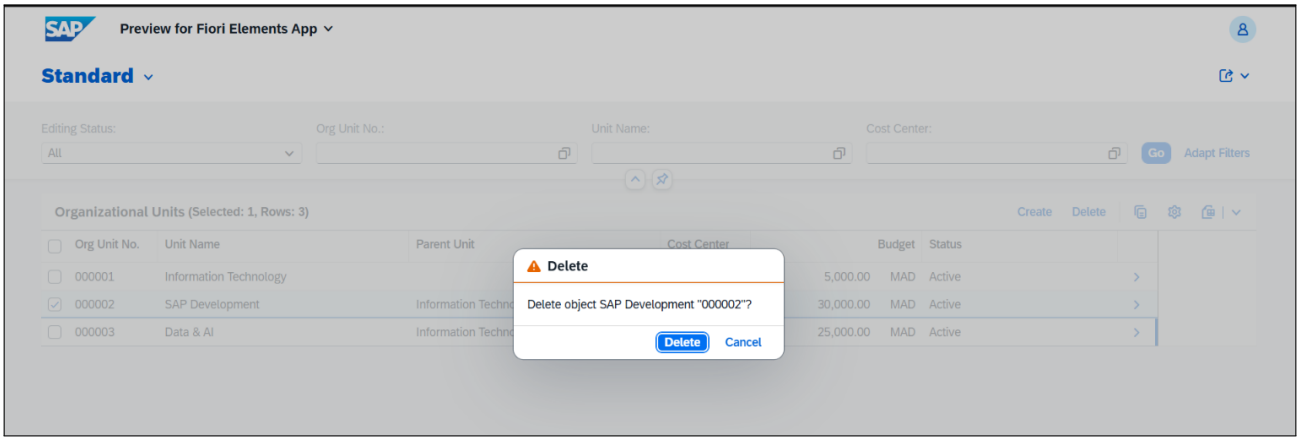


Figure 5.23: Deletion Request of an Organizational Unit

Back in the list report, the *Parent Unit* column now shows Information Technology for the two sub-units. To test the integrity rules, the HR administrator selects SAP Development and clicks *Delete* (Figure 5.23). Before anything is deleted, a dialog asks for a confirmation and shows the name and number of the unit. The user confirms with *Delete* or stops with *Cancel*.

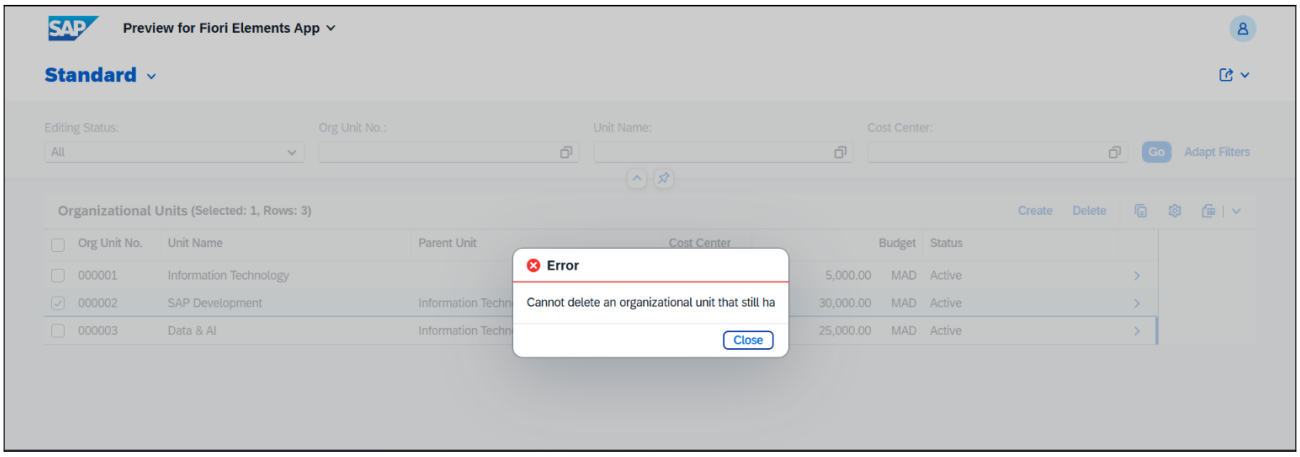


Figure 5.24: Deletion Refused for a Unit That Still Has Positions

After the confirmation, the backend refuses the deletion and returns the error *Cannot delete an organizational unit that still has positions* (Figure 5.24). The same rule protects a unit that still has sub-units. The user closes the message with *Close*, and the unit stays unchanged. In this way, a position can never lose its organizational unit.

5.2.2 Job Offer Creation and Publication

Recruitment starts in the *Job Offers* application of the HR cockpit, exposed by the service binding `ZUI_HCM_RECRUIT`.

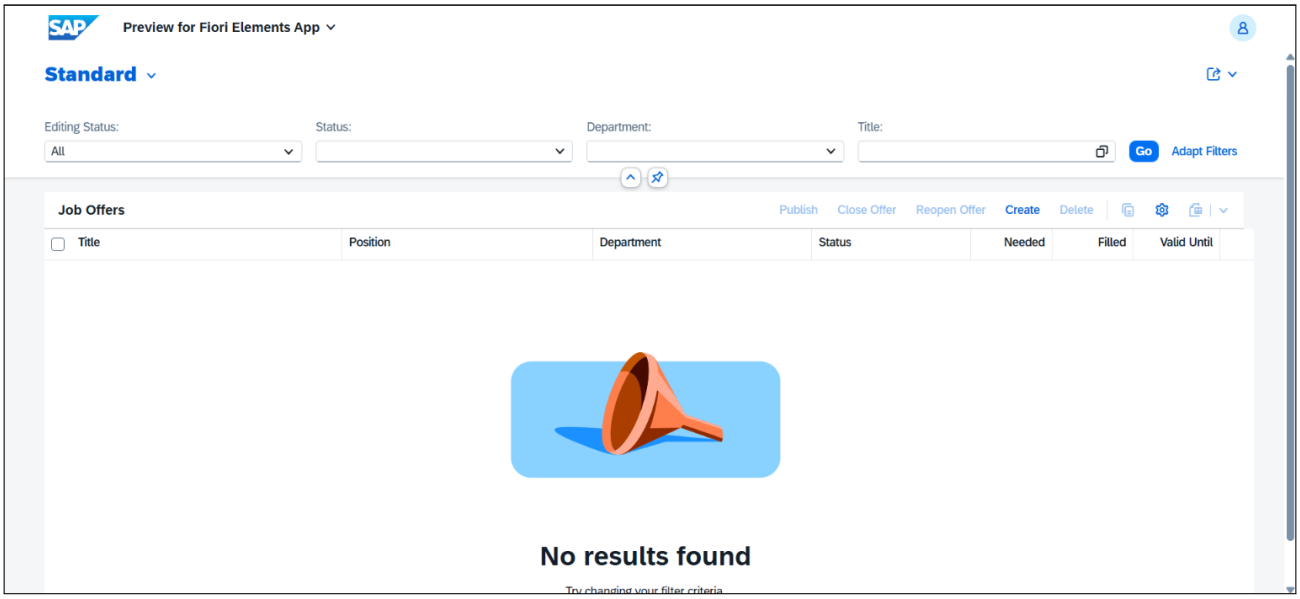


Figure 5.25: Job Offers List Report

The HR administrator opens the Job Offers application, which is empty at the start of the scenario (Figure 5.25). The filter bar searches the offers by editing status, status, department, and title. For each offer, the table shows the title, the position, the department, the status, the number of positions needed and filled, and the validity date. The toolbar groups the actions of the offer life cycle: *Publish*, *Close Offer*, and *Reopen Offer* are enabled only when an offer with a suitable status is selected, *Create* starts a new offer, and *Delete* removes the selected offers.

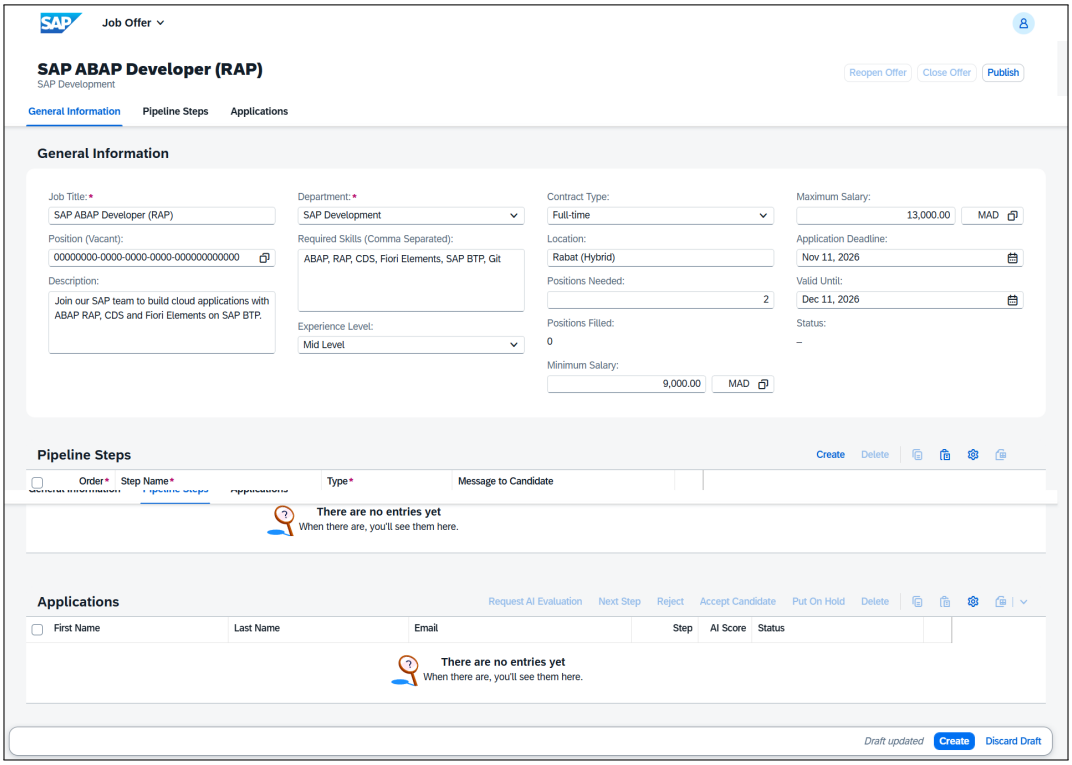


Figure 5.26: Job Offer in Draft Mode

Clicking *Create* opens a new job offer in draft mode (Figure 5.26). The HR administrator fills the *General Information* section: the title *SAP ABAP Developer (RAP)*, the department *SAP Development* chosen in a drop-down list, the description, the required skills separated by commas, the experience level (*Mid Level*), the contract type (*Full-time*), the location (Rabat, hybrid), two positions needed, a salary range from 9,000 to 13,000 MAD, the application deadline, and the validity date. The *Position (Vacant)* field offers a value help limited to vacant positions. *Positions Filled* and *Status* are read-only, because the system manages them. The *Pipeline Steps* and *Applications* sections are still empty.

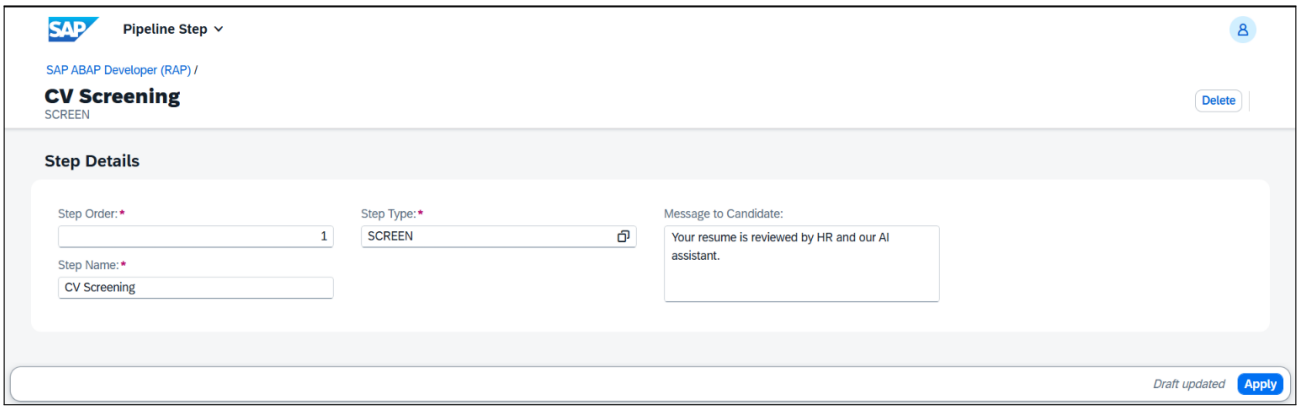


Figure 5.27: Details of a Pipeline Step

In the *Pipeline Steps* section, the HR administrator clicks *Create* to define the recruitment steps (Figure 5.27). Each step has an order, a name, a type chosen with a value help (here *SCREEN* for the CV screening), and a message to the candidate, which the portal displays while the candidate is at this step. The *Apply* button adds the step to the offer, and the breadcrumb *SAP ABAP Developer (RAP)* returns to the offer.

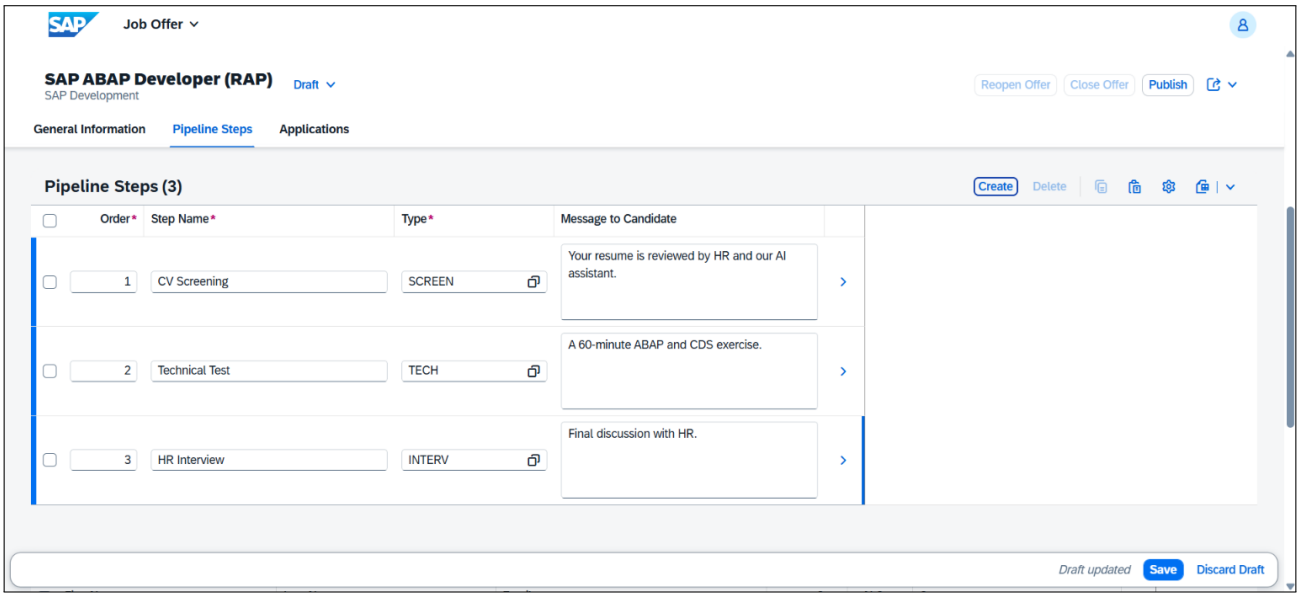


Figure 5.28: Pipeline Steps of the Job Offer

The HR administrator creates three steps: CV Screening (SCREEN), Technical Test (TECH, a 60-minute ABAP and CDS exercise), and HR Interview (INTERV) (Figure 5.28). The steps can still be edited directly in the table, and the arrow of each row opens its details. The user saves the offer with *Save* and then clicks *Publish* in the header. An offer cannot be published without at least one pipeline step. Once published, it appears immediately on the candidate portal.

5.2.3 Candidate Application on the Portal

The candidates use the Talently portal, a React application that reads and writes the same SAP data through the OData services.

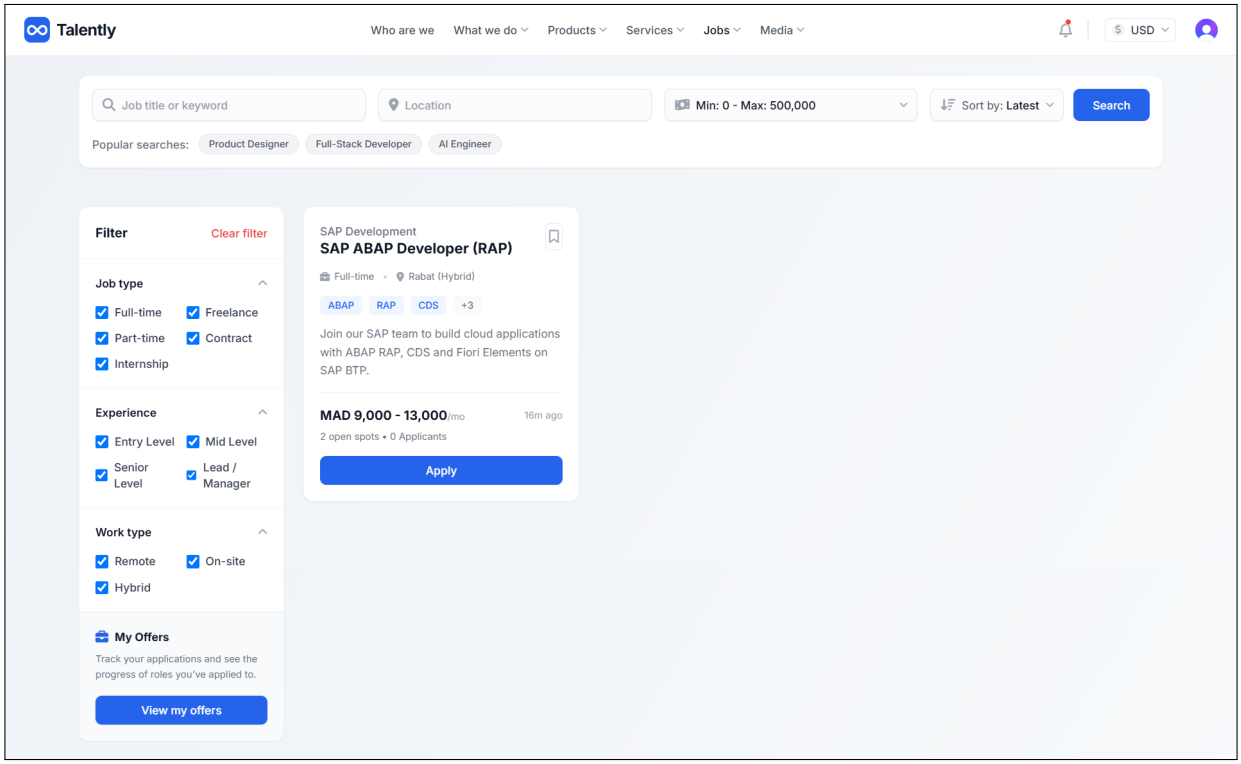


Figure 5.29: Job Board of the Candidate Portal

The published offer appears on the job board of the portal (Figure 5.29). The candidate can search the offers by keyword, location, and salary range, sort them with *Sort by*, or use the popular searches. The *Filter* panel on the left narrows the list by job type, experience level, and work type, and *Clear filter* resets it. Each offer card shows the department, the contract type, the location, the main skills, the salary range, the time since publication, and the number of open spots and applicants. *Apply* opens the offer, the bookmark icon saves it, and *View my offers* lists the applications of the candidate.

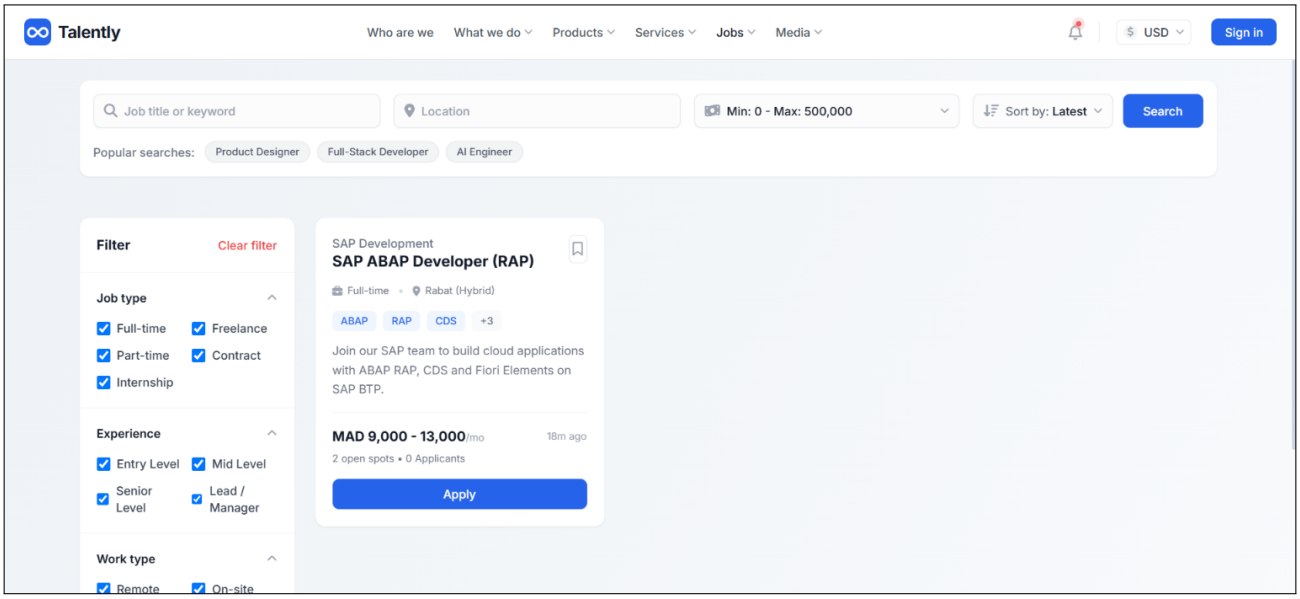


Figure 5.30: Job Board for a Visitor Who Is Not Signed In

A visitor who is not signed in can also browse the job board and read the offers (Figure 5.30). In this case, a *Sign in* button is shown at the top right instead of the user avatar. Applying to an offer requires an account, so that each application is linked to a known candidate who can follow it later.

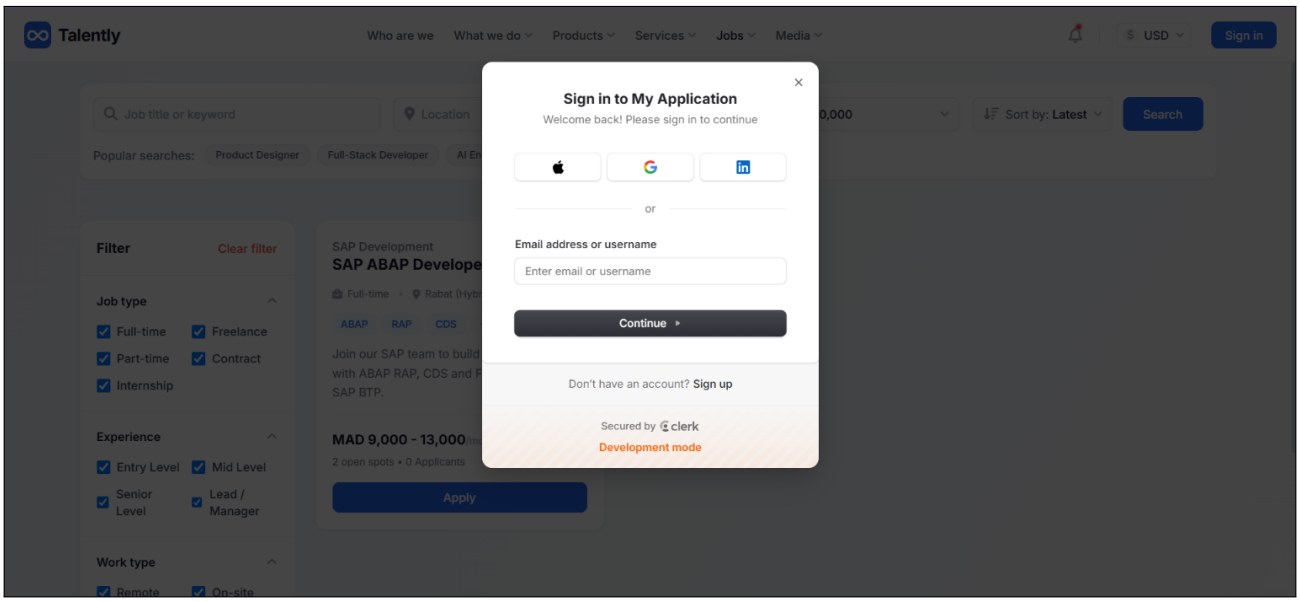


Figure 5.31: Candidate Sign-In with Clerk

Clicking *Sign in* opens the Clerk sign-in window (Figure 5.31). The candidate can sign in with an Apple, Google, or LinkedIn account, or enter an e-mail address or user name and click *Continue*. A new candidate creates an account with the *Sign up* link, and the cross at the top right closes the window. The authentication of the candidates is fully handled by Clerk, separately from the SAP users of the HR team.

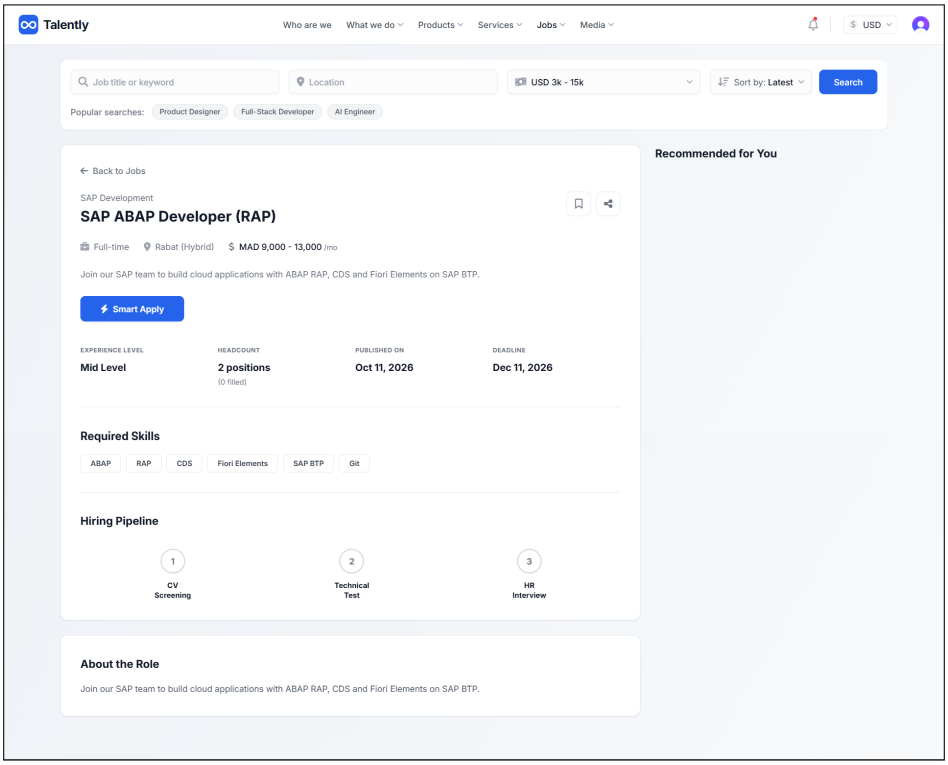


Figure 5.32: Job Details Page

Once signed in, the candidate opens the offer (Figure 5.32). The details page shows the department, the title, the contract type, the location, the salary range, and the description, followed by the experience level, the headcount with the number of filled positions, the publication date, and the deadline. The *Required Skills* section lists the skills entered by HR, and the *Hiring Pipeline* section shows the three steps of the recruitment. The candidate can go back with *Back to Jobs*, bookmark or share the offer with the icons at the top right, and start the application with *Smart Apply*.

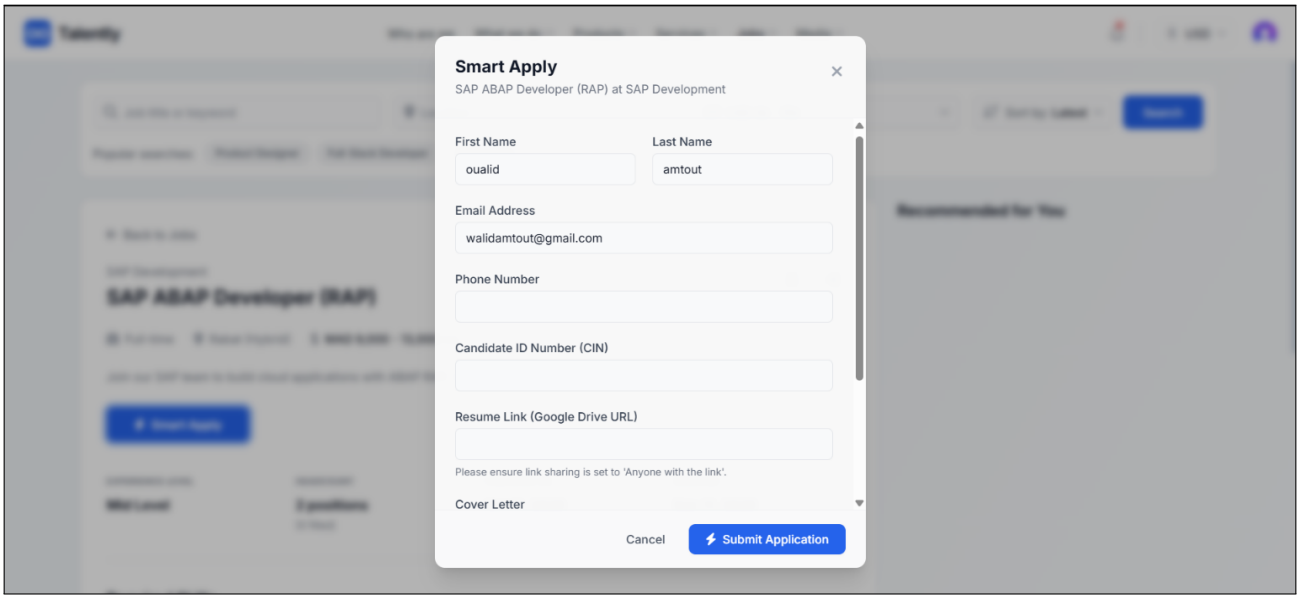


Figure 5.33: Smart Apply Form

Smart Apply opens the application form (Figure 5.33). The first name, the last name, and the e-mail address are pre-filled from the Clerk account and can be changed. The candidate adds a phone number, the national identity card number (CIN), the link to the resume on Google Drive, a cover letter, the notice period, and the salary expectation. The resume must be shared with *Anyone with the link*, so that the AI engine can read it. The candidate sends the application with *Submit Application* or closes the form with *Cancel*.

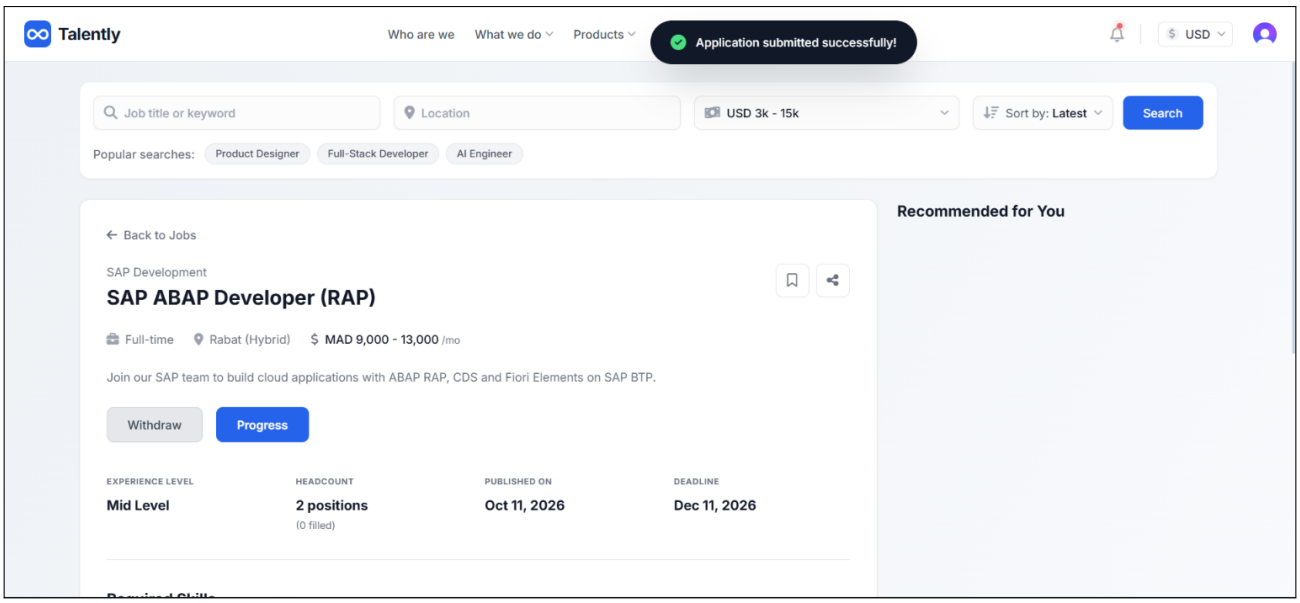


Figure 5.34: Application Confirmation on the Portal

After the submission, the message *Application submitted successfully!* confirms that the application has been created in SAP (Figure 5.34). On the offer page, the *Smart Apply* button is replaced by two buttons: *Withdraw*, which lets the candidate withdraw the application, and *Progress*, which shows the current step of the application in the pipeline. In the background, the portal sends the resume to the AI microservice for evaluation. The second candidate applies to the same offer in the same way with another account.

5.2.4 AI Evaluation and Recruiter Review

Back in the HR cockpit, the HR administrator follows the applications in the object page of the job offer.

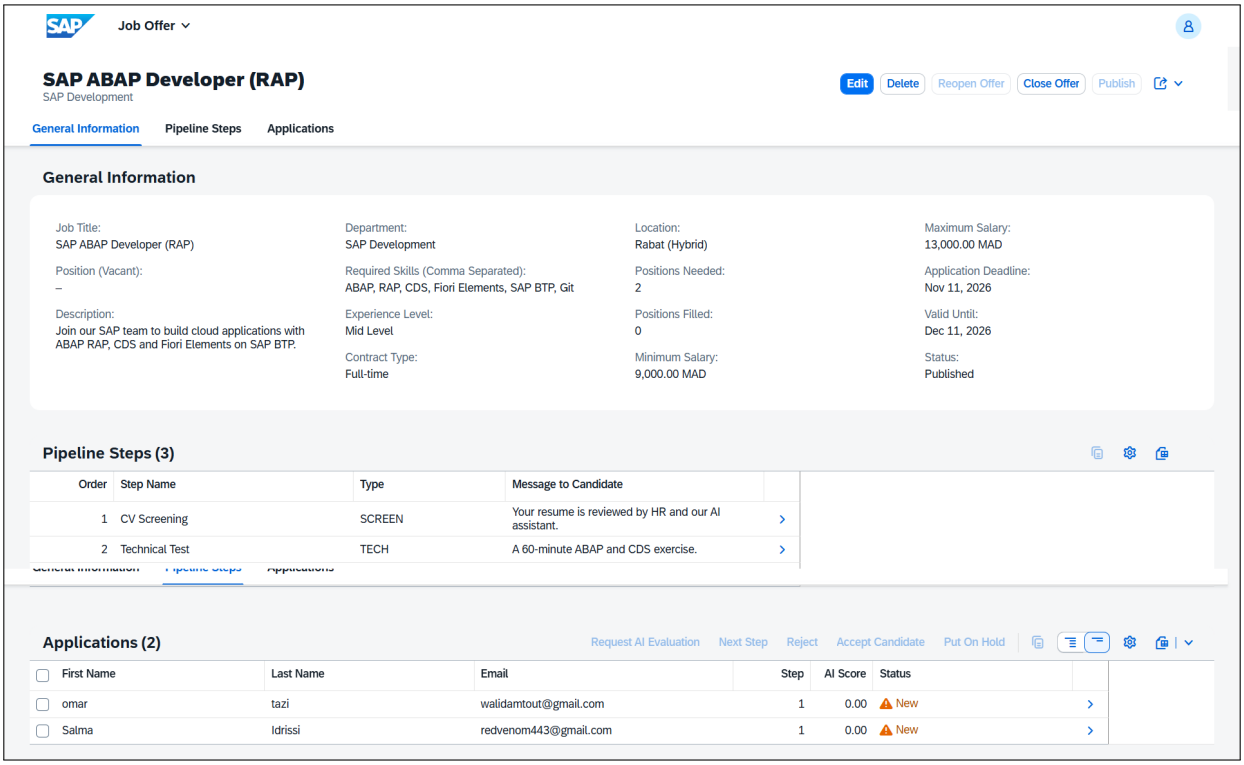


Figure 5.35: Published Job Offer with Its Applications

In the HR cockpit, the job offer now has the status *Published* (Figure 5.35). The header buttons follow the status of the offer: *Publish* and *Reopen Offer* are disabled, while *Close Offer*, *Edit*, and *Delete* are enabled. The *Applications* section lists the two applications received from the portal, with the name and e-mail of each candidate, the current step (1, CV Screening), the AI score, and the status *New*. The AI score stays at 0.00 until the result of the AI evaluation is received.

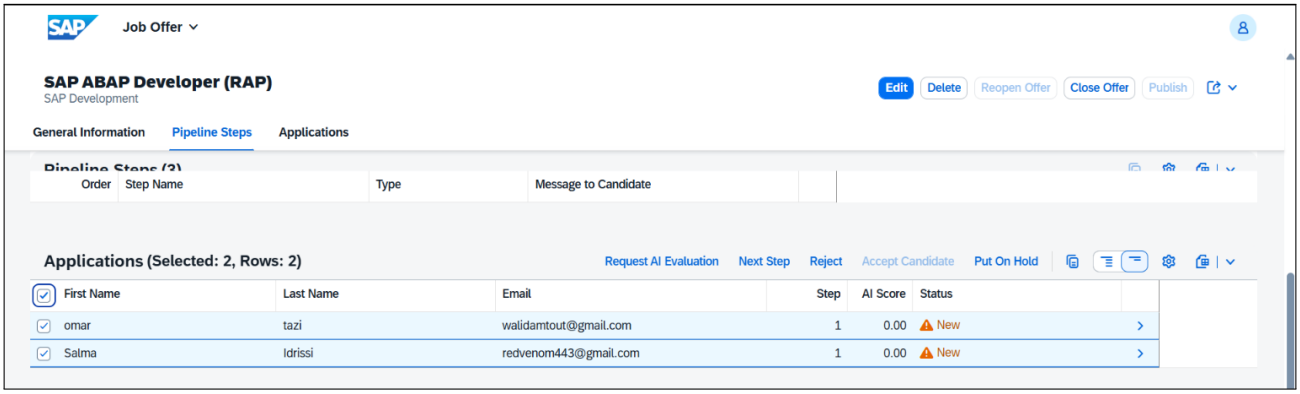


Figure 5.36: Selection of Applications and AI Evaluation Request

The HR administrator can select one or several applications to act on them (Figure 5.36). The toolbar actions are enabled according to the state of the selected applications: *Request AI Evaluation*, *Next Step*, *Reject*, and *Put On Hold* are available, while *Accept Candidate* stays disabled, because the

candidates have not reached the last step of the pipeline. *Request AI Evaluation* asks for a new evaluation of the selected resumes: the recommendation is set to *PENDING* until the AI engine returns its result.

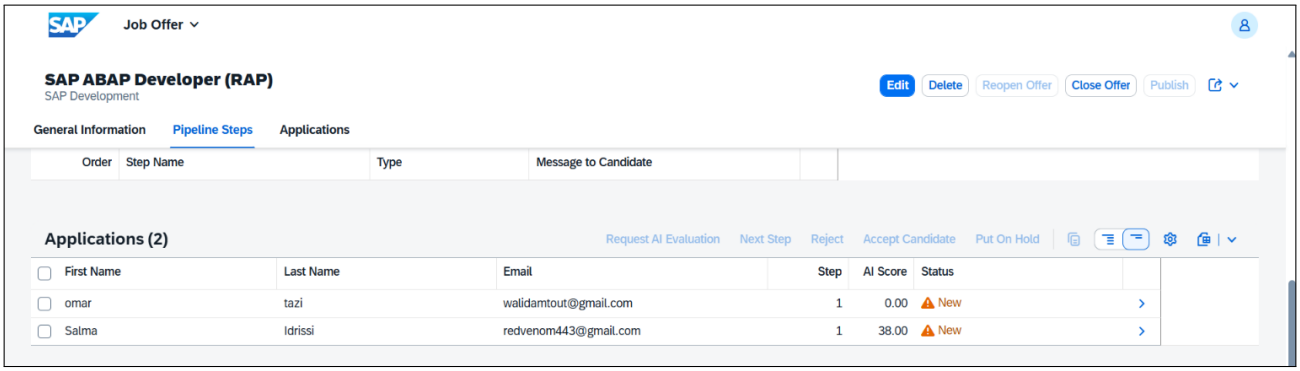


Figure 5.37: Applications with Their AI Scores

When the AI engine has evaluated the resumes, the scores are written back to SAP with the `setAIEvaluation` action and appear in the *AI Score* column (Figure 5.37). Salma Idrissi, who has a first RAP experience, obtains 38 out of 100, while Omar Tazi, who has no IT experience, obtains 0. The engine therefore clearly separates the two profiles, and the HR administrator can sort the table by this column to review the best candidates first.

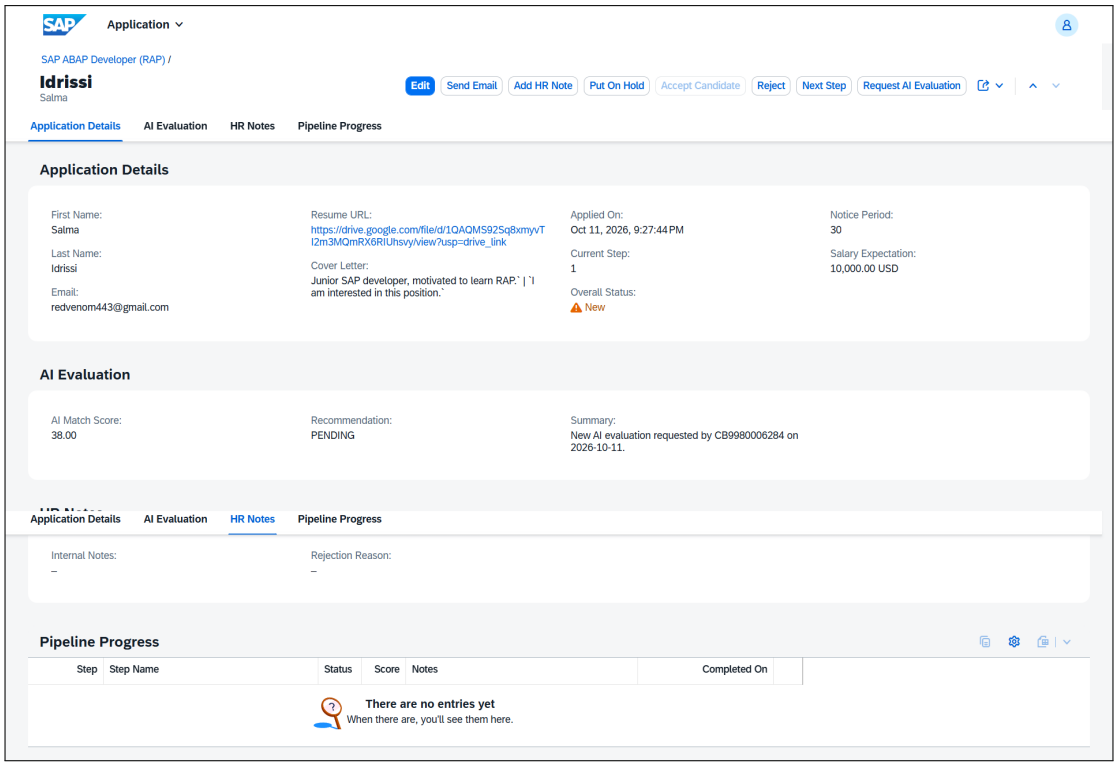


Figure 5.38: Application Details in the HR Cockpit

Clicking a row opens the application (Figure 5.38). *Application Details* shows the candidate data, the resume link, the cover letter, the application date, the current step, the overall status, the notice

period, and the salary expectation. *AI Evaluation* shows the match score (38), the recommendation, and the summary; here a new evaluation has just been requested, so the recommendation is *PENDING* and the summary records who requested it and when. *HR Notes* and *Pipeline Progress* complete the page. From the header, the HR administrator can edit the application, send an e-mail, add an HR note, put it on hold, reject it, move it to the next step, or request an AI evaluation.

5.2.5 Pipeline Progression and Hiring Decision

The selection is done step by step, following the pipeline defined in the job offer.

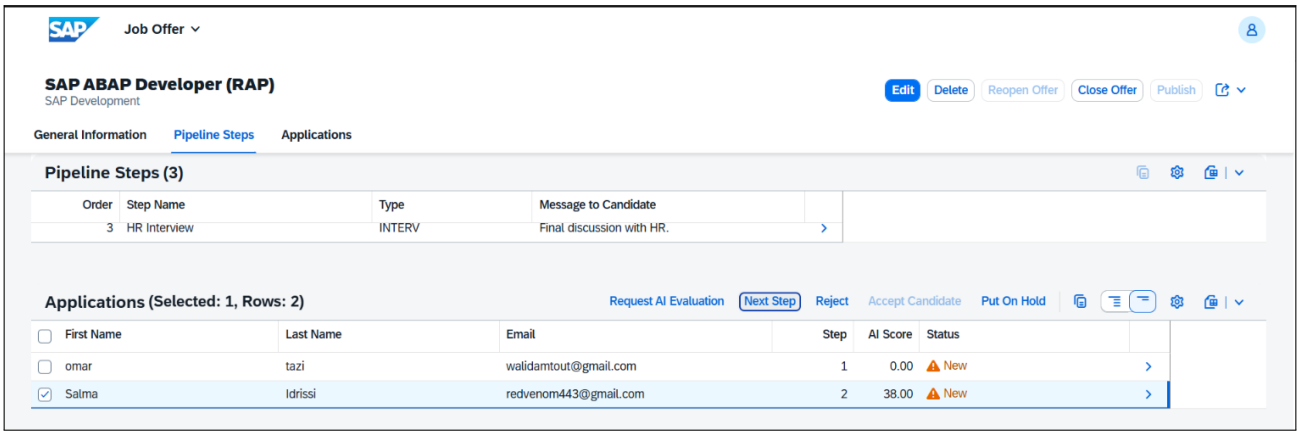


Figure 5.39: Moving a Candidate to the Next Step

After the CV screening, the HR administrator selects Salma Idrissi and clicks *Next Step* (Figure 5.39). The application moves to step 2, the Technical Test, and the completed step is recorded in the *Pipeline Progress* section of the application. The same action is also available in the header of the application page, so the HR administrator can move a candidate forward from the list or from the details.

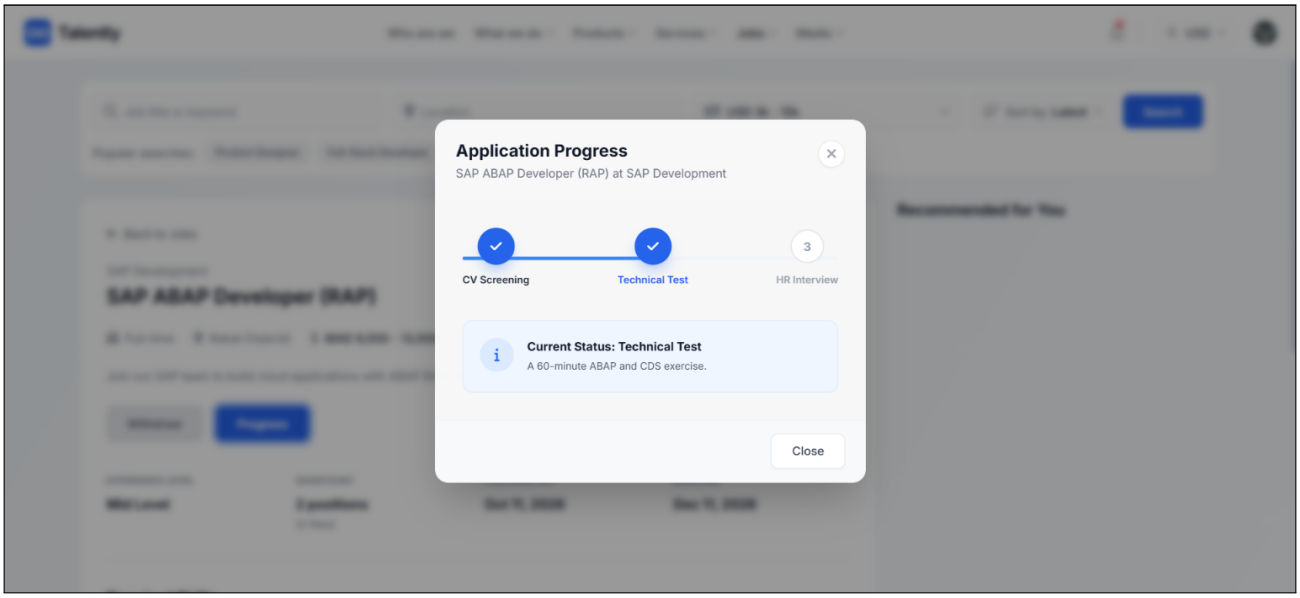


Figure 5.40: Candidate Progress on the Portal

On the portal, the candidate clicks *Progress* on the offer page to follow the application (Figure 5.40). The *Application Progress* window shows the three steps of the pipeline: the steps already reached are marked, the current step, Technical Test, is highlighted, and the box below shows the message defined by HR for this step, *A 60-minute ABAP and CDS exercise*. The candidate closes the window with *Close* or the cross. The progress is read directly from SAP, so the candidate always sees the real state of the application.

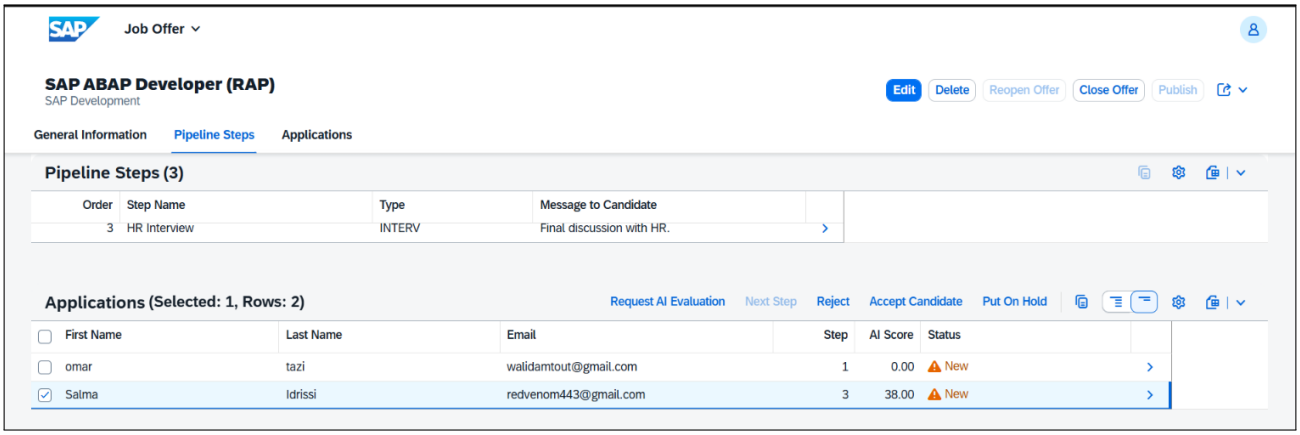


Figure 5.41: Last Step of the Pipeline with Accept Candidate Enabled

The HR administrator moves Salma Idrissi to step 3, the HR Interview, which is the last step of the pipeline (Figure 5.41). The feature control of the backend then changes the available actions: *Next Step* is disabled, because there is no further step, and *Accept Candidate* becomes enabled. In this way, a candidate can only be accepted after going through the whole pipeline.

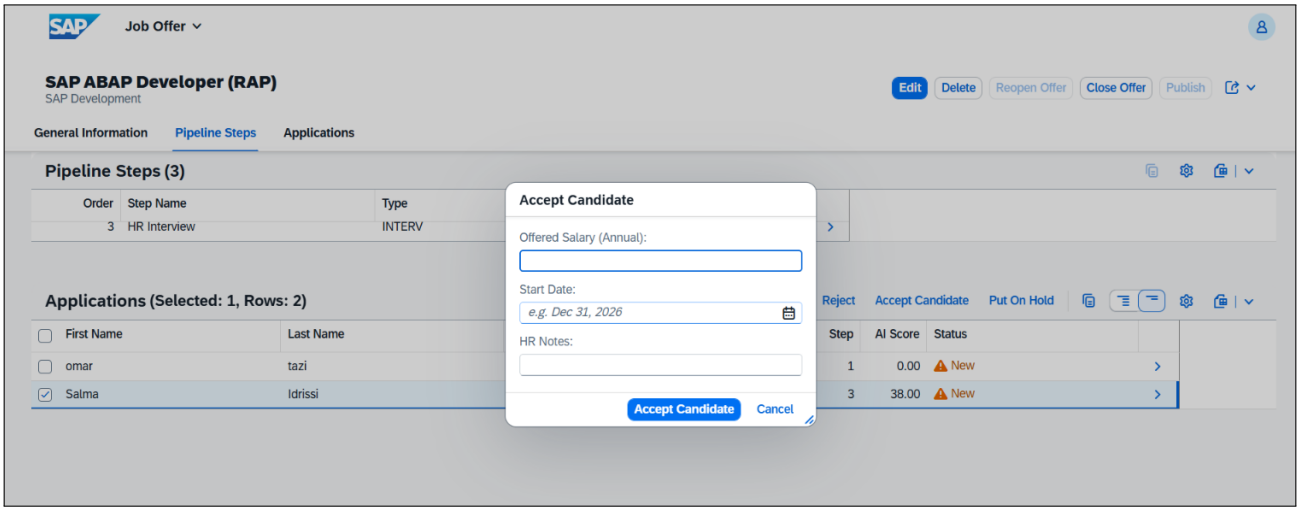


Figure 5.42: Accept Candidate Dialog

Clicking *Accept Candidate* opens a dialog where the HR administrator enters the hiring information: the offered annual salary, the start date chosen with a calendar, and HR notes (Figure 5.42). The user confirms with *Accept Candidate* or cancels with *Cancel*. After the confirmation, the status of the application becomes *Accepted*, the number of filled positions of the offer increases by one, and an onboarding form is created for the candidate. When all the positions of an offer are filled, the offer is closed automatically.

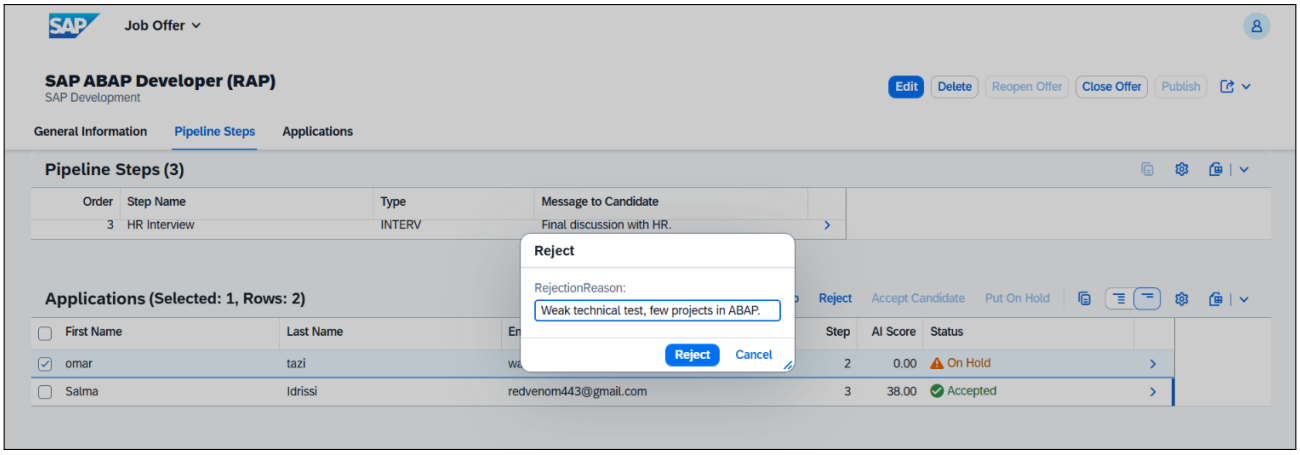


Figure 5.43: Rejection of a Candidate

Omar Tazi, whose profile does not match the offer, was moved to the Technical Test and then put on hold with *Put On Hold*, as the status *On Hold* shows (Figure 5.43). The HR administrator selects this application and clicks *Reject*. A dialog asks for the rejection reason, here *Weak technical test, few projects in ABAP*. The user confirms with *Reject* or cancels with *Cancel*. The status becomes *Rejected*, and the reason is kept in the HR notes of the application. The same table shows Salma Idrissi with the status *Accepted*.

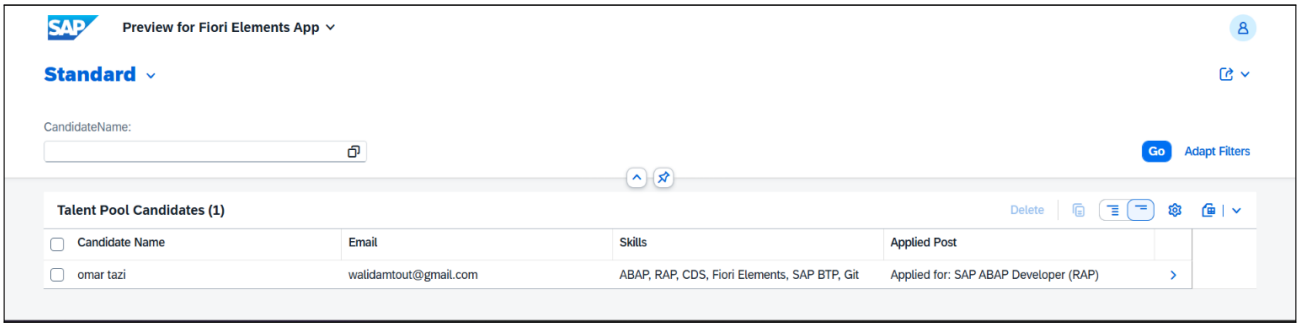


Figure 5.44: Talent Pool

A rejected candidate is not lost: the rejection automatically adds the candidate to the talent pool (Figure 5.44). The *Talent Pool* application lists these candidates with their e-mail, the skills of the post, and the post they applied for, so HR can contact them for a future offer. The HR administrator can search a candidate by name with the filter and *Go*, open an entry with the arrow, or remove the selected entries with *Delete*.

5.2.6 Digital Onboarding

Once accepted, the candidate provides the administrative data needed for the hiring through the portal.

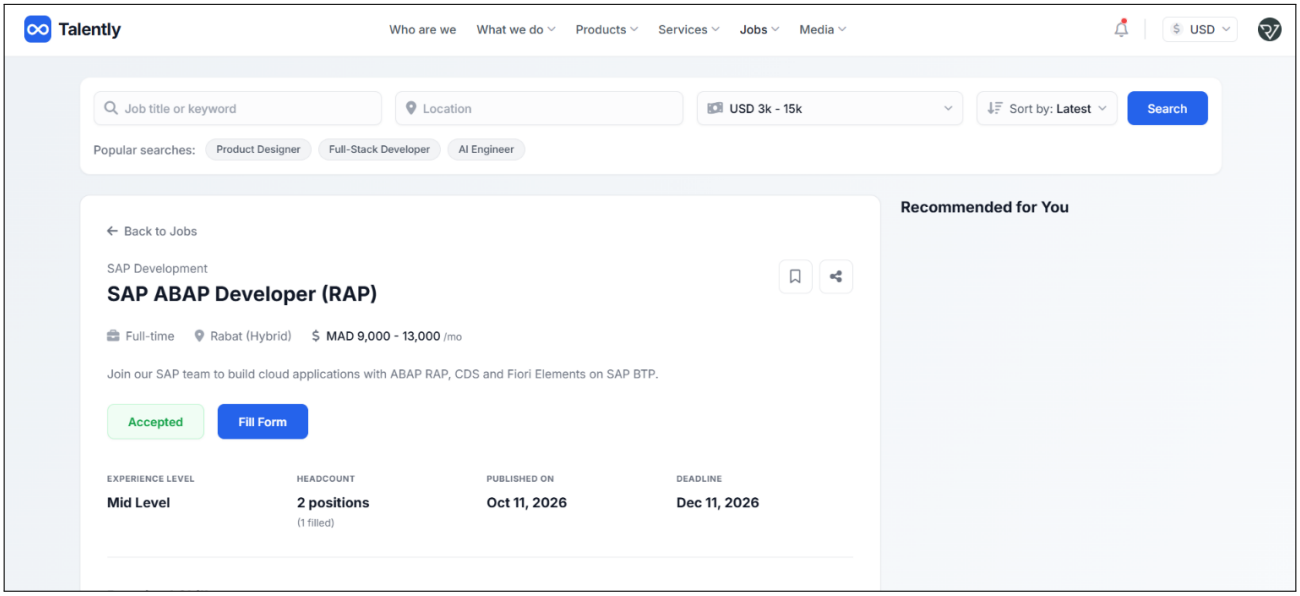


Figure 5.45: Accepted Application on the Portal

On the portal, the accepted candidate sees the status *Accepted* on the offer page, and the headcount now shows one filled position out of two (Figure 5.45). A new button, *Fill Form*, gives access to the onboarding form, which collects the administrative data needed for the hiring.

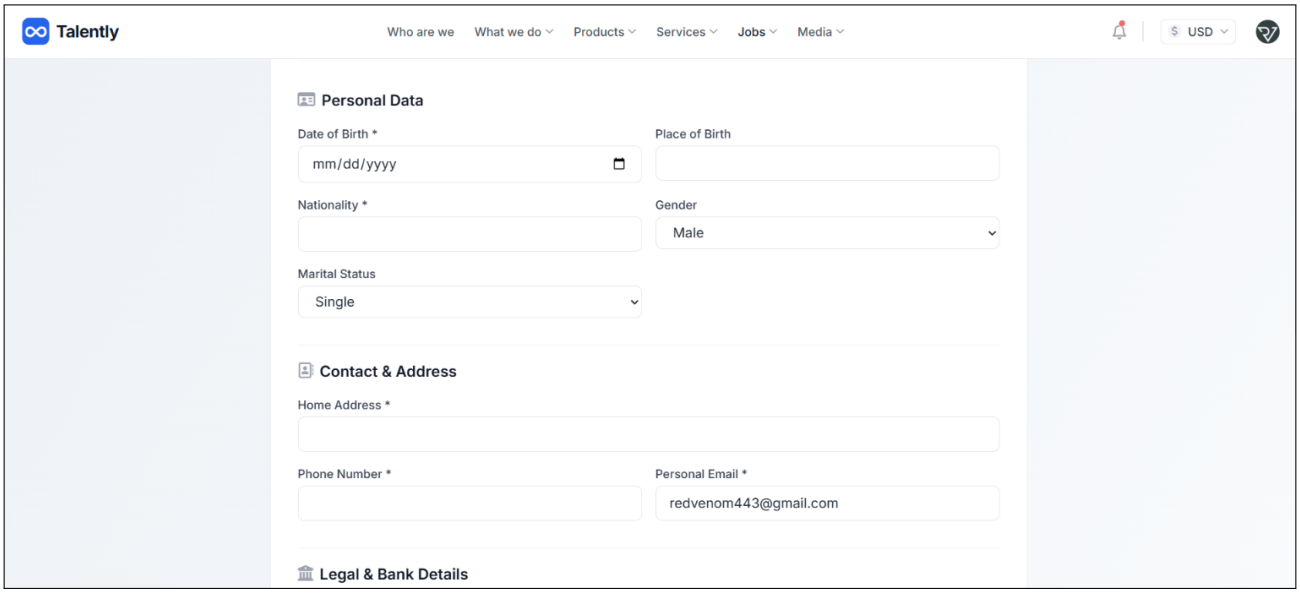


Figure 5.46: Onboarding Form of the Candidate Portal

The *Fill Form* button opens the onboarding form (Figure 5.46). The candidate enters the personal data (date of birth, place of birth, nationality, gender, and marital status) and the contact data (home address, phone number, and personal e-mail, which is pre-filled), then the legal and bank details and the emergency contact in the next sections. The fields marked with an asterisk are mandatory. Once submitted, the form is stored in SAP with the status *Submitted* and waits for the review of HR.

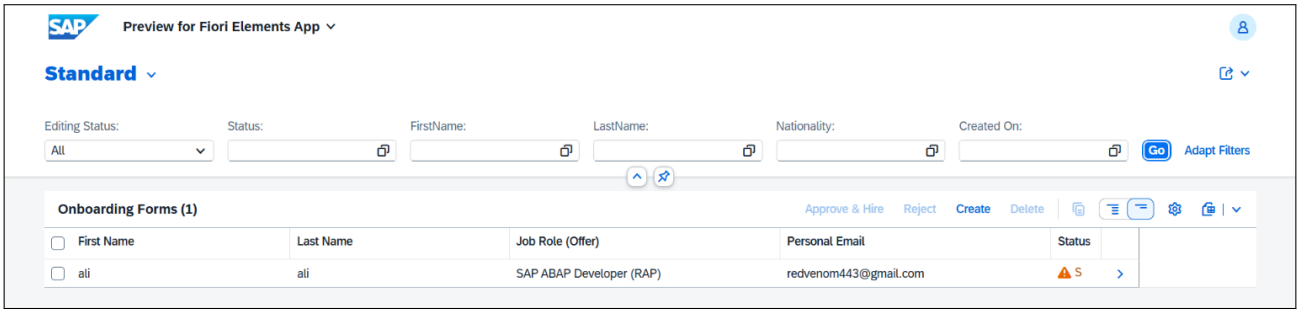


Figure 5.47: Submitted Onboarding Form in the HR Cockpit

The HR administrator opens the *Onboarding* application, where the submitted form appears with the name entered by the candidate, the job role of the offer, the personal e-mail, and the status *S* (Submitted) (Figure 5.47). The forms can be filtered by status, name, nationality, or creation date. After checking the form, the user selects it and clicks *Approve & Hire*. If some information is wrong, *Reject* sends the form back to the candidate with the status *Pending*, for correction.

5.2.7 Employee Creation and Personnel Administration

The approval of the onboarding form creates the employee, who is then managed in the *Personnel Administration* application, exposed by the service binding `ZUI_HCM_PA`.

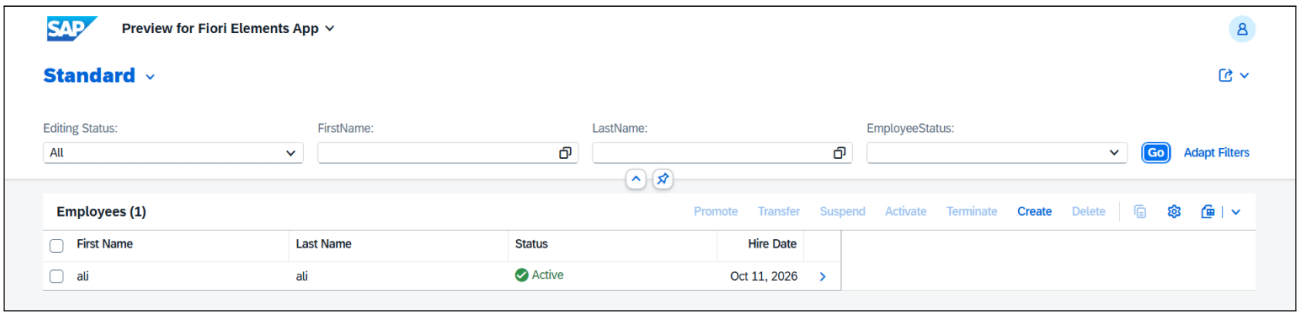


Figure 5.48: Employees List After the Hiring

Approve & Hire creates the employee master record in a single transaction, together with the contact, bank, emergency contact, organizational assignment, and employment history records, without any manual re-entry. The new employee appears in the Personnel Administration application with the status *Active* and the hire date (Figure 5.48). The list can be filtered by first name, last name, and employee status. When an employee is selected, the toolbar offers the life-cycle actions *Promote*, *Transfer*, *Suspend*, *Activate*, and *Terminate*, in addition to *Create* and *Delete*.

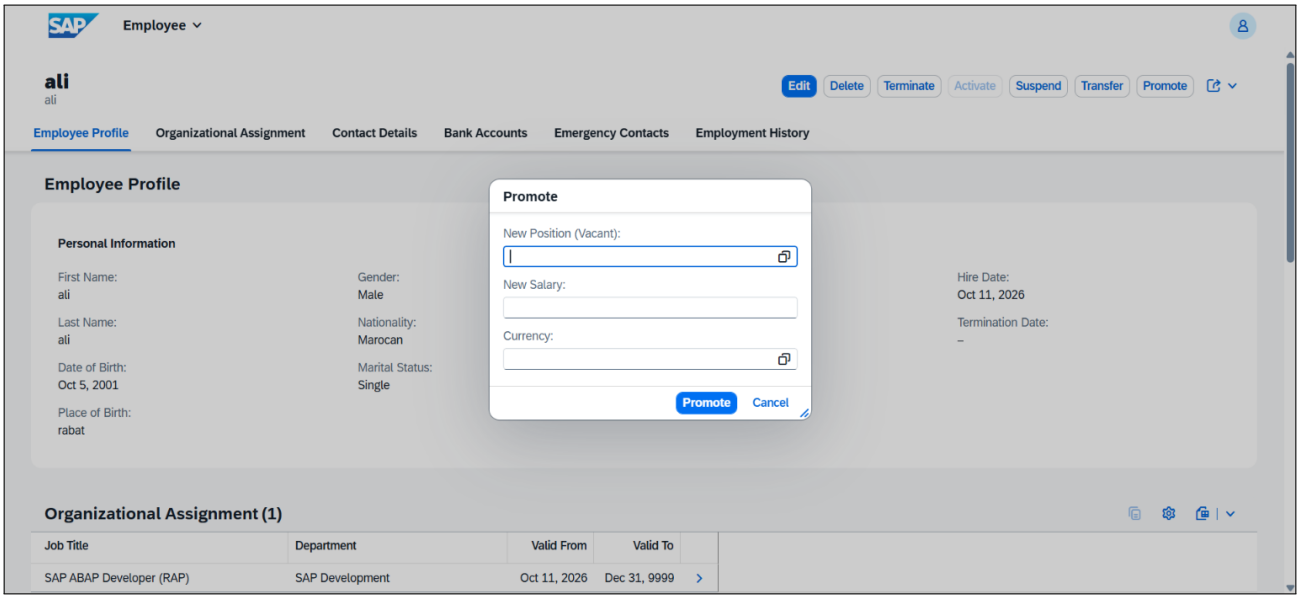


Figure 5.49: Employee Object Page and Promote Dialog

Clicking the employee opens the object page, with six sections: *Employee Profile*, *Organizational Assignment*, *Contact Details*, *Bank Accounts*, *Emergency Contacts*, and *Employment History* (Figure 5.49). The profile contains the data of the onboarding form, and the assignment shows the post *SAP ABAP*

Developer (RAP) in SAP Development from the hire date. The header buttons follow the status of the employee: *Activate* is disabled, because the employee is already active. *Promote* opens a dialog where HR selects a new vacant position with the value help, the new salary, and the currency. The promotion closes the current assignment, opens a new one, and records the change in the employment history.

5.3 Testing

Two levels of tests were carried out: unit tests on the backend and the microservice, and an evaluation of the capability of the AI engine.

5.3.1 Unit Testing

The AI microservice is covered by 19 automated tests written with pytest, which run without network access. The RAP backend is tested with an ABAP Unit test class covering the main business rules. Table 5.1 summarizes the test suites and their results.

Table 5.1: Unit Test Suites and Results

Test suite	**Tests**	**Scope**	**Result**
`test_api_endpoints.py`	7	Root, health, and roles endpoints; error handling of `/evaluate/score`, `/evaluate/full`, and `/evaluate/csv`; missing payload	7 passed
`test_converters.py`	7	TXT and DOCX conversion, Google Drive link parsing, format detection	7 passed
`test_github_enricher.py`	5	Extraction of the GitHub username from URLs and handles	5 passed
ABAP Unit class `ZCL_HCM_RAP_TEST` (RAP behavior)	4	Publication refused without pipeline steps; duplicate application refused; offer closed when all positions are filled; employee created with its child records	4 passed

The 19 tests of the microservice pass in 16.3 seconds (Figure 5.50), and the four ABAP Unit tests pass in 9.4 seconds (Figure 5.51). The ABAP Unit tests run in an isolated test environment with test doubles of the database tables, so they do not change the data of the system.

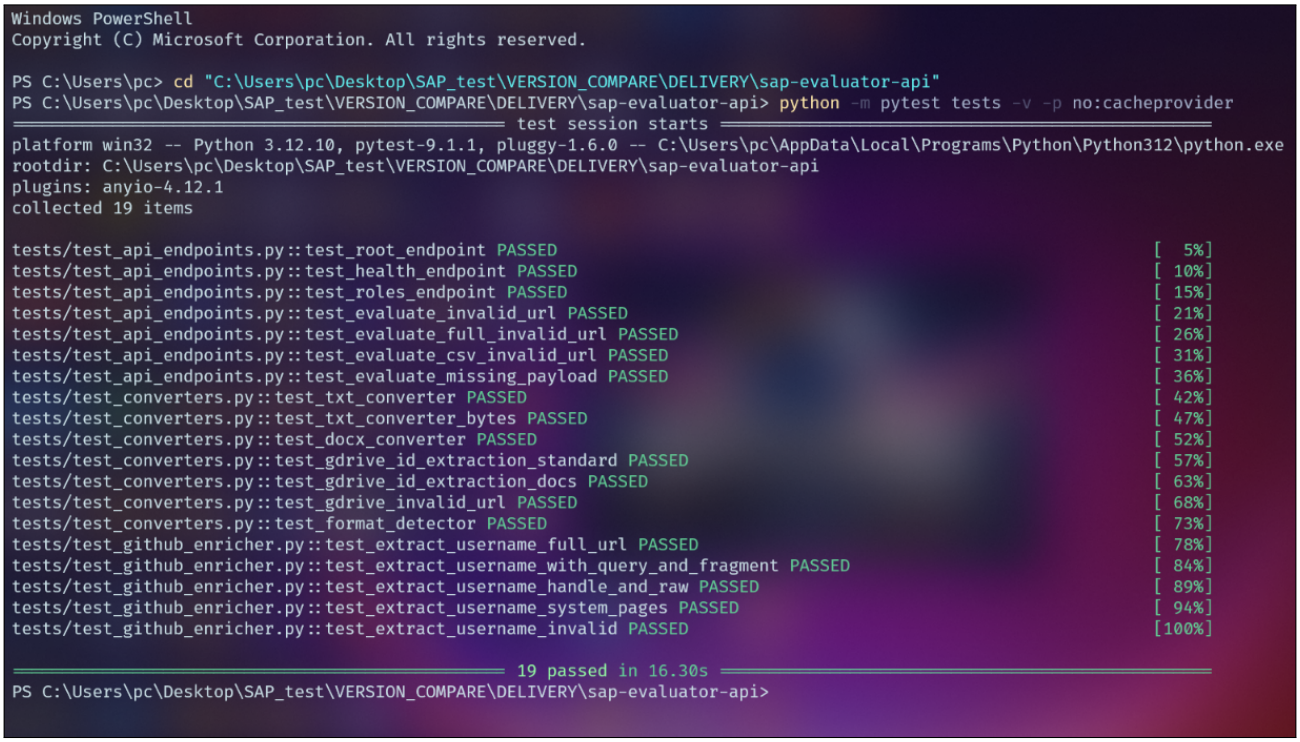


Figure 5.50: Execution of the Microservice Unit Tests (pytest)

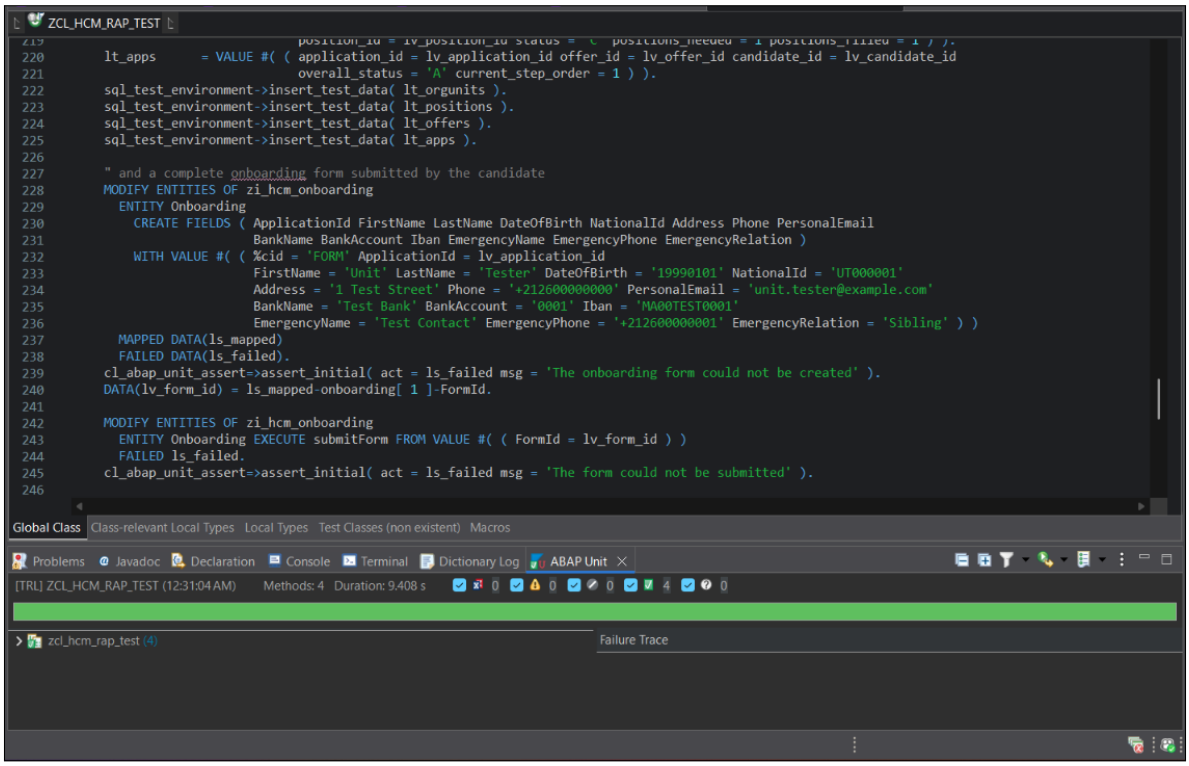


Figure 5.51: Execution of the RAP Unit Tests (ABAP Unit)

5.3.2 AI Engine Capability

The capability of the AI engine was evaluated on five criteria, summarized in Table 5.2. The tests use four fictitious resumes written for this purpose: a strong, a medium, and a weak SAP profile, and a copy of the medium resume with another name and gender, city, and university. They run the real evaluation pipeline of the microservice with the text of the walkthrough job offer. For the consistency, format, and fairness tests, the GitHub data of the candidate is fetched once and reused, so that the only input that changes is the one under test. In total, 70 evaluations were run, for a cost of 0.19 USD.

Table 5.2: Evaluation of the AI Engine

Criterion	**Protocol**	**Result**
Consistency	The medium resume is evaluated 10 times.	Mean 47.5/100, standard deviation 6.0, range 43–63; recommendation DO_NOT_HIRE in 9 runs and CONSIDER in 1.
Format robustness	The medium resume is submitted as PDF (10 runs), DOCX, and TXT (3 runs each).	Means 47.5 (PDF), 44.7 (DOCX), and 49.7 (TXT): largest difference 5.0 points, within the run-to-run spread.
Fairness	The medium resume and its copy with another name, gender, city, and university, 10 runs each.	First version: +5.3 points for the copy (Welch $t = 2.3$). After removing personal data from the scoring input: -4.0 points ($t = -1.4$), not significant.
Ranking of known profiles	The strong, medium, and weak resumes, 3 full runs each.	61.7, 44.7, and 0.0 out of 100: expected order in every run; HIRE or CONSIDER, DO_NOT_HIRE, and DO_NOT_HIRE.
Latency and cost	Full pipeline with GitHub calls (9 runs).	28.0 s per resume on average (12 to 47 s); about 13 800 input and 1 500 output tokens; about 0.003 USD per resume.

The engine ranks the three profiles in the expected order in every run, and the format of the resume has no significant effect. The engine is stable but not fully deterministic. Most of the spread comes from one category: with exactly the same GitHub evidence, the language model scores open source contributions sometimes 10 and sometimes 25 out of 35. One run out of ten therefore reaches the CONSIDER band. The low temperature (0.1), the evidence required for each category, and the computation of the final score in Python limit this variation. A further step would be to use the median of three scoring calls, which would still cost less than 0.01 USD per resume.

The fairness test revealed a real defect in the first version of the engine. The prompt forbade using personal details, but the name, contact details, location, institution, and grades were still sent to the scoring model, and the copy of the resume obtained 5.3 more points on average. The scoring input was

then anonymized, and the test was run again: the difference fell to −4.0 points in the other direction, within the run-to-run spread. The remaining difference comes from the parsing step, which structured the two near-identical resumes slightly differently: the GitHub profile was kept for one of them and dropped for the other. Always adding the GitHub link found by the pipeline to the parsed profile would remove this cause.

5.4 Validation

5.4.1 Functional Validation

Table 5.3 links each group of functional requirements defined in Chapter 3 to the walkthrough figures that demonstrate it.

Table 5.3: Validation of the Functional Requirements

Functional requirement	**Evidence**	**Status**
Organizational structure and positions	Figures 5.13 to 5.24	Validated
Job requisition and pipeline lifecycle	Figures 5.25 to 5.28 and 5.35	Validated
Public candidate portal	Figures 5.29 to 5.34, 5.40, and 5.45	Validated
AI resume analysis	Figures 5.36 to 5.38, Table 5.2	Validated
Candidate evaluation and stage progression	Figures 5.39 to 5.43	Validated
Automated onboarding and employee creation	Figures 5.45 to 5.48	Validated
Employee lifecycle and talent pool	Figures 5.44, 5.48, and 5.49	Validated

5.4.2 Non-Functional Validation

Table 5.4 shows how each non-functional requirement is satisfied and how it was verified.

Table 5.4: Validation of the Non-Functional Requirements

Requirement	Mechanism	Verification
Security	CSRF token on every modifying request; ETag (`If-Match`) optimistic locking; separate identity providers (Clerk for candidates, SAP IAS for HR); instance authorization; technical communication user for the Web API; no personal data sent to the scoring model	Tested on the system: an action sent with an outdated ETag is rejected (HTTP 412), a write without `If-Match` header is rejected (HTTP 428), and a write without CSRF token is rejected (HTTP 403)
Performance	In-memory SAP HANA Cloud database; asynchronous AI evaluation	Measured times in Table 5.5
Maintainability and Clean Core	ABAP for Cloud Development with released APIs only; layered RAP model	All development objects activate without errors in ADT; the four ABAP Unit tests pass; no SAP standard object modified
Availability	Draft handling of organizational units, job offers, onboarding forms, and employees	Every change is kept in the draft (*Draft updated*, Figure 5.26), which stays available until it is saved or discarded
Interoperability	Standard OData V4 and REST services	The same OData V4 services are consumed by the SAP Fiori apps and by the React portal; the AI microservice is called through its REST API

The response times of the main operations were measured on the SAP BTP trial system with a script that calls the OData services with the same session as the portal (Table 5.5). The list was loaded 10 times, an application was submitted 5 times, and the AI evaluation and the hiring were each run 3 times. The write operations used temporary test records, which were deleted after the measurement.

Table 5.5: Measured Response Times

Operation	Average time
Loading the job offers list in the HR cockpit	226 ms (213 to 238 ms)
Submitting an application (`applyToJob`)	306 ms (269 to 380 ms)
AI evaluation of one resume, end to end (offer read, `/evaluate/score` with the Google Drive download, `setAIEvaluation`)	64.7 s (59.9 to 67.8 s)
Hiring: `approveAndCreateEmployee` (employee and its child records)	428 ms (359 to 518 ms)

All SAP operations answer in less than half a second. The AI evaluation takes about one minute end to end, and 95 % of this time (61.8 s on average) is spent in the AI microservice: downloading the resume from Google Drive, seven to eight calls to GPT-4o-mini, and the calls to the GitHub API. This is longer than in Table 5.2, where the resumes were read from local files. Since the portal runs the evaluation in the background, the candidate never waits for it.

Conclusion

This chapter presented the realization of Talently. The development environment combined the SAP cloud ecosystem for the transactional core with open-source web and AI tools for the portal and the evaluation engine. The end-to-end walkthrough demonstrated the complete recruitment process, from the creation of the organization to the automatic creation of the employee master record. Finally, the unit tests, the evaluation of the AI engine, and the validation tables linked every requirement of Chapter 3 to a concrete piece of evidence.

General Conclusion and Perspectives

This internship, carried out at the Cognitus Global Delivery and Support Center in Tangier from June 15 to August 15, 2026, was my first immersion in the SAP ecosystem. Its goal was to bring together two worlds that recruitment usually keeps apart: a modern self-service experience for external candidates and the governance of HR data in the enterprise system. The answer to this challenge is Talently, a recruitment and employee administration platform built as a side-by-side extension on SAP BTP.

The work started with an upskilling phase on SAP Learning (BTP100, S4D400, and S4D437), followed by the analysis of the recruitment process, the specification of the requirements, and the design of the solution. Talently relies on three components. The backend, developed with ABAP RAP on the SAP BTP ABAP Environment, holds all business data and rules in six business objects, from the organizational structure to the employee master record, and exposes them through OData V4 services; the HR cockpit is generated by SAP Fiori Elements from the annotations of the CDS views. The candidate portal, built with React and Clerk, lets candidates find offers, apply, follow their progress, and complete their onboarding form. Finally, a FastAPI microservice analyzes each resume with GPT-4o-mini, enriches it with the candidate's GitHub projects, and scores it against the rubric of the role: the language model judges, but the final score is computed in Python.

The results show that the objectives set in Chapter 2 have been reached. The end-to-end scenario covered the whole process, from the creation of the organizational units to the administration of the new employee, and every group of functional requirements was validated. On the SAP BTP trial system, all SAP operations answer in less than half a second, and the hiring of an accepted candidate creates the employee master record and its child records in a single transaction, in about 0.4 s and without any manual re-entry. The 19 tests of the microservice and the four ABAP Unit tests pass, and the security checks confirmed that write requests with an outdated ETag, without an `If-Match` header, or without a CSRF token are rejected. The AI engine ranked three fictitious profiles of known level in the expected order in every run, for about 0.003 USD per resume. Its evaluation also revealed a fairness defect: personal details were still sent to the scoring model. Once the scoring input was anonymized,

the difference between a resume and its copy with another name, gender, city, and university was no longer significant. This experience showed the value of testing an AI component as rigorously as the rest of the code.

The prototype also has limitations. It runs on an SAP BTP trial account: the portal reaches the OData services through a development proxy that reuses an SAP session, and the AI microservice runs on the development machine. The AI score is stable but not fully deterministic, since the evaluation of the open source contributions varies from one run to another, and the engine was only tested on a small set of fictitious resumes and with a single role rubric. Finally, the candidates must share their resume through a public Google Drive link, which is practical for a prototype but not suited to personal data in production.

These limitations open several perspectives:

- **Production deployment:** replace the development proxy with a server-side integration that calls the Web API `ZAPI_HCM_RECRUIT` through its communication arrangement, and host the portal and the Docker image of the AI microservice in the cloud.
- **Integration with the SAP HR core:** replicate the employees created by Talently to the standard employee master data of SAP S/4HANA or SAP SuccessFactors Employee Central through their released APIs.
- **Reliability and fairness of the AI engine:** use the median of three scoring calls, always add the GitHub profile found by the pipeline to the parsed resume, define rubrics for other roles, and compare the rankings of the engine with those of recruiters on a larger set of anonymized resumes.
- **Data protection:** store the resumes in a secure document service instead of Google Drive, and define the consent, retention, and deletion rules required by the applicable data protection laws, such as the Moroccan law 09-08 and the GDPR.
- **Recruiter experience:** send automatic notifications to the candidates at each pipeline step, and add recruitment indicators such as the time-to-hire and the conversion rate of each step through analytical CDS queries.

On a personal level, this internship was a rich learning experience. I discovered the ERP world and the SAP technologies, from ABAP Cloud to RAP, CDS, OData, and Fiori Elements, and I learned to apply SAP's Clean Core principle to a concrete business case. Building a complete product, from its charter and planning to its tests, taught me to organize my work in sprints, to document my decisions,

and to measure results instead of assuming them. I also learned that integrating artificial intelligence into an enterprise process requires as much care for fairness and transparency as for performance. These skills will be a solid basis for the rest of my engineering studies and for my future career.

Appendix A

Glossary

abapGit: Open-source Git client for ABAP. It serializes the ABAP development objects into files so that they can be versioned on GitHub and pulled into another system.

Action (RAP): Non-standard operation of a business object, such as *publishOffer* or *approveAndCreateEmployee*. It may take parameters (an abstract entity) and return a result.

Behavior definition (BDEF): Source that declares the transactional behavior of a business object: the standard operations, actions, validations, determinations, locking, ETag, authorization, and draft handling.

Behavior pool: ABAP class that implements a behavior definition. Its handler classes run during the interaction phase and its saver classes during the save sequence.

Business object (RAP): Tree of CDS entities linked by compositions (for example, a job offer with its pipeline steps and applications) that is created, locked, and saved as one unit.

CDS view entity: Data model defined with Core Data Services on top of the database tables. Interface views (`ZI_`) define the data model, projection views (`ZC_`) adapt it to one service.

Clean Core: SAP principle that keeps the ERP standard unmodified: extensions are built with released APIs only, on SAP BTP or with ABAP Cloud, so that the core remains upgradable.

Composition: Parent-child relationship in which the child cannot exist without its parent; it is created, locked, and deleted with it.

Determination: Logic that computes or completes data automatically when an instance is created or changed, for example the default status of a new job offer.

Draft: Temporary version of a business object, stored in a draft table, that keeps the changes of a user

until they are activated (saved) or discarded.

ETag (optimistic locking): Version stamp sent with every modifying request (`If-Match` header). A request based on an outdated version is rejected, so that one user cannot overwrite the changes of another one.

Feature control: Mechanism that enables or disables an action or a field for each instance according to its state, for example *Accept Candidate* only at the last pipeline step.

Human-in-the-loop: Design in which an automated system only supports the decision: the AI score helps the recruiter, who keeps the final decision.

List Report and Object Page: Standard SAP Fiori Elements floorplans: a filterable list of instances, and the detail page of one instance with its sections and actions.

Onboarding form: Form completed by an accepted candidate with the personal, legal, bank, and emergency contact data needed to create the employee record.

Rubric: Set of scoring categories with a maximum number of points each (open source, personal projects, production experience, and technical skills) used by the AI engine to score a candidate.

Save sequence: Second phase of a RAP transaction, in which the validations run and the transactional buffer is written to the database in a single commit.

Service binding: Object that publishes a service definition through a protocol, here OData V4, either as a UI service for Fiori or as a Web API.

Side-by-side extension: Application built next to the ERP, here on SAP BTP, instead of inside it, which keeps the ERP standard untouched.

Structured output: Mode of a language model in which the answer must follow a given JSON schema, so that it can be validated and processed by a program.

Talent pool: List of candidates who were not selected for an offer but are kept for future offers.

Transactional buffer: In-memory area in which RAP collects the changes of a request during the interaction phase, before the save sequence.

Validation: Check that runs before data is saved and rejects inconsistent data with a message, for example a job offer without any pipeline step.

Appendix B

Acronyms

The following table presents the list of acronyms and abbreviations used throughout this report:

ABAP	Advanced Business Application Programming
ADT	ABAP Development Tools (Eclipse Plugin)
AI	Artificial Intelligence
AMS	Application Management Services
ANRT	Agence Nationale de Réglementation des Télécommunications
API	Application Programming Interface
ASEDS	Advanced Software Engineering for Digital Services
ATS	Applicant Tracking System
BDEF	Behavior Definition (SAP RAP)
BTP	Business Technology Platform (SAP Cloud PaaS)
CAP	Cloud Application Programming Model
CDS	Core Data Services
CIN	Carte d'Identité Nationale (National Identity Card)
CLM	Contract Lifecycle Management
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CV	Curriculum Vitae (Resume)
DDLX	Data Definition Language Extension (Metadata Extension)

ECC	SAP ERP Central Component
EMEA	Europe, the Middle East, and Africa
ERP	Enterprise Resource Planning
ETag	Entity Tag (HTTP Optimistic Concurrency Control)
FTE	Full-Time Equivalent
GDPR	General Data Protection Regulation
GDSC	Global Delivery and Support Center
GUI	Graphical User Interface
HANA	High-Performance Analytic Appliance
HCM	Human Capital Management
HR	Human Resources
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IAS	Identity Authentication Service
INE	Ingénieur d'État
INPT	Institut National des Postes et Télécommunications
JSON	JavaScript Object Notation
LLM	Large Language Model
MAD	Moroccan Dirham
MRP	Material Requirements Planning
OData	Open Data Protocol (OData V4)
PA	Personnel Administration
PaaS	Platform as a Service
PDF	Portable Document Format
RAP	ABAP RESTful Application Programming Model
REST	Representational State Transfer
S/4HANA	SAP Business Suite 4 SAP HANA
SaaS	Software as a Service
SADL	Service Adaptation Definition Language
SAP	Systeme, Anwendungen und Produkte in der Datenverarbeitung
SDP	Software Development Plan
SQL	Structured Query Language

SRVB	Service Binding
SRVD	Service Definition
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier (16-byte raw format)
UX	User Experience
WBS	Work Breakdown Structure
WRICEF	Workflows, Reports, Interfaces, Conversions, Enhancements, and Forms

Appendix C

Complementary Code Excerpts

This appendix gives four short excerpts of the source code of Talently, chosen to illustrate the design decisions described in Chapters 4 and 5. The complete sources are versioned on GitHub: the ABAP objects through abapGit, and the AI microservice and the React portal as standard repositories.

C.1 Behavior Definition of the Job Offer

The behavior definition declares the managed implementation in strict mode with draft, the optimistic locking with an ETag, the instance-based feature control of the lifecycle actions, and the validations that run before an offer is saved (excerpt of `ZI_HCM_JOBOFFER`).

```
managed implementation in class zcl_bp_i_hcm_joboffer unique;
strict ( 2 );
with draft;

define behavior for ZI_HCM_JobOffer alias JobOffer
persistent table zhcm_joboffer
draft table zhcm_d_joboffer
with additional save
lock master
total etag LastChangedAt
authorization master ( instance )
etag master LocalLastChangedAt
{
  create; update; delete;

  draft action Edit;
```

```
  draft action Activate optimized;
  draft action Discard;
  draft action Resume;
  draft determine action Prepare
  {
    validation validateMandatoryFields;
    validation validateApplicationDeadline;
    validation validatePositionVacant;
    validation validateBeforePublish;
    validation PipeTmpl~validateMandatorySteps;
  }

  field ( readonly, numbering : managed ) OfferId;
  field ( mandatory ) Title, Department;

  action ( features : instance, authorization : instance ) publishOffer result [1] $self;
  action ( features : instance, authorization : instance ) closeOffer result [1] $self;
  action ( features : instance, authorization : instance ) reopenOffer result [1] $self;

  validation validateBeforePublish on save { create; update; }
  validation validateApplicationDeadline on save { create; update; field AppDeadline, ValidUntil; }

  association _Applications { create; with draft; }
  association _Steps { create; with draft; }
}
```

Listing C.1: Behavior definition of the job offer (excerpt)

C.2 Validation Before Publication

A job offer cannot be published without at least one pipeline step. The steps are read from the transactional buffer, so steps that are not saved yet are counted too; the message is attached to the offer and displayed by Fiori Elements.

```
METHOD validateBeforePublish.
  READ ENTITIES OF ZI_HCM_JobOffer IN LOCAL MODE
    ENTITY JobOffer FIELDS ( Status ) WITH CORRESPONDING #( keys ) RESULT DATA(lt_offers).

  " Steps are read from the transactional buffer, so new (unsaved) steps are counted too
```

```
  READ ENTITIES OF ZI_HCM_JobOffer IN LOCAL MODE
    ENTITY JobOffer BY \_Steps FIELDS ( OfferId )
    WITH CORRESPONDING #( keys ) RESULT DATA(lt_steps).

  LOOP AT lt_offers INTO DATA(offer).
    APPEND VALUE #( %tky = offer-%tky %state_area = 'VALIDATE_PUBLISH' ) TO reported-joboffer.

    IF offer-Status = 'P'
       AND NOT line_exists( lt_steps[ OfferId = offer-OfferId %is_draft = offer-%is_draft ] ).
      APPEND VALUE #( %tky = offer-%tky ) TO failed-joboffer.
      APPEND VALUE #( %tky        = offer-%tky
                      %state_area = 'VALIDATE_PUBLISH'
                      %msg        = new_message_with_text(
                        severity = if_abap_behv_message=>severity-error
                        text     = 'Cannot publish: define at least one pipeline step.' ) )
      TO reported-joboffer.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

Listing C.2: Validation `validateBeforePublish` of the job offer

C.3 Score Computation in the AI Microservice

The language model only returns the score and the evidence of each category; the final score and the recommendation are computed in Python, so that they always respect the limits of the rubric.

```
def _compute_score(self, evaluation_data: Any) -> Dict[str, Any]:
    """Compute total score, cap bounds, and recommendation."""
    base_score = 0.0
    max_score = sum(cat.max for cat in self.role.categories)

    if hasattr(evaluation_data, "scores") and evaluation_data.scores:
        scores_dict = evaluation_data.scores.model_dump()
        for cat in self.role.categories:
            cat_score_data = scores_dict.get(cat.key, {})
            awarded = float(cat_score_data.get("score", 0.0))
            # Ensure category score does not exceed its maximum
            base_score += min(awarded, float(cat.max))
```

```
13
14      # Add bonus points (capped at role.bonus_max)
15      bonus_points = 0.0
16      if hasattr(evaluation_data, "bonus_points") and evaluation_data.bonus_points:
17          bonus_points = min(float(evaluation_data.bonus_points.total), float(self.role.bonus_max))
18
19      # Subtract deductions
20      deductions = 0.0
21      if hasattr(evaluation_data, "deductions") and evaluation_data.deductions:
22          deductions = float(evaluation_data.deductions.total)
23
24      final_score = base_score + bonus_points - deductions
25      max_possible = max_score + self.role.bonus_max
26      final_score = max(float(self.role.min_final_score), min(final_score, float(max_possible)))
27
28      percentage = round((final_score / max_score) * 100.0, 1) if max_score > 0 else 0.0
29
30      # Recommendation logic
31      if percentage >= 80.0:
32          recommendation = "STRONG_HIRE"
33      elif percentage >= 65.0:
34          recommendation = "HIRE"
35      elif percentage >= 50.0:
36          recommendation = "CONSIDER"
37      else:
38          recommendation = "DO_NOT_HIRE"
39
40      return {"final_score": round(final_score, 1), "max_score": max_score,
41              "percentage": percentage, "recommendation": recommendation}
```

Listing C.3: Deterministic score computation (`services/scoring_engine.py`)

C.4 Anonymized Scoring Input

After the fairness test of Chapter 5, the text sent to the scoring model no longer contains the name, contact details, or location of the candidate (and, further in the same function, the name and grades of the institution). The skills, experience, projects, and links are kept.

```
1 def convert_json_resume_to_text(resume_data: JSONResume) -> str:
```

```
    """Format JSONResume object into human-readable text block for LLM evaluation."""
    text_parts = []

    if resume_data.basics:
        basics = resume_data.basics
        # Fairness: name, e-mail, phone and location are not sent to the scoring model
        text_parts.append("=== BASIC INFORMATION ===")
        text_parts.append(f"Website: {basics.url or 'Not provided'}")

        if basics.summary:
            text_parts.append(f"Summary: {basics.summary}")

        if basics.profiles:
            text_parts.append("Profiles:")
            for profile in basics.profiles:
                text_parts.append(f"  - {profile.network}: {profile.username or 'N/A'} ({profile.url
})")
```

Listing C.4: Start of the text sent to the scoring model (`data/transform.py`)

Bibliography

[1] SAP SE, *ABAP RESTful Application Programming Model*, SAP Help Portal, ABAP Cloud documentation. https://help.sap.com/docs/abap-cloud/abap-rap/abap-restful-application-programming-model

[2] SAP SE, *SAP BTP, ABAP Environment*, SAP Help Portal. https://help.sap.com/docs/btp/sap-business-technology-platform/abap-environment

[3] SAP SE, *ABAP CDS – View Entities*, ABAP Keyword Documentation. https://help.sap.com/doc/abapdocu_latest_index_htm/latest/en-US/index.htm

[4] SAP SE, *SAP Fiori Elements for OData V4*, SAPUI5 Software Development Kit documentation. https://ui5.sap.com/

[5] SAP SE, *SAP Fiori Design Guidelines*. https://www.sap.com/design-system/fiori-design-web/

[6] SAP SE, SAP Learning courses *Exploring SAP Business Technology Platform* (BTP100), *Learning Basic ABAP Programming* (S4D400), and *Building Transactional Apps with the ABAP RESTful Application Programming Model* (S4D437). https://learning.sap.com/

[7] OASIS, *OData Version 4.01. Part 1: Protocol*, OASIS Standard, 2020. https://docs.oasis-open.org/odata/odata/v4.01/odata-v4.01-part1-protocol.html

[8] abapGit project, *abapGit Documentation*. https://docs.abapgit.org/

[9] S. Ramírez, *FastAPI Documentation*. https://fastapi.tiangolo.com/

[10] OpenAI, *Structured Outputs* and *GPT-4o mini* model documentation, OpenAI API Platform. https://platform.openai.com/docs/guides/structured-outputs

[11] JSON Resume, *JSON Resume Schema*. https://jsonresume.org/schema

[12] Artifex Software, *PyMuPDF Documentation*. https://pymupdf.readthedocs.io/

[13] GitHub, *GitHub REST API Documentation*. https://docs.github.com/en/rest

[14] Meta Open Source, *React Documentation*, and Vite, *Vite Guide*. https://react.dev/, https://vite.dev/

[15] Clerk, *Clerk Documentation*. https://clerk.com/docs

[16] K. Schwaber and J. Sutherland, *The Scrum Guide*, November 2020. https://scrumguides.org/scrum-guide.html

[17] GanttProject, *GanttProject: Free Project Scheduling and Management Software*. https://www.ganttproject.biz/